AlphaEngine

From market signal to governed decision — architecture, mathematics and operations of a quantitative trading desk

Revision A • 24 August 2026

Every figure in this document is either measured and sourced to the artefact that produced it, or explicitly labelled as illustrative. No benchmark is quoted that was not run. Where a capability is absent it is named as absent, with the reason it waits.

0 Contents

1 Executive abstract and system topology

1.1 From market signal to governed decision

A trading desk is a machine for turning information into obligations. Market data arrives as an unbounded, unordered, partially duplicated stream of observations from venues that owe the desk nothing; what leaves is a set of commitments that are legally and financially binding, and that a supervisor, a regulator or an investor may later ask the desk to account for. Everything between those two ends is the system. This document is about the argument that the transformation is **governed**: that at each point where a number is produced the number is reproducible, at each point where a number is missing the absence is named, and at each point where the same arithmetic is written more than once the copies are held to each other by a test rather than by a promise.

Stated as a pipeline, the desk is the composition

$$\text{tape} \longrightarrow \text{consolidated book} \longrightarrow \text{decision} \longrightarrow \text{commitment} \longrightarrow \text{ledger} \longrightarrow \text{view} \tag{1}$$

and the governance claim is a claim about each arrow. The first is a deterministic fold of venue deltas into one price ladder. The second is a pure function of that ladder, the held book, the configured limits and the kill switch, evaluated under a single lock. The third is an append to a store from which the second's inputs can be rebuilt. The fourth is a projection, and every projection in this system is derived, droppable and re-creatable from the ledger. The word doing the work is **pure**: everything that decides is a function, everything that persists is append-only, and everything that is neither is a view that may be thrown away without losing a fact.

The distinguishing property is not speed. The pre-trade decision measures 12.4 µs p50 [latency-bench, native engine, dev Mac] against a 72.7 ms [`tools/colocation_probe.py`, 2026-08-17] round trip to the venue that will actually match the order, so the compute is 0.02 % [`LATENCY_BUDGET.md` §2.3, against the 68 ms one-way hop] of the path and no further optimisation of it changes anything a trader can observe. The system is fast because it is small and deterministic, not the other way round; the speed is a consequence of the discipline, and the discipline is the deliverable.

1.2 What this system is, and what it refuses to be

AlphaEngine is three independently deployable units sharing one append-only audit log: a stateful FastAPI **risk gateway** that owns everything which must not be forked or forgotten, a serverless Next.js **desk workspace** whose ten tabs give eight desk roles one each and the Kalshi research engine two, and which holds no backend credential in the browser bundle, and a stateless **OpenBB research service** that can scale without touching risk state. Around them sit five datastores of which exactly one is authoritative, a Telegram companion inside the gateway process, and a research plane that performs semantic recall over the desk's own output.

What it refuses to be is the more informative half of the description, because each refusal is a capability that was available and was declined for a reason recorded in the tree.

- **It refuses to route a real order.** Orders are paper, capped by the gateway's own gates. The venue adapters read; nothing writes. A system that claimed live routing on the strength of an untested code path would be asserting the one thing it cannot demonstrate.
- **It refuses to run more than one gateway process.** Not "has not yet scaled out" but refuses: a second worker forks the position book and localises the kill switch, and both the container contract test and a POSIX advisory lock on the state directory enforce it.
- **It refuses to coerce a null to zero.** A missing measurement renders as a dash and states why it is missing. `?? 0` on a nullable metric turns "we do not know" into "it is fine" and passes every type check on the way through.
- **It refuses to make an optional dependency load-bearing.** Every credential in the deployment is optional except the gateway's own auth token, and the whole test suite passes with an empty environment.
- **It refuses to depend on a package where the argument is the point.** The workspace ships on Next, React, `lucide-react`, `@supabase/supabase-js` and `oracledb` and nothing else; charts are hand-rolled SVG. A chart library would change what the project claims about itself.
- **It refuses to quote a benchmark it did not run.** Where the tree needs a number it does not have, the number is derived symbolically, marked illustrative, or absent with its reason.

The rule this document is written under

Every figure below is attributed to the artefact that produced it. Where two places in the repository disagree, the generated artefact is quoted and the disagreement is named. Two such disagreements are live as of this revision and both are recorded in situ: the committed test counts against the prose copies in `CLAUDE.md`, and the OpenAPI surface size in `README.md` against the committed contract.

1.3 The argument, in three properties

Determinism wherever a number is produced

Let $\mathcal{O}$ be the space of order requests, $\mathcal{B}$ the space of consolidated books, $\mathcal{P}$ the space of position books, $\mathcal{L}$ the space of configured limit sets and $\mathcal{K}$ the space of control state (kill switch, symbol halts, reduce-only override, duplicate-id set, rate-limit bucket, working-order book). The pre-trade decision is a function

$$D : \mathcal{O} \times \mathcal{B} \times \mathcal{P} \times \mathcal{L} \times \mathcal{K} \times \mathbb{T} \longrightarrow \mathcal{V} \tag{2}$$

where $\mathbb{T}$ is an explicitly injected clock and $\mathcal{V}$ is the verdict space: an accept-or-reject flag together with an ordered vector of check results, one per gate that ran, each carrying the observed quantity, the limit it was compared against and a rendered detail string. D is pure. It reads no wall clock it was not handed, opens no socket, and touches no store; the gateway's `submit()` is D composed with an append to the ledger and a non-blocking enqueue to a mirror, and both of those are downstream of the verdict rather than inputs to it.

Seventeen gates are defined and they run in a fixed order, which is itself part of the cross-language contract:

```
kill_switch -> symbol_halt -> symbol_whitelist -> paper_execution_model
  -> reference_freshness -> duplicate_order -> rate_limit -> price_available
  -> order_sized -> max_order_notional -> symbol_concentration
  -> gross_exposure -> price_band -> working_book -> daily_drawdown
  -> reduce_only -> est_slippage
```

Fifteen of the seventeen are reachable by any single order on the crypto path; the remaining two, `paper_execution_model` and `reference_freshness`, run only for paper-equity orders, which are priced from a vendor quote rather than a book. Order matters because a rejection cites the **first** gate that binds, and a reordering would change which of two simultaneously breached limits is reported as the cause. The order is asserted from both languages against one fixture (`web/tests/gate-parity.test.ts` pins the gate names and their evaluation order as a subsequence of the seventeen).

Determinism is what makes the audit log sufficient. Because D is pure, the tuple recorded at decision time is a complete description of the decision, and replay is not a reconstruction but a re-evaluation. That is why startup rehydration is defensible: a restarted process rebuilds the session baseline from two numbers written at the boundary that produced them, then replays this session's accepted fills, and refuses to start rather than begin on a fabricated baseline if either read is unreadable or ambiguous.

Absence is a typed state, not an exception and not a zero

The second property is the one that shows up in every layer. For a measured quantity of type V, the system does not use $V \cup \{0\}$ and does not use $V \cup \{\text{raise}\}$. It uses

$$V^{\perp} = V \cup \{\perp_r : r \in R\} \tag{3}$$

where R is a small, closed, enumerated set of reasons. The reason is carried to the surface and rendered; the value is never invented. Concretely, the gateway's boundary to Oracle types its failure as one of nine codes (`oracle_not_configured`, `oracle_auth_failed`, `oracle_unreachable`, `oracle_timeout`, `oracle_busy`, `oracle_schema_missing`, `oracle_wallet_invalid`, `oracle_service_unknown`, `oracle_invalid_payload`), and the workspace's boundary to the gateway types its own as one of seven (`gateway_not_configured`, `gateway_misconfigured`, `gateway_auth_failed`, `gateway_unreachable`, `gateway_timeout`, `gateway_rejected`, `gateway_invalid_payload`). Neither ever renders as an empty array.

Three consequences follow, and each is enforced by a suite rather than by convention.

Not configured is not a fault. The public deployment has no Oracle credentials, and that state must never render as red. Distinguishing "no credentials in this deployment" from "configured and did not answer" from "answered, and found nothing" is the entire job of those boundary layers, and collapsing the three into an empty result destroys the only information an operator needs.

A refusal is a result. The research plane's `corpus_silent` verdict means the corpus was searched and had nothing to say; it is a correct answer, not an error, and it is distinct from `unavailable`, which means the corpus could not be reached. "Could not search" and "found nothing" are different facts about the world and the API keeps them apart.

A zero vector is worse than no vector. When the embedding function is unreachable, an ingested document is stored with `embedding_status = 'pending'` and no vector at all. The rejected alternative was a zero vector, which under cosine similarity is equidistant from everything and therefore ranks as plausibly similar to every query — an absence that has disguised itself as a weak positive.

The same discipline governs statistics. Extrema and means over the audit log's own backtest runs exclude NULL metrics **and report how many were excluded**, so a mean over three of nine runs is not silently presented as a mean over nine. A rank that a traversal never reached is `None`, never `0`, because zero would sort ahead of first place.

The same arithmetic, implemented two and three times, held to a contract

Neither runtime in this system can call the other. The Telegram companion is a Python process that cannot execute a browser bundle; the browser cannot reach into the gateway's memory. So the risk arithmetic exists twice by necessity, Python as the reference and TypeScript for the client, and the pre-trade arithmetic exists a third time in C++ for speed. Two implementations of one calculation is two chances to be wrong, and the characteristic failure is the worst kind: a trader reads one value on a phone and a different one on a screen, and neither is flagged as suspect.

The system answers this with two different relations held at two different strengths, and the difference is deliberate.

Bit-exactness, for the pair that runs on the same machine over the same IEEE-754 doubles. For the twenty-scenario fixture S, with $|S| = 20$ counted from `web/tests/fixtures/gate-parity.json`:

$$\forall s \in S: \quad \mathrm{bits}_{64}\big(D_{\mathrm{cpp}}(s)\big) = \mathrm{bits}_{64}\big(D_{\mathrm{py}}(s)\big) \tag{4}$$

Equality of the 64-bit pattern is strictly stronger than `==` on doubles, since it separates -0.0 from $+0.0$ and admits no NaN comparison. Holding it is not free, and the costs are instructive. CPython's `sum()` performs Neumaier compensated summation, so a plain C++ `+=` fold lands one unit in the last place away and fails the fixture; the core reproduces each fold with the **matching** algorithm, compensated where Python compensates and plain where an explicit loop was written. Fused multiply-add contraction moved a result by one ULP even under `-ffp-contract=off` until pinned with `\#pragma STDC FP_CONTRACT OFF`. And Python's `list.sort` is stable and remains stable under `reverse=True`, so two venues quoting the same price fill in feed-iteration order; getting that tie-break backwards moves a blended VWAP by a ULP, which a randomised differential test against the reference router catches because 106 of its 125 multi-venue cases diverge under the reversal.

Tolerance, for the pair that does not share a machine or a fold order. The TypeScript side is an independent re-derivation for a different runtime, and the contract there is

$$|x_{\mathrm{ts}} - x_{\mathrm{py}}| \le \max\big(\tau_{\mathrm{abs}}, \tau_{\mathrm{rel}} \cdot |x_{\mathrm{py}}|\big) \tag{5}$$

with the pair $(\tau_{\mathrm{abs}}, \tau_{\mathrm{rel}})$ chosen per quantity rather than globally: 10^{-9} [`web/tests/parity.test.ts`] for exposure, turnover and win rate; $(10^{-6}, 10^{-9})$ [same] for total return, CAGR, Sharpe, Sortino, maximum drawdown and Calmar; $(10^{-6}, 10^{-6})$ [same] for fees paid; and 10^{-3} [`web/tests/risk-parity.test.ts`] absolute for the Kupiec test statistic, whose inputs are counts.

The subtler contract is what the cross-language gate fixture asserts and what it declines to assert. It pins gate **names** and **order**; it does not pin the observed and limit numbers, because the browser sandbox has no ladder, reads its caps off the book rather than off settings, and has no paper-equity or per-venue routing. Those scenarios are structurally inexpressible on that side, and asserting them anyway would be a looser test wearing a stricter name. The gateway's own numbers are pinned in Python, where they are expressible. Naming the boundary of a proof is part of the proof.

The practical effect is that a one-sided formula change is impossible to land quietly: editing the arithmetic in either language turns the other language's suite red, and the fixture must then be regenerated deliberately rather than loosened.

1.4 Global topology

Three deployment units, one shared ledger, and a strict rule that every arrow into the ledger is a write and every arrow out of it is a view.

Why three units and not one: the gateway needs a long-lived process because it holds sockets and mutable risk state open; the workspace is serverless because a research portal should scale to zero and be redeployed without touching risk state; the OpenBB service is separate so that a slow upstream fetch can never queue behind, or crash beside, the process deciding orders. A fourth tracked application in the tree is experimental, deployed nowhere, and shares no code or data with the three; it is named here so the tree and the documentation agree.

Unit 1: the risk gateway

A single Uvicorn process on an always-on virtual machine in Singapore. Region is load-bearing rather than incidental: egress from the United States receives HTTP 451 from one venue and 403 from the other, so the host's jurisdiction is a functional requirement and not a latency preference. The process owns the venue WebSocket subscriptions, the consolidated book, the paper position book, the resting-order book, the token bucket, the kill switch, the seventeen gates and the audit log. Its HTTP surface is a committed contract of 73 paths carrying 76 operations [`tools/openapi.json`, counted 2026-08-24] whose SHA-256 the workspace's build

verifies before Next.js starts; a mismatch fails the build with the reason inline. That is two separately deployed units asserting their interface against each other before either ships. The Telegram companion rides inside this process rather than forming a fourth unit, with 136 registered commands of which exactly six are gated controls [`modules/telegram/registry.py`, tabulated by `tools/telegram_catalogue.py`, whose `--check` is green on this tree] each control requires membership of an allow-list separate from the read allow-list and empty by default, so the bootstrap fails closed and the bot cannot open a position.

Note how a divergence of this kind is resolved, because this document has watched one close. The tech-stack table in the gateway's own README recorded 38 paths and 39 operations as of 2026-08-17, counted from the route decorators declared in `main.py` at that date, while the committed contract already carried more; the prose had drifted, which is precisely the failure mode that motivated counting from the artefact in the first place. That table has since been rewritten against the artefact and now states its basis, and gives the route-decorator total separately as what it is — a different count of a different thing, since four decorators never reach the schema: the `/ws/book/{symbol}` WebSocket, which OpenAPI does not describe, and three HTML console routes marked `include_in_schema=False`. Two counts on two stated bases is the resolution. One count with no basis was the defect.

Unit 2: the desk workspace

A Next.js application on serverless infrastructure in the same city as the gateway, presenting ten tabs. The first eight are the decision loop itself: Overview, Research, Execution, Portfolio, Risk, Data, Reliability, Developer. The last two are one self-contained research engine over a different venue — Markets for what that venue quotes, Coherence for what the engine proves about it — and they are the reason the count is ten rather than eight. Between them the ten carry 65 rail sections [`web/lib/sections.ts`, counted 2026-08-24; `web/scripts/desk-sweep-plan.mjs` mirrors the same ten tabs by hand and asserts the total, so a section added to one and not the other fails]. Every subtab is URL-addressable, and the navigation rail is pinned to the tour documentation by a test so the two cannot drift apart silently. The browser bundle holds zero backend credentials; the server-side proxy is the only path to the gateway's address and token, and the one credential that **is** published to the browser is an anonymous key whose scope is enforced in Postgres by row-level security rather than by client-side filtering, because a client-side filter is a suggestion and a policy is a boundary.

Unit 3: the OpenBB research service

A second serverless project with no Telegram lifecycle, no trading route, no portfolio state, no database and no writable runtime dependency. Because it holds nothing it scales horizontally without coordination, and because it is a separate deployment its cold starts and upstream timeouts are structurally incapable of reaching the process that decides orders. Its committed suite is 24 tests [`web/lib/test-counts.generated.ts`], which is small because the unit is small; a stateless adapter that grew a large suite would be evidence that it had stopped being stateless.

1.5 The five datastores, and which one is authoritative

Exactly one store is authoritative, and the choice is unusual: it is the embedded one.

Store	What it holds	Status	Why it sits there
DuckDB audit log	orders, risk events, backtest runs, equity snapshots, session rollovers	**Authoritative**	Embedded on purpose: the desk must keep deciding and recording when every network dependency is gone. Append-only by convention enforced in the module; nothing issues UPDATE or DELETE against orders or risk events.
Supabase Postgres	`order_blotter` decision mirror, `desk_risk_limits`, `research_documents` under a 384-dimension pgvector HNSW cosine index	Derived	Durability and reach beyond one volume, plus the research corpus. Never a second decision-maker; the write path cannot block an order.

Data-operations store	data quality findings and escalations, schedule runs, work items	Operational	SQLite on the mounted volume by default, the same Supabase over PostgREST when selected. Selecting Postgres without credentials raises at startup rather than falling back, so a deployment can never report one backend and use another.
Neo4j Aura	community labels and centrality scores over the research corpus	Optional projection	Postgres owns the edges; the graph is MERGEd from that derived state on a sweep. A dual write was the rejected alternative because two systems that must agree drift undetectably.
Oracle Autonomous DB	`VECTOR(384, FLOAT32)` research corpus with an HNSW index, and an in-database terminal-value VaR under geometric Brownian motion	Optional	Demonstrates the arithmetic executing inside the database rather than beside it. Nothing is persisted per call: the simulated paths live in an inline view and vanish when the statement ends.

Redis is a sixth optional dependency and is deliberately not on this list, because it is a broker and not a store: setting `REDIS_URL` switches the job queue from an in-process pool to Celery with the **same task callables** either way, and no fact lives only there.

The authoritative choice deserves its own defence. A managed Postgres would be the conventional answer, and it is the one this system declines. The audit log is the input to rehydration and the arbiter of what the desk believes it did; if it lives across a network, then a network partition does not merely degrade the desk, it removes the desk's memory of its own actions at exactly the moment when that memory matters most. Placing it on a mounted volume beside the process inverts the dependency: the managed store becomes the thing that may be absent, and the embedded store becomes the thing that cannot be. Everything downstream, the mirror, the corpus, the workspace blotter, is a view of decisions the log has already recorded.

The fallback inside that store is itself a lesson the tree records. The audit connection used to catch every exception from `duckdb.connect` and fall back to SQLite at a different path, which is correct for one of the two things that exception can mean and catastrophic for the other. If DuckDB is simply unavailable on this platform, SQLite is exactly the right fallback and nothing is lost but analytical SQL. If instead **another live process already holds this database**, the same fallback opened a private ledger and began writing a divergent history while the health endpoint reported the SQLite backend as though somebody had chosen it. An append-only ledger silently forking in two is the worst outcome this subsystem admits, and the bare exception handler was the reason it was silent. The second case is now a raised conflict, never a fallback.

1.6 One process, no workers, and why the mutable book forces it

The single most consequential architectural constraint in the system is that the gateway runs one process with one worker, and that this is enforced rather than recommended.

The position book, the resting-order book, the token bucket and the kill switch are plain objects on the heap, mutated under one `asyncio.Lock` and read on the sub-millisecond order path. Suppose that constraint is violated and N independent workers serve the same traffic, each holding its own copy of that state. Four failures follow, and they are ordered by how much they cost.

The **kill switch becomes local**. A halt request lands on whichever process served it; the remaining $N-1$ keep accepting. The probability that a single halt request reaches every worker is N^{-1} under uniform load balancing, so the halt is effective with probability $\frac{1}{N}$ and silently ineffective otherwise. This is the failure that matters most, because the halt is the last line of defence and the one an operator most believes in.

The **token bucket multiplies**. Each worker enforces its own rate r, so the system emits at up to Nr against a venue that measured one desk. The bucket exists specifically to keep the desk off an exchange's ban list, and sharding it defeats it exactly.

The **limits become systematically permissive**. If the true gross exposure is G and orders are distributed uniformly across workers, each worker evaluates its concentration, gross-exposure and drawdown gates against an expected G/N. The limits do not drift randomly; they are biased loose by a factor of N, and loose in the direction that admits more risk.

The **audit log hides all three**, through precisely the silent-fallback path described above.

Enforcement is layered, because the three ways to violate the rule are visible at three different times. A contract test reads the committed container definition and fails the build if `--workers` or a pre-forking server appears in it. That catches the committed configuration and cannot catch anything typed at a shell. So a POSIX advisory lock on one file in the state directory is taken when the gateway starts and held for the life of the process, and a second writer on the same state directory refuses to start and says why. And the audit store's conflict detection sits behind both, because an audit log is opened by several things that are not the gateway: the Telegram companion, the job runner, the command-line tools and the tests.

What this explicitly does **not** claim is that the gateway is now multi-process-safe. Nothing here shares a book. The boundary is exactly as true as it was; what changed is the failure mode, from a shadow desk running quietly to a refusal that names itself. Overstating that distinction would be worse than not shipping the lock at all.

The storage half of the boundary is separable from the state half and has been separated: moving the four data-operations tables to Postgres frees them from one filesystem, so a redeploy reads the same rows. It does not make a second gateway possible, and the documentation says so in the same paragraph that describes the migration, because a reader who conflates the two will draw exactly the wrong conclusion about scale-out.

1.7 State synchronisation

Three mechanisms move state between the units, and each is constrained so that it cannot become load-bearing for the order path.

Bounded queues that count what they drop

The decision mirror and the research corpus writer share one discipline. The enqueue operation is non-blocking: it cannot block, cannot raise past its own frame, and on a full queue it **counts the drop** rather than waiting. The queue is bounded, at a default depth of 1000 [`config.py`, `SUPABASE_MIRROR_QUEUE_MAX`]. A single drain task batches and delivers, retrying with capped exponential backoff and then giving up into a counter.

The invariant this maintains is a conservation law, and it is the reason the mechanism is defensible at all. Writing n_{enq} for decisions offered, n_{del} for rows confirmed, n_{drop} for enqueue refusals on a full queue, n_{fail} for deliveries abandoned after the retry budget, and q for current queue depth:

$$n_{\text{enq}} = n_{\text{del}} + n_{\text{drop}} + n_{\text{fail}} + q \tag{6}$$

Every term is published on the health endpoint. Loss is permitted; **unmeasured** loss is not. A mirror that can slow an order down has become load-bearing, and this one is structurally incapable of it, because the only operation the order path performs against it is one that cannot wait.

The research writer reaches the same discipline from the other end and the gap is recorded rather than smoothed over. Its queue always matched the mirror's, but for a period only the queue did: the drain made one delivery attempt and discarded, where the mirror retried three times with backoff. That asymmetry is now closed with the mirror's own attempt count, curve and reason vocabulary, with authentication failures held apart from rejections because an expired service-role key is an operator's problem and a rejected row is a developer's. A document that never lands goes to a bounded in-memory dead-letter book that reports its depth, its recent entries, and what it discarded when full. That is a diagnosis and explicitly not a durable replay queue; replaying a dead letter remains the backfill tool's job, and calling the dead-letter book a replay queue would be claiming durability the memory does not have.

The decision mirror

Every gateway decision streams into `public.order_blotter` with provenance, the measured decision latency, and the full check vector: every gate that ran, out of the seventeen defined. The mapping from gate name to the Postgres enum label is an explicit dictionary asserted against **both** sides, the engine's own emission sites and the committed enum, so a rename on either side becomes a red test rather than a silently dropped mirror row. The mapping is the identity today; the indirection exists for the day it is not.

The realtime tape

The workspace subscribes to new blotter rows through an anonymous client whose scope is one thing: gateway-decided, unowned rows on the public demo desk, enforced by Postgres policy. The tape is deliberately **not** an alternative data path. The ledger stays authoritative, the workspace still polls it, and every number the desk acts on comes from there; what the tape adds is new decisions appearing as they land, which is the one thing a poll genuinely cannot do well. A deployment without the two public variables loses the tape and nothing else.

The same restraint governs the server-sent stream between the workspace and the gateway, whose transport is chapter 6′s subject. The stream carries risk **state** and a sequence number that moves only when the state actually changed; the panels' numbers still come from the larger portfolio payload. Rebuilding the panels on the stream's shape would give one set of figures two sources that can disagree, which is what the reconciliation tests exist to prevent. A signal is not a second copy of the data.

1.8 Degraded operation and disaster recovery

This is the strongest single property of the deployment, and it is worth stating precisely rather than as a slogan. The claim is **not** high availability in the replicated sense; there is one gateway process and it is a single point of failure for order decisions. The claim is that the set of things whose absence stops the desk is very small, that every other absence produces a named, reported state, and that the whole test suite passes with an empty environment.

Availability under this design does not compose multiplicatively. If the desk's function were a series chain over m dependencies, its availability would be $A = \prod_{i=1}^{m} a_i$ and every added integration would reduce it. Here the integrations are not in series with the decision path. Writing F for the decision function and g_i for an optional dependency, the design property is

$$F(x; g_1, \ldots, g_m) = F\big(x; \bot_{r_1}, \ldots, \bot_{r_m}\big) \tag{7}$$

for the verdict component of the output, with the g_i affecting only which reasons are reported alongside it. The decision is invariant to the presence of every optional dependency; only the **reporting** varies, and it varies in a typed, enumerated way.

Absent	What still works	What is lost, and how it says so
Supabase	Everything on the order path. Decisions are made, gated and written to the authoritative ledger; the workspace's blotter reads the gateway.	Mirror writes become no-ops with counted drops. Every research route returns a typed `unavailable`, never an empty list. The browser tape disappears. Data-operations state falls to the SQLite backend if it was not explicitly switched.
Neo4j	Everything, including request-time graph traversal, which runs as a recursive CTE in Postgres and never depended on the graph being up.	The community and centrality reports fall back to the in-process computation and **mark which one answered**. The projection sweep reports a named reason rather than raising.
Oracle	Everything. The Oracle surface is a demonstration of in-database analytics, not a dependency of any decision.	The readiness probe reports "not configured" as a first-class state that must never render as a fault, or "unavailable" if configured and unanswering, since an idle Always Free instance auto-stops and "did not answer" genuinely does not mean "broken".
Model providers	Everything. Retrieval, fusion, re-ranking and grading are all deterministic arithmetic over stored signals.	Generation reports `verdict: refused` with its reason, and the spend bound goes inert by design, since refusing a free query because a paid one would be expensive is an outage rather than a bound.
The re-ranker weights	Retrieval at the caller's requested width, with the fusion order passing through untouched.	`rerank_state` names the absence, and the grader's fifth signal is simply not read rather than being defaulted.

The venue feeds	The gateway, its gates, its ledger and its API. Price-dependent gates refuse on absent price rather than guessing.	A watchdog classifies a venue as down, transitions an alert, and only then may substitute a synthetic book, which is gated behind an explicit environment flag and tags every downstream payload so nothing derived from it can be mistaken for market data.
The gateway itself	The workspace's keyless crypto surface: live order books straight from the venues to the browser, depth, TCA and a sandbox backtest.	Every gateway-backed panel reports which of seven failure codes applies, and the three ways a gateway URL can be wrong are three distinct statements so that a typo is never reported as an outage.
The network entirely	The gateway decides, gates and records. The ledger is on a local volume; rehydration reads it.	Every outbound integration reports its own absence. No decision is blocked and no figure is invented.

Recovery has the same shape as the steady state, which is the point. On restart the gateway reads two numbers written at the session boundary that produced them (what closed sessions banked, and the equity this session opened on), then replays this session's accepted fills from the ledger. Both reads are strict: an unreadable or ambiguous record aborts construction rather than starting the desk on a fabricated baseline. That strictness matters more than it looks, because the naive version of the same fix merely relocated its discontinuity. A process restarted after a session rollover republished an equity lower by the entire banked carry, a step inside one session's equity curve with no order behind it and nothing in the panel able to explain it, and re-anchored the drawdown baseline to the configured starting balance, which after a losing session quietly handed back drawdown budget the desk had already spent. Absence of a rollover record is correctly **not** an error: a first-ever session has genuinely banked nothing.

The projection is disposable by the same logic: if the graph is wrong the response is to drop it and re-project, never to reconcile two writers.

1.9 Units, responsibilities and failure modes

Unit	Owns	Scaling	Dominant failure mode and its containment
Risk gateway (FastAPI, always-on VM)	Venue subscriptions, consolidated book, position and resting books, seventeen gates, kill switch, token bucket, authoritative ledger, Telegram companion, research plane	Vertical only. One process, no workers, enforced by container contract test and a `flock(2)` claim	**Process loss stops order decisions.** Contained by: the ledger surviving on a mounted volume; strict rehydration on restart; a refusal-to-start on a second writer rather than a shadow desk. Not contained by replication, and this is stated rather than implied.
Desk workspace (Next.js, serverless)	Ten tabs — eight desk roles and the Kalshi engine's two, server-side proxy, provider registry with quota budgeting and ranked failover, operator write path behind a token	Horizontal, scale-to-zero. Holds no risk state	**Upstream provider failure or a gateway outage.** Contained by: seven typed gateway failure codes that keep "not configured" apart from "unreachable"; keyless crypto surface that works with no gateway at all; per-provider circuit breaking.
OpenBB service (FastAPI, serverless)	Quotes, bars, company news, fundamentals through pinned fetchers	Horizontal, unconstrained. No state, no database, no writable dependency	**Cold start or slow upstream.** Contained by separation: it is a different deployment, so its latency cannot queue behind or crash beside the decision process. That separation is the entire reason it is a third unit.

Shared ledger (DuckDB on a volume)	Orders, risk events, backtest runs, equity snapshots, session rollovers	Not scaled. Single writer by construction	**Two writers forking the history.** Contained by raising a ledger conflict instead of falling back to a private SQLite file, behind the advisory lock, behind the container contract.

1.10 What is not built, and why each waits

Naming an absence costs nothing and buys the reader calibration on everything else in the document. Six are load-bearing at this scale.

Real order routing is not built. Orders are paper, capped by the gateway's own gates. Live routing would need venue credentials, a fills reconciliation path and an operational risk posture that a case-study deployment cannot honestly claim to have.

Multimodal generation over chart images is not built. The embedding runtime available here exposes a text model and nothing in its inference interface takes an image. The substitution that holds meanwhile is exact where a vision model would be approximate: every figure a model would read off the pixels is a number the desk computed in order to draw the chart, so the chart's meaning is rendered as a sentence and the sentence is embedded. Where a vision path **does** exist it carries its own wall-clock budget, set at 45 s [`modules/research_generate_vision.py`] against the text path's 20 s, from two live calls measured at 20,590 ms and 29,924 ms [same]. A ceiling set at the slowest observed sample aborts roughly half the population by construction; 45 s is one and a half times the slower measurement, and 30 s was rejected for sitting exactly on the data.

Row-level security on the research corpus is bypassed. The gateway reads with a service-role key and the writer sets no user identity, so the scope is per-desk rather than per-user. What landed instead is an optional desk predicate applied inside the candidate selection **before** either ranking is taken, so a scoped rank is a rank among rows the caller was allowed to see rather than a position among everybody. One shared gateway token means there is no per-user identity to key on yet, and that is the blocker, not effort.

Live emission of session execution summaries is not built. The renderer, its figures and its document are built and tested and a backfill tool calls them, but nothing emits one in process, because that needs a hook at the session-rollover site. On a running desk the summaries appear when the backfill is run and not before.

The re-ranker's weights do not run in continuous integration. The suite is network-free by construction and the model would have to be downloaded, so what the pipeline proves is the wiring and the arithmetic around the model, exercised through a fake cross-encoder at the import seam. It is not proof of the model. The model's cost, measured directly, is what sets its concurrency limit: twenty pairs of roughly 200-character rows took 197 ms of wall clock and 1,776 ms of CPU [`modules/research_rerank.py`, 18-core arm64 laptop] across roughly nine cores, and twenty pairs at the 2,000-character truncation ceiling took about 1.5 s [same]. Running two at once bought 1.30 to 1.37 times the throughput for 1.46 to 1.54 times the latency on every request [`modules/research_stages.py`], and four at once reached only 1.52 times [same]. Since this event loop also serves pre-trade risk, whose budget is microseconds, the bulkhead is a semaphore of one. Research may wait; that was always the premise, and this is the version of the number that acts on it rather than asserting it.

Multi-process scale-out is not built, and at this architecture it is not a roadmap item. Sharing a position book across processes requires either a shared mutable store on the order path, which reintroduces the network into the sub-millisecond section, or a partition of the book, which makes every portfolio-level limit unenforceable. Both are real designs; both are a different system.

1.11 How the rest of this document is organised

The chapters that follow take these claims into depth from the direction of the people who use them: the researcher and portfolio manager on whether a strategy is evidence or a search artefact, the risk manager and trader on whether the limits hold and what an order will cost, the data engineer, site-reliability engineer and quantitative developer on trust in the data and safe change, then the mathematics in full, then the infrastructure and the telemetry that measures it.

Two artefacts in the tree carry the numbers this chapter has summarised and should be read against it: the latency budget, which defends every timing figure with its method and machine and refuses to merge two machines into one flattering number, and the committed test counts, generated because the prose

copies had drifted three times before the generator existed. As of this revision those counts read 2,998 gateway tests, 2,996 passed and two skipped [`web/lib/test-counts.generated.ts`, generated 2026-08-24 with `RERANK_TEST_MODEL_PATH` blanked — the CI shape, and the shape to state whenever this line is quoted], 4,487 web tests across 984 suites [same] and 24 service tests [same]. The two gateway skips are the two deliberate opt-ins, and each names what it did not exercise: the Postgres data-operations backend reporting no credentials in a network-free environment, and the real cross-encoder re-ranker reporting that no weights were offered. Both are the expected state and say so with a reason rather than passing silently. Seeding the re-ranker weights moves the same tree by eight passes gained and one skip lost [`tests/test_research_rerank_real.py`, which calls `pytest.skip(..., allow_module_level=True)`, so its eight tests are not collected at all and the file contributes exactly one skip], which is why a gateway figure quoted without its shape is not a measurement.

2 The quantitative researcher and the portfolio manager

Two roles share one pipeline and disagree about what a number means. The researcher produces a candidate: a rule, a pair of parameters, and a claim that the pair was not chosen by luck. The portfolio manager decides what fraction of a finite book that claim is worth, and then explains at the close where the day's money came from. The first half of this chapter is the arithmetic that prices a search; the second spreads risk and decomposes a session. Both are held to the same discipline: a figure nobody measured is not written down, and a figure that could not be measured is reported as absent with its reason rather than rounded to zero.

The two implementations are pinned against each other throughout. The sweep engine exists twice, in `Part2_Infrastructure/modules/backtester/` and in `Part2_Infrastructure/web/lib/`, and the fixture `web/tests/fixtures/parity.json` holds 48 recorded cases covering all 46 strategies over 1,200 bars at four parameter combinations each [web/tests/fixtures/parity.json]. The risk and allocation arithmetic exists twice as well, pinned by `web/tests/fixtures/risk-parity.json`, and the pre-trade gate by twenty bit-exact scenarios [web/tests/fixtures/gate-parity.json] in `web/tests/fixtures/gate-parity.json`. Two independent implementations that agree to the last float are the only cheap evidence that either is what its documentation says it is.

2.1 Part A: the quantitative researcher

The signal surface as implemented

The research surface holds 46 strategies [web/lib/types/strategies.ts], grouped into seven families: trend, breakout, mean reversion, momentum, volume, volatility and one called fitted. Every one of them takes exactly two parameters. That is a selection criterion rather than a coincidence, and the criterion is written at the declaration site: a model needing a third axis, Ichimoku being the named example, is held back until the request carries named parameters, because folding a third value into one of the two existing ones produces a control that lies about its units. Fractional axes were the first relaxation of that rule; named axes are recorded as the next one, and are **not built**.

The fitted family contains one member, `linreg_forecast`, and it differs in kind rather than in method: every other strategy applies a fixed rule the researcher chose, while this one estimates its coefficients from the data inside the backtest. Its second parameter is an entry threshold expressed as a multiple of the fit's own residual standard error, so that zero means "any positive forecast" and one means "the forecast must beat the noise the fit could not explain".

A strategy emits a long state $\ell_t \in \{0,1\}$. The traded position is built from the *transitions* of that state, not from the state itself:

$$p_t = \begin{cases} 1 & \text{if } \ell_t \wedge \neg \ell_{t-1} \\ f & \text{if } \neg \ell_t \wedge \ell_{t-1} \\ p_{t-1} & \text{otherwise} \end{cases} \qquad f = \begin{cases} 0 & \text{long only} \\ -1 & \text{long/short} \end{cases} \tag{8}$$

Reading the comparison directly instead would open a short at the instant the warm-up window ends, on a signal that never fired. The distinction costs three lines and is the difference between a position somebody asked for and one the indicator's initial conditions produced.

Signals form on bar t and execute on bar $t+1$. With x_t the instrument's simple return, c the per-unit-turnover cost and h_t the per-bar holding cost, the strategy's return and equity are

$$r_t = p_{t-1} x_t - |p_{t-1} - p_{t-2}| c - h_t, \quad E_t = \prod_{s \le t} (1 + r_s) \tag{9}$$

Returns compound on equity, which is constant-fraction sizing at one hundred per cent. The lag is the whole of the look-ahead protection in the accounting, and it is applied identically in both engines. Exposure is measured on the lagged position for the same reason: the bar a signal appears on is not a bar the book was in the market.

The cost model is a flat leg plus three optional frictions, all defaulting to zero so that an unconfigured run evaluates the identical expression the parity fixture pins:

$$c = \frac{f_{\text{bps}} + s_{\text{bps}}}{10^4} + k\sqrt{\min(1, Q/\ \text{ADV})} \tag{10}$$

The square-root impact law is the standard concave form, so doubling order size costs about $\sqrt{2}$ rather than two. It is a model and the interface says so, because a slippage figure a researcher chose is an assumption they are making rather than a fact the backtest discovered. Average daily volume is computed once over the whole series and reused for every combination and every walk-forward slice: recomputing it per slice would make an order's modelled impact depend on which fold it landed in, so two identical trades would be charged differently for a reason having nothing to do with the trade.

The sweep and its combination budget

A sweep evaluates a grid. For the strategies whose two parameters are both lookback periods the grid is the triangle

$$G = \{(f, s) \in F \times S : f < s\} \tag{11}$$

with F and S generated by an inclusive float-safe axis constructed by index multiplication rather than repeated addition, because at step 0.25 repeated addition drifts to 2.7499999999999996 and shows the reader a different number from the control they moved.

The triangular filter is right when both axes are periods and nonsense otherwise. Fifteen strategies declare a free second axis, where the second parameter is a level rather than a lookback: a band width in standard deviations, an oscillator threshold, a $\%B$ position, an ulcer index. Applying $f < s$ to a 20-bar mean against a 2.0σ band fails $20 < 2.0$, discards every combination, and reports a strategy that silently took no trades. One strategy, `linreg_forecast`, declares a free *first* axis as well: the default fast sweep of 5 to 40 bars is a sensible moving-average period and an unusable training window for a four-parameter regression.

At the shipped defaults the budget is small and worth stating exactly, because it is the N that later deflates the winner's Sharpe.

Quantity	Definition	Value
Fast axis	5 to 40 step 5	8 values [web/lib/types/sweep.ts]
Slow axis	20 to 200 step 20	10 values [web/lib/types/sweep.ts]
Grid $\lvert G\rvert$	the $f < s$ triangle of the two axes	74 combinations [derived from `paramGrid`]
Folds k	request default, clamped to 2..10	4 [web/lib/types/sweep.ts]
Bars N	request default at the 4h interval	2,000 [web/lib/types/sweep.ts]
Bars per year	`BARS_PER_YEAR["4h"]`	2,190 [web/lib/types/sweep.ts]

The grid is hard-capped at 400 combinations [web/lib/types/sweep.ts], and the cap is applied by taking every $\lceil \lvert G\rvert/400 \rceil$-th combination rather than by truncating the list. Truncation would keep every slow period against the smallest fast periods and none against the largest; decimation by stride keeps the shape of both axes.

Walk-forward multiplies that budget. Each fold selects on the training window by running the whole grid, scores the winner out-of-sample, and then re-scores the whole grid out-of-sample as well, so the total number of combination evaluations in a run is

$$\lvert G\rvert + k(2\lvert G\rvert + 1) = 74 + 4 \times 149 = 670 \tag{12}$$

which is 670 evaluations [derived from `runSweep` and `walkForward`] at the defaults. The third term is the one that could be dropped and is not; the subsection on out-of-sample rank explains what it buys.

Where the cost of a combination is unusual, it is recorded. `linreg_forecast` refits a regression one hundred times per pass and costs approximately 15 ms per combination against approximately 0.4 ms for the parametric strategies [web/lib/strategies/grid.ts], which is why its threshold axis is stepped at 0.2 rather than 0.1: the finer step would make a 77-combination grid take about a second where every other sweep takes about forty milliseconds.

Walk-forward optimisation: the fold structure actually used

Let N be the bar count and k the requested fold count clamped to $[2, 10]$. The segment length is

$$\sigma = \lfloor N/(k+1) \rfloor \tag{13}$$

and fold $i \in \{0, \ldots, k-1\}$ uses, with an embargo of e bars,

$$\text{train}_i = [i\sigma, (i+1)\sigma - e), \quad \text{test}_i = [(i+1)\sigma, (i+2)\sigma) \tag{14}$$

Three guards apply. A segment shorter than 100 bars produces no folds at all, and the response carries the warning "walk-forward skipped: not enough bars for the requested folds" rather than an empty list that reads as a clean pass. A train or test window shorter than 50 bars breaks the loop. The embargo is clamped to $[0, \max(0, \sigma - 50)]$.

The windows are rolling and fixed-length, not expanding. The embargo is taken out of the *training window's tail* rather than by shifting the test window forward, and the reason is recorded at the site: shifting would walk the last fold off the end of the data and quietly drop it, so a researcher who switched the embargo on would silently lose a fold and see the median efficiency move for a reason they did not ask for.

At the shipped defaults $\sigma = 400$ and the layout is:

```
bar 0        400       800      1200      1600      2000
    |---------|---------|---------|---------|---------|
f1  [ train  ][  test  ]
f2            [ train  ][  test  ]
f3                      [ train  ][  test  ]
f4                               [ train  ][  test  ]
                 out-of-sample coverage: 1600 of 2000 bars (80%)
```

Adjacent folds leak without an embargo, and the leak is structural rather than accidental: a 200-bar moving average evaluated on the first test bar is mostly made of training bars, so that bar's "out-of-sample" score is partly in-sample. The embargo defaults to zero, which reproduces the Python reference exactly and is what the parity fixture pins; switching it on is an explicit choice.

Per-fold walk-forward efficiency is

$$\mathrm{WFE}_i = \begin{cases} \mathrm{OOS}_i / \mathrm{IS}_i & \text{if } \mathrm{IS}_i > 0 \\ \text{null} & \text{otherwise} \end{cases} \tag{15}$$

The null is the point. Dividing by a negative in-sample Sharpe produces a *positive* ratio for a fold that lost money in both windows, which reads as success on a chart. Those folds report null, are excluded from the median, and are counted separately.

Two further per-fold statistics are computed. Parameter drift is measured in grid-*index* space, not in raw parameter units, because the grid is a sparse lattice: with a fast step of 5 the neighbour of 25 is 20, and measuring distance in parameter units would call 24 a neighbour when 24 was never evaluated. Parameter persistence is then the share of fold transitions whose drift is exactly zero, or null when there are no transitions to measure.

The aggregate out-of-sample Sharpe is computed on the *concatenated* OOS return vectors rather than as a mean of the per-fold Sharpes. A mean of Sharpes would weight a fold with 50 surviving bars as heavily as one with 400.

The walk-forward verdict bands are:

Level	Condition	What it says
pass	median WFE ≥ 0.5 and $\geq 60\%$ of folds profitable OOS	parameters chosen on one window kept working on the next
marginal	median WFE > 0 and $\geq 50\%$ of folds profitable OOS	the edge decays; expect live results nearer the OOS column
fail	anything else	the edge does not survive the fold boundary

Out-of-sample testing, and what the fold rank buys

Scoring only the winner out-of-sample yields one number per fold, and one number cannot separate two very different situations: "this parameter choice was right" and "this fold was easy for everything in the grid". So the whole grid is re-scored on each test window, sorted, and the in-sample winner's rank among its peers is recorded together with the number of combinations ranked. That is what doubles the walk-forward cost, and it is the input the overfitting probability is computed from.

The Probabilistic Sharpe Ratio, derived

Let $x_1, \ldots, x_n$ be per-observation strategy returns with central moments $\mu_1, \mu_2, \mu_3, \mu_4$, standardised skewness $\gamma_3 = \mu_3 / \mu_2^{3/2}$ and *raw Pearson* kurtosis $\gamma_4 = \mu_4 / \mu_2^2$, so that a normal sample gives $\gamma_4 = 3$. The per-observation Sharpe ratio and its estimator are $S = \mu_1 / \sqrt{\mu_2}$ and $\hat{S} = m_1 / \sqrt{m_2}$ for the corresponding sample moments.

The estimator is a smooth function $g(m_1, m_2) = m_1 / \sqrt{m_2}$ of the first two sample moments, whose joint asymptotic covariance for an i.i.d. sample is

$$\mathrm{Var}(m_1) = \frac{\mu_2}{n}, \quad \mathrm{Var}(m_2) = \frac{\mu_4 - \mu_2^2}{n}, \quad \mathrm{Cov}(m_1, m_2) = \frac{\mu_3}{n} \tag{16}$$

The partial derivatives at the true moments are $\partial g/\partial m_1 = \mu_2^{-1/2}$ and $\partial g/\partial m_2 = -\mu_1/\left(2\mu_2^{3/2}\right)$. The delta method then gives

$$\operatorname{Var}\left[\hat{S}\right] \approx \frac{1}{\mu_2}\frac{\mu_2}{n} + \frac{\mu_1^2}{4\mu_2^3}\frac{\mu_4-\mu_2^2}{n} - 2\frac{\mu_1}{2\mu_2^2}\frac{\mu_3}{n} = \frac{1}{n}\left(1 - \gamma_3 S + \frac{\gamma_4 - 1}{4}S^2\right) \tag{17}$$

Each term is interpretable. The first is what a Gaussian sample would give on its own; the second says fat tails inflate the denominator's sampling error as the square of the Sharpe; the third says *negative* skew makes the estimator noisier than a normal sample of the same length, so a strategy that wins often and loses violently has a Sharpe to be believed less, not more.

Replacing n by $n-1$, the small-sample convention Bailey and López de Prado adopt, and standardising, gives the Probabilistic Sharpe Ratio, which is the probability that the true Sharpe exceeds a stated benchmark S^*:

$$\mathrm{PSR}(S^*) = \Phi\left(\frac{\left(\hat{S} - S^*\right)\sqrt{n-1}}{\sqrt{1 - \gamma_3\hat{S} + \left(\frac{\gamma_4-1}{4}\right)\hat{S}^2}}\right) \tag{18}$$

The implementation in `web/lib/stats.ts` is this expression and nothing else. It returns zero for $n < 3$ rather than raising, because fewer than three observations is an unusable sample rather than an error. The variance term is clamped at 10^{-12} from below, which matters under extreme negative skew where the bracket can go non-positive, and the clamp is duplicated exactly in the minimum-track-record inverse so the two stay exact inverses of each other. Φ is evaluated through Abramowitz and Stegun 7.1.26, whose error is bounded at 1.5×10^{-7} [web/lib/stats.ts], and Φ^{-1} through Acklam's rational approximation at 1.15×10^{-9} [web/lib/stats.ts]. Both bounds are far below any resolution at which a promotion decision changes.

Units are load-bearing

Every Sharpe entering these formulas is per-observation, never annualised. The sweep de-annualises the grid by dividing each combination's reported Sharpe by $\sqrt{\text{bars per year}}$, and computes the winner's per-bar Sharpe directly from its own bar returns. The response re-annualises exactly one figure, `expectedMaxSharpe`, for display, and the engine carries a comment warning that the minimum track record length benchmarks against the *per-bar* expected maximum rather than that displayed one. Mixing the two silently produces a hurdle wrong by a factor of $\sqrt{2190}$ at the 4h interval.

Minimum track record length

Solving $\mathrm{PSR}(S^*) = \alpha$ for n inverts the expression above exactly:

$$N^* = 1 + \left(1 - \gamma_3\hat{S} + \frac{\gamma_4 - 1}{4}\hat{S}^2\right)\left(\frac{z_\alpha}{\hat{S} - S^*}\right)^2 \tag{19}$$

with $z_\alpha = \Phi^{-1}(\alpha)$ and $\alpha = 0.95$ as shipped. The function returns ∞ when $\hat{S} \le S^*$, which is the honest answer: no finite record can demonstrate an edge that is not there. The engine converts a finite N^* to whole bars by ceiling, divides by the bars-per-year constant to report years, and reports a boolean `sufficient` comparing the run's own bar count against the requirement. Two benchmarks are reported side by side: $S^* = 0$, which asks how long a record must be to establish an edge against nothing, and $S^* = S_0^*$, the per-bar search hurdle derived in the next subsection, which asks how long it must be to establish an edge against *this search*.

The formula is worth a symbolic reading, because the shape of the answer is the argument for walk-forward in the first place. At $\gamma_3 = 0$, $\gamma_4 = 3$ and $S^* = 0$ the requirement collapses to $N^* \approx 1 + \left(1 + \hat{S}^2/2\right)\left(z_\alpha/\hat{S}\right)^2$, so the bar requirement falls as the *square* of the Sharpe. An annualised Sharpe of 1.0 on 4h bars is $\hat{S} = 1/\sqrt{2190} \approx 0.02137$ per bar, giving $N^* \approx 5\{,\}927$ bars, or approximately 2.71 years [derived from the formula above at the stated inputs] of continuous 4h data before a 95 per cent statement can be made. Halving the Sharpe quadruples that. This is arithmetic from the stated formula, not a measurement of any strategy.

The Deflated Sharpe Ratio, derived

The best Sharpe in a grid of N is the maximum of N draws, not an estimate of edge. A sweep over a pure random walk reliably produces an impressive-looking winner, and the Deflated Sharpe Ratio exists to price that. Under the null hypothesis every trial's true Sharpe is zero and the trial estimates are $N(0, V)$ where V is the variance of the Sharpes actually observed across the grid. Write M_N for the maximum of N i.i.d. standard normals. Its distribution converges to a Gumbel: with $a_N = \Phi^{-1}(1 - 1/N)$ and $b_N = 1/(N\varphi(a_N))$,

$$P(M_N \le a_N + b_N z) \longrightarrow \exp(-\exp(-z)), \quad \mathbb{E}[M_N] \approx a_N + \gamma b_N \tag{20}$$

where $\gamma \approx 0.5772156649015329$ is the Euler-Mascheroni constant, the mean of the standard Gumbel. The scale b_N has a convenient closed form. The normal tail is locally exponential with that same scale, so $1-\Phi(a_N+b_N) \approx (1-\Phi(a_N))\mathrm{e}^{-1} = 1/(N\mathrm{e})$, which says $a_N + b_N \approx \Phi^{-1}(1-1/(N\mathrm{e}))$ and therefore $b_N \approx \Phi^{-1}(1-1/(N\mathrm{e})) - a_N$. Substituting and collecting terms:

$$S_0^* = \sqrt{V}\left[(1-\gamma)\Phi^{-1}\left(1-\frac{1}{N}\right) + \gamma\Phi^{-1}\left(1-\frac{1}{N\mathrm{e}}\right)\right] \tag{21}$$

which is exactly the expression `deflatedSharpe` evaluates. The Deflated Sharpe Ratio is then the Probabilistic Sharpe Ratio taken against that hurdle rather than against zero:

$$\mathrm{DSR} = \mathrm{PSR}(S_0^*) = \Phi\left(\frac{\left(\hat{S}-S_0^*\right)\sqrt{n-1}}{\sqrt{1-\gamma_3\hat{S}+\left(\frac{\gamma_4-1}{4}\right)\hat{S}^2}}\right) \tag{22}$$

The hurdle is dispersion only, and that is deliberate

The sample *mean* of the candidate Sharpes is not added back. It is a common slip, and the file records why it is refused: on a uniformly losing grid the mean is negative, so a hurdle built from mean plus dispersion goes negative and a losing strategy clears it. Under the null every true Sharpe is zero, so the hurdle comes purely from how widely the search scattered.

Two degenerate cases are handled explicitly. With one trial the dispersion is zero, the hurdle is zero, and the DSR equals the PSR, which is correct: with no search there is nothing to deflate. With every trial identical the dispersion is also zero and the same thing happens, which is likewise correct, because a grid in which every combination produces the same Sharpe has not searched anything.

Two limitations are real and are stated rather than assumed away. First, the trials are *dependent*: a 5/20 crossover and a 5/40 crossover share most of their returns, so the effective number of independent trials is below N. That pushes the Gumbel term, which reads N literally, toward too high a hurdle; meanwhile the dependence also depresses the observed dispersion V, which pushes the hurdle down. The two errors point in opposite directions and nothing measured here signs the net. Second, the DSR prices the search *within* one sweep and cannot price the search *across* sweeps. Forty runs over forty hypotheses is itself a multiple-testing problem, and the winner's DSR does not know the other thirty-nine happened. The experiment log records the run count as prominently as it records the results, precisely because a tool that silently accumulates attempts while displaying per-attempt significance is actively misleading.

The probability of backtest overfitting

The reference construction, combinatorially purged cross-validation, partitions T observations into S groups, forms all $\binom{S}{S/2}$ train/test splits, and for each split c computes the out-of-sample relative rank ω_c of the configuration that was optimal in-sample. With $\lambda_c = \log(\omega_c/(1-\omega_c))$, the probability of backtest overfitting is $P(\lambda \le 0)$: the chance that the in-sample winner lands in the losing half out-of-sample.

Full CPCV is **not built**, and the reason is recorded at the site in `modules/backtester/engines.py`: it costs factorially more compute for a tighter estimate of the same quantity. What is built is the cheap sequential reading over the walk-forward folds that were computed anyway. With R the set of folds carrying both a rank ρ_k and a ranked-combination count $M_k > 1$,

$$\widehat{\mathrm{PBO}} = \frac{1}{|R|}\sum_{k\in R}\mathbb{1}\left[\rho_k > \frac{M_k+1}{2}\right] \tag{23}$$

rounded to four decimal places. It returns *null*, never zero, when no fold produced a rank, because "no fold could be ranked" and "no fold was overfit" are opposite claims.

Its coarseness is the fold count. With the default four folds the estimator can only take the values $0, 0.25, 0.5, 0.75, 1$, and with the maximum ten folds it takes eleven values. That is a derived property of the definition, and it is the reason the desk does *not* gate on it: the promotion gate contains no PBO check. A hurdle on a five-valued statistic estimated from four observations would be a coin flip wearing the costume of a control. Instead the figure is displayed, and it enters the quality score's robustness category at 35 per cent of that category's 20 points, inverted so that a high probability of overfitting costs points rather than earning them.

The data hash that ties a result to the exact bars

A symbol and a date range do not identify a dataset. The same window can be a live venue pull, a cached copy, or the synthetic fallback, and the synthetic series is not stable across processes. Both engines therefore fingerprint the bars a run actually saw.

	Python gateway	**TypeScript portal**
Function	`dataset_fingerprint`	`datasetFingerprint`
Digest	SHA-256, truncated to 16 hex characters	FNV-1a 32-bit, 8 hex characters
Covers	open, high, low, close as float64 bytes, plus index bounds and length	close rounded to 10^{-6}, plus length and first and last timestamps
Source	`modules/backtester/engines.py`	`web/lib/engine/frame.ts`

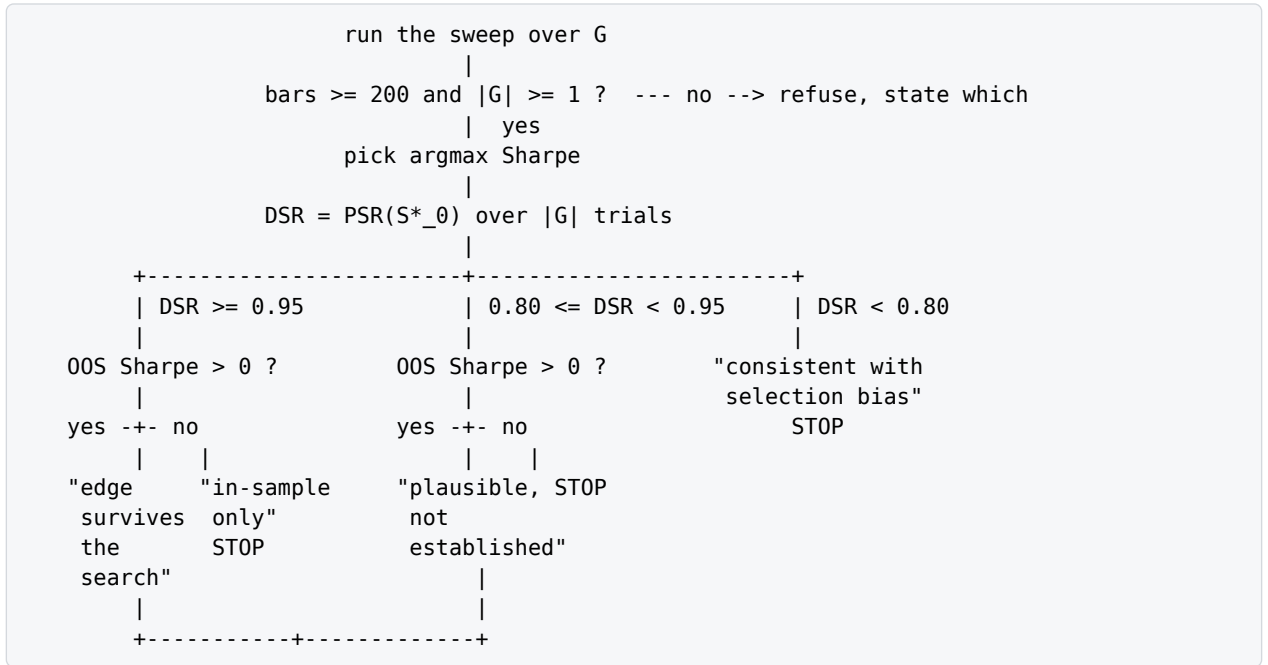
The Python side hashes every price column, not just the close, and the argument is written beside it: `donchian` reads highs and lows, so a vendor revising a session high changes the signal while the close is untouched, and a fingerprint that missed that would certify two runs as comparable at exactly the moment they stopped being so. The browser side hashes closes only, at six decimal places, which is enough to separate real revisions and coarse enough that a float round-trip through JSON does not change the answer.

The two are *deliberately* not expected to agree. They fingerprint their own inputs, which arrive over different transports with different float formatting. What each guarantees is internal consistency: two runs in the same engine over the same bars agree, and two that do not agree are not comparable however alike their headers look. Saying so plainly is better than a cross-engine hash that would have to normalise formatting and would then be trusted for a property it does not have.

The browser digest is 32 bits and the file says outright that it is not cryptographic. The consequence is quantifiable rather than hand-waved: a 32-bit digest reaches a fifty per cent collision probability at about $\sqrt{2 \ln 2 \cdot 2^{32}} \approx 77\{,\}000$ distinct datasets, and the experiment log is capped at 60 records [web/lib/experiments.ts], so the probability that any two records in a full log collide is about 4×10^{-7} [derived from the birthday bound at 60 records]. That is the argument for a cheap hash, made in numbers.

Downstream, the hash is what makes a research corpus queryable by data rather than by title. `tools/graph_recall.py` exposes a query over one `data_hash` that returns every run over the same bars together with what the graph says followed each, and its own summary line states the reason it is useful: results that disagree over one `data_hash` disagree about method, not data. The experiment record carries the field as optional, because runs recorded before it existed cannot be back-filled with anything honest, and the research cards print “Data hash: unrecorded” rather than leaving a blank that reads as agreement.

The decision tree from sweep to promotion



```
                |
  promotion gate: six vetoes, all displayed
                |
all six pass ? --- no --> not eligible; the failing rows are the work list
                |  yes
Kelly sizing from the run's own realised trades
(quarter Kelly, capped at 20% of the book)
                |
     hand to a sleeve
```

The verdict bands and the gate are two different objects and are deliberately not merged. The verdict is a sentence about the evidence; the gate is a vector of vetoes.

Check	Hurdle	Why it is there
Deflated Sharpe	≥ 0.95	prices the search itself: the probability the edge is real after paying for how many combinations were tried
Walk-forward OOS Sharpe	> 0	measured on data the parameters never saw; in-sample results are a fit, this is a test
Walk-forward efficiency	≥ 0.5	how much of the in-sample edge survives; below half the backtest is mostly fitting
Parameter neighbourhood	plateau or slope	a real edge degrades smoothly; an isolated spike is a coordinate found in noise
Alpha t-statistic	$\lvert t\rvert \geq 2$	return not explained by market, trend or volatility exposure, else it is a factor bet in disguise
Trade count	≥ 30	below about thirty trades the Sharpe is dominated by a handful of outcomes

Every check is a veto and every check is displayed whether it passes or fails. A gate panel that only appears on failure teaches people that the absence of a warning means safety; showing the full vector every time makes the one failing row the thing a reader looks for.

The parameter-neighbourhood check deserves its own definition, because it is the one that is not a threshold on a published statistic. Every grid point is classified by what its neighbours do, with adjacency taken in grid-index space for the sparse-lattice reason given earlier. With $\overline{S}_{\mathrm{nb}}$ the mean Sharpe of a cell's tested neighbours, the retention ratio is $\overline{S}_{\mathrm{nb}}/S_{\mathrm{cell}}$, and the classification is: retention ≥ 0.6 [web/lib/quant/stability.ts] is a plateau, retention ≤ 0.2 [web/lib/quant/stability.ts] is a cliff, between them is a slope, fewer than three tested neighbours is isolated, and a non-positive Sharpe is dead. A winner classified isolated sits on the grid boundary, and the verdict says so and asks for a wider range, because an optimum at the boundary is usually a sign that the true optimum is outside the grid.

Retention is reported as a percentage only inside a band where a percentage means something. The denominator is a Sharpe that can be a hair above zero, and the file records the pathology it produced: a winner at 0.006 with negative neighbours gave -8268% [web/lib/quant/stability.ts], arithmetically correct and communicating nothing. Outside $[0, 2]$ the two Sharpes are quoted directly. Percentages are a convenience, not the measurement.

Beyond the gate, one 0-100 score exists so that many runs can be ranked at a glance. Its weights are a deliberate departure from the obvious ones:

Category	Weight	Composition
Risk-adjusted	35 [web/lib/quality-score.ts]	DSR at 0.67, raw Sharpe at 0.33
Robustness, out-of-sample	20 [web/lib/quality-score.ts]	efficiency 0.40, inverted PBO 0.35, OOS Sharpe 0.25
Drawdown and tail	15 [web/lib/quality-score.ts]	max drawdown 0.60, Calmar 0.40
Versus benchmark	15 [web/lib/quality-score.ts]	Sharpe edge 0.70, return edge 0.30
Trade quality	8 [web/lib/quality-score.ts]	sample size 0.65, win rate 0.35
Absolute return	7 [web/lib/quality-score.ts]	total return over the window

The sibling weights this replaced scored on raw Sharpe and raw return, statistics this engine already knows to be inflated by selection; scoring on the naive numbers while owning the corrected ones would publish a figure the repository's own machinery disagrees with. So robustness gets its own twenty points and the risk-adjusted category is DSR-led. Absolute return is last and lightest on purpose: a large return earned by taking a large risk is already counted twice above, and weighting it heavily is how a scoring system learns to prefer leverage. The benchmark category carries a scar worth recording, because it is the exact failure this chapter is about. It originally compared the *in-sample* Sharpe against the benchmark, and a run with a grid-inflated Sharpe of 3.2, a DSR of 0.2 and 80 per cent PBO collected full marks in that category and reached 55 overall [web/lib/quality-score.ts]. The whole point of the robustness category is that the in-sample number is not to be trusted, and then the category beside it trusted it. It now compares the walk-forward out-of-sample Sharpe wherever walk-forward ran, and labels the result in-sample where it did not.

The guardrails, and the one they do not cover

Four of the mechanisms have already been derived above: grid hygiene, so that no strategy is swept over a space in which every combination is silently discarded; the separation of the code allowed to pick a winner from the code that reports one; the embargo; and neighbourhood classification, under which a cliff fails the gate even when its Sharpe is the highest in the grid. Two more carry content of their own.

- **The factor regression.** Three time-series factors built from the same instrument's own bars over a 30-bar [web/lib/quant/factors.ts] lookback: the asset itself, time-series momentum, and a volatility-regime factor whose threshold is an *expanding* mean of prior observations rather than a full-sample median. The look-ahead avoided there is the insidious direction: a benchmark built with hindsight is stronger than one anybody could have traded, so a real edge could be argued away by a factor that could not have existed. The factors are executed with the same one-bar lag as the strategy, so a beta is an exposure rather than a timing artefact. The t-statistics are plain OLS, and the file states that a Newey-West correction would widen them, meaning the significance reported is generous rather than conservative. Newey-West is **not built**.
- **Sizing that refuses.** Kelly sizing is quartered and capped at 20 per cent [web/lib/quant/sizing.ts] of the book. A negative f^* returns zero rather than an inverted position, because an edge that only exists when you flip it is a fitting artefact. A strategy with no losing trades has an *undefined* payoff ratio, not an infinite one, and also returns zero. The file records the measured case that motivated the thin-sample flag: on live BTC 4h at the defaults, `ma_cross` produced a 17.7 per cent allocation, 1.77M USD of a 10M USD book, from six trades [web/lib/quant/sizing.ts]. The formula is indifferent to whether the payoff ratio came from six samples or six hundred; the consequence of being wrong is not.

The guardrail that does not exist is the cross-sweep one. Nothing prices the number of hypotheses a researcher tried before this one. The experiment log makes the count visible and says in its own header why visibility is the most it can offer; a corrected across-sweep statistic is **not built**, and it waits on a definition of what counts as one attempt that survives a researcher reloading a tab.

2.2 Part B: the portfolio manager

Mean-variance, and the part of it the desk actually solves

The canonical program is, for a long-only fully-invested book,

$$\max_{w} \; w^{\top}\mu - \frac{\lambda}{2} w^{\top}\boldsymbol{\Sigma} w \quad \text{subject to} \quad \mathbf{1}^{\top}w = 1, \quad w \geq 0 \tag{24}$$

whose unconstrained solution is $w \propto \boldsymbol{\Sigma}^{-1}\mu$. The desk does not solve it, and the reason is written at the top of `modules/quant_risk/allocation.py`: the allocator is deliberately naive about expected return, because forecasting covariance is hard and forecasting returns is harder, and a proposal that pretends otherwise is an opinion dressed as arithmetic. An expected-return model is **not built**. What is solved is the μ-free corner of the same program, $\lambda \to \infty$, together with three risk-only heuristics, in increasing order of what they claim to know.

Equal weight is $w_i = 1/n$ over *distinct* symbols. It knows nothing and says so, which makes it the honest baseline the other three have to beat. The distinctness matters: both engines key weights by symbol but iterate the position list when building targets, so a duplicated symbol would collect the same weight twice and silently inflate gross.

Inverse volatility is $w_i \propto 1/\sigma_i$, so a quiet instrument carries more notional than a violent one for the same risk. It ignores correlation entirely.

Equal risk equalises each position's contribution to book volatility. By Euler's theorem on the homogeneous-of-degree-one function $\sigma_{p(w)} = \sqrt{w^\top \Sigma w}$,

$$\sigma_p = \sum_i w_i \frac{\partial \sigma_p}{\partial w_i} = \sum_i \frac{w_i (\Sigma w)_i}{\sigma_p}, \quad \text{so} \quad \mathrm{RC}_i = w_i (\Sigma w)_i, \quad \sum_i \mathrm{RC}_i = \sigma_p^2 \tag{25}$$

The target is $\mathrm{RC}^* = \sigma_p^2 / n$ and the solver is the multiplicative fixed point $w_i \leftarrow w_i \sqrt{\mathrm{RC}^* / \mathrm{RC}_i}$ followed by renormalisation, seeded at inverse volatility because that is already the exact answer when the correlations are zero, so the iteration only has to undo the correlation. A non-positive contribution or a non-positive marginal is held flat rather than updated, because $\sqrt{\text{negative}}$ would put a NaN into every weight at the next renormalisation.

Minimum variance is the long-only fully-invested portfolio with the smallest variance the estimated covariance allows. Its Lagrangian is

$$\mathcal{L}(w, \theta, \nu) = w^\top \Sigma w - 2\theta (\mathbf{1}^\top w - 1) - \sum_i \nu_i w_i \tag{26}$$

with stationarity $2(\Sigma w)_i - 2\theta - \nu_i = 0$ and complementary slackness $\nu_i w_i = 0$, $\nu_i \geq 0$. On the support, where $w_i > 0$, this forces $\nu_i = 0$ and therefore

$$(\Sigma w)_i = \theta \quad \text{for every } i \text{ with } w_i > 0, \quad (\Sigma w)_j \geq \theta \text{ otherwise} \tag{27}$$

Every marginal variance is equal on the support. That is a fixed point of exactly the same shape as the equal-risk update, which is why the file has one solver family rather than two, and no simplex projection or step size to tune. The update is $w_i \leftarrow w_i \sqrt{\sigma_p^2 / (\Sigma w)_i}$, seeded at inverse *variance* because that is the exact answer at zero correlation. A negative marginal variance is a hedge, has no update in this fixed point, and is held flat.

Two implementation choices are load-bearing and both are recorded. Both iterative solvers run a *fixed* 60 iterations [modules/quant_risk/allocation.py] rather than testing for convergence, because a tolerance check lets two implementations stop on different iterations and disagree by more than the cross-language fixture allows, whereas a fixed count cannot. And the minimum variance solver keeps the *best* iterate rather than whichever the loop ended on, because a multiplicative fixed point is not proven to decrease the objective at every step, and a method called minimum variance that returned something more volatile than inverse volatility would be indefensible.

The parity fixture records what all four produce on one three-symbol book, and the numbers below are read from it rather than recomputed here.

Method	AAA	BBB	CCC	Gross after
Current	120,000	180,000	−60,000	360,000
Equal weight	0.33333	0.33333	0.33333	360,000
Inverse volatility	0.25432	0.21306	0.53262	360,000
Equal risk	0.22806	0.07710	0.69484	360,000
Minimum variance	0.30859	0.00000	0.69141	360,000

All twelve target weights and both gross figures [web/tests/fixtures/risk-parity.json]. The ordering is instructive and is a property of the methods rather than of this book: minimum variance is the most concentrated of the four by construction, here driving one sleeve to exactly zero, which is why it clips against a symbol cap most often.

Clipping is where the allocator meets the gateway. Every proposal is clipped by the same limits the risk gateway enforces, and each clipped weight names the constraint that bound it, because a proposal that ignored the limits would be rejected order by order at the gate, which is a worse way to discover it. On the same fixture, with a symbol cap of 150,000 and a gross cap of 300,000, the budget becomes $\min(360\{,\}000, 300\{,\}000) = 300\{,\}000$; the minimum-variance weights put $0.69141 \times 300\{,\}000 = 207\{,\}423$ into CCC, which is clipped to 150,000, tagged `max_symbol_notional_usd` [web/tests/fixtures/risk-parity.json], and the proposal's gross after falls to 242,577 [web/tests/fixtures/risk-parity.json]. The weights no longer sum to one after clipping and the proposal carries a `clipped` flag saying so rather than renormalising, which would quietly hand the clipped notional to whichever sleeve happened to be uncapped.

The trades that reach a proposal are filtered by a drift band of 5 per cent of gross [modules/quant_risk/allocation.py]. The band is what stops a rebalance from being a fee-generating machine: a position one per cent from target costs more to correct than the correction is worth. On the fixture's unclipped minimum-variance proposal the three drifts are −2.47% for AAA, −50.0% for BBB and +52.5% for CCC, so AAA is left alone and two trades

are produced. The direction of the CCC trade is the detail that makes the band non-trivial: CCC is a *short*, so increasing its target notional means selling more of it, and the fixture records SELL 188,907.60 on CCC and SELL 180,000.00 on BBB [web/tests/fixtures/risk-parity.json]. The proposal alone cannot produce that direction, which is why the current positions are still an argument to the trade builder.

The covariance model and where it is measured from

The estimate is a plain sample covariance over the window every symbol shares:

$$\Sigma_{ij} = \frac{1}{W-1}\sum_{k=1}^{W}(r_{ik}-\bar{r}_i)(r_{jk}-\bar{r}_j), \quad W = \min_i |r_i| \tag{28}$$

Truncating to the shortest series rather than padding is the point. A symbol with a shorter history would otherwise have its missing bars treated as zero-return days, which understates its variance and, because the zeros line up across symbols, inflates every correlation toward one: the two errors that both make a book look safer than it is. Correlation is the usual normalisation with an explicit zero-denominator guard, and annualisation by $\sqrt{\text{BARS\_PER\_YEAR[interval]}}$ is applied to the volatilities, not to the matrix. The observation count W travels with the estimate, because a small W is a weak covariance and the consumer has to be able to see that.

Live, the matrix is built from the returns of the symbols actually held, on demand, by the risk commands that report contributions and exposure. Nothing caches it: a PM asking why a number says what it says gets a query for an answer.

One divergence between the two implementations is worth recording rather than smoothing over. The Python builder requires only two observations per symbol and a shared window of two; the TypeScript builder requires 20 observations per symbol and a shared window of 20 [web/lib/portfolio-risk/covariance.ts], returning null below that. The TypeScript floor is the stricter and the more defensible one. The parity fixture runs on 120 observations per symbol [web/tests/fixtures/risk-parity.json], so the divergence does not bite there, which is exactly why it is stated here instead of being discovered later.

Both implementations sum sequentially, never through `math.fsum` or a vectorised dot product. Floating-point addition is not associative, and pairwise summation rounds differently from JavaScript's left-to-right accumulation; the parity fixture exists to catch that class of drift, so the summation order is part of the contract rather than an implementation detail.

Risk contributions, and why share of notional is not share of risk

With w_i the signed notional of position i divided by equity, the marginal variance vector is $(\Sigma w)_i$, the book variance is $w^\top \Sigma w$, and each position's contribution and share are

$$\text{RC}_i = w_i(\Sigma w)_i, \quad \text{share}_i = \frac{\text{RC}_i}{w^\top \Sigma w}, \quad \sum_i \text{share}_i = 1 \tag{29}$$

The weights are signed and scaled by equity rather than normalised to one, so a short enters the quadratic form with a negative weight and its covariance with the longs subtracts. That is the reason the decomposition exists: a 13 per cent sleeve in a volatile name can carry more risk than a 42 per cent one in a quiet name, and a hedging short contributes a *negative* amount, a number a notional-weighted view cannot produce at all. These weights sum to net exposure over equity, not to one, so they are a leverage-aware weighting, distinct from the normalised weights the allocation solvers work in.

A non-positive computed variance returns *null* rather than a zero volatility, because a book whose estimated variance is degenerate has not been measured as riskless.

The diversification ratio reported beside the contributions is

$$\text{DR} = \frac{\sum_i |w_i|\sigma_i}{\sigma_p} \tag{30}$$

the weighted sum of standalone volatilities over the realised book volatility, both per-bar so the ratio is scale-free. It is one when the book is a single bet and rises as the positions stop moving together.

Intraday P and L attribution: the waterfall

A day's profit-and-loss number answers "how much" and nothing else. The question a PM asks at the close is how much of it was the market moving and how much was the desk, because the answer decides whether a good day is worth repeating. The session is decomposed into four legs that sum, by construction, to the number they decompose:

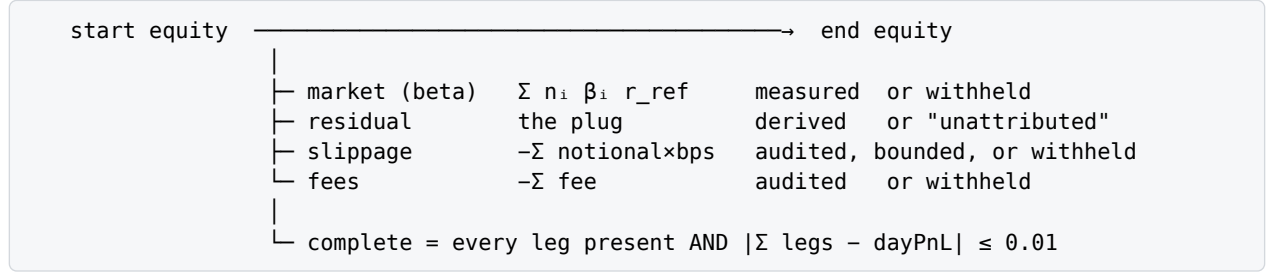
$$\text{dayPnL} = \text{market} + \text{residual} + \text{slippage} + \text{fees} \tag{31}$$

where the residual is a plug, dayPnL − market − slippage − fees. The accounting is not double counting, and the argument is written at the module head: the position state subtracts each fee from realised P and L as the fill is applied, and paper fills print at the smart-route VWAP rather than at mid, so fees and slippage are *already inside* day P and L. Pulling them out as their own bars and letting the residual absorb the remainder counts each exactly once.

The market leg is measured exposure against a measured reference move:

$$\text{market} = \sum_{i \in M} n_i \beta_i r_{\text{ref}}, \quad M = \{i : \beta_i \text{ is measurable}\} \tag{32}$$

with n_i the signed notional and $\beta_i = \text{Cov}(r_i, r_{\text{ref}}) / \text{Var}(r_{\text{ref}})$ estimated on at least 20 shared observations [web/lib/portfolio-risk/stress.ts]. The beta function returns null rather than defaulting to 1.0, and that null has to survive all the way into the leg: defaulting an unknown beta to one is the quiet way an attribution starts inventing exposure, because every unmeasurable instrument would then move exactly with the reference and the resulting number would look like a measurement. The leg is computed on *closing* exposure, because closing exposure is what the payload carries, so a book that traded during the session did not hold this exposure all day and the leg carries that error. That error is one of the several things living in the residual.



Every leg can refuse to exist, and a refusal is never a zero. The five refusals are the substance of the module:

Condition	Consequence
No reference return could be measured	the market leg is withheld *and so is the residual* the remainder is relabelled "unattributed", because a residual with no market leg subtracted from it is day P and L wearing a more flattering name
Some betas unmeasurable	those positions leave the leg, which is then understated; their names are reported and their P and L falls into the residual
No held position has a measurable beta	the leg is withheld outright, and the excluded-names list is emptied, because a leg that does not exist cannot be understated by the names missing from it
$0 <$ fills without slippage $<$ fills	the slippage leg is a *lower bound*: a measurement with a known direction of error
Fills without slippage $\geq$ fills	the leg is withheld. The cost query sums notional times slippage coalesced to zero, so an all-null session reports a cost of exactly 0.0, which is a sum over nothing rather than a measurement of nothing

The last of those is the sharpest, and it is reachable from one mark outage during a maker-filled session. Drawing it as an audited leg of zero would say execution was free on a session where nobody could measure what execution cost.

Every cost figure is scoped to the session. The gateway zeroes per-position realised P and L at UTC midnight, so today's P and L contains only today's costs, and the lifetime aggregates in the payload are not merely useless here but dangerous: subtracting a lifetime fee total from one day's P and L reports a loss the desk did not take, and an unweighted mean of basis points cannot become dollars at all. The session block is accepted only when it describes this book and this day, with three rejections for the three lies it could otherwise tell: a basis disagreeing with whether the book is generated; a session date other than the book's; and a block with no basis at all, which is a gateway without an audit log rather than one reporting zero cost. A block naming *no* date is

accepted, since its costs may well be this session's, but the note then says the date could not be checked rather than asserting a day the block never claimed.

Completeness is a statement about arithmetic closure and deliberately not a quality claim: the legs reconcile to within one cent [web/lib/pnl-attribution/types.ts], the residual being a plug, so anything larger is a bug rather than a rounding artefact. A waterfall can be complete while its market leg is understated by an unmeasurable beta, which is why the unmeasured-symbol list and the lower-bound flag are separate fields a caller has to read as well. A further field carries dayPnL − (realised + unrealised), which is legitimately non-zero in a correct multi-day book: mark-to-market carried in on positions opened before the session.

The residual is never called alpha. It contains genuine idiosyncratic moves, but also intraday trading P and L, beta-estimation error, the P and L of every position whose beta could not be measured, and the error from computing the market leg on closing rather than opening exposure. Naming that alpha would be exactly the fabrication the rest of the system refuses.

Concentration and the effective-position count

With $s_i = |n_i|/$ gross the share of gross of position i, the Herfindahl-Hirschman index and the effective number of positions are

$$H = \sum_{i=1}^{n} s_i^2, \quad N_{\text{eff}} = \frac{1}{H} \tag{33}$$

The bounds follow immediately. By Cauchy-Schwarz, $\sum_i s_i^2 \geq \left(\sum_i s_i\right)^2 / n = 1/n$ with equality if and only if every share is equal, and $H \leq 1$ with equality when one position is the book. So $N_{\text{eff}} \in [1, n]$: it is n for a perfectly spread book and one for a single-name book, and it is *scale-free*, which is what makes it comparable day to day as the book grows.

On the three-position book in the parity fixture, shares of 0.5, 0.3333 and 0.1667 give $H = 0.3889$ and $N_{\text{eff}} = 2.57$ [derived from `build_portfolio` over the positions in web/tests/fixtures/risk-parity.json]: three positions that behave, from a concentration standpoint, like about two and a half. Largest share and top-two share are reported alongside, because one large position and many equal ones are both concentrated in different ways and a single metric hides one of them.

What the code *enforces* here is nothing. Concentration is measured and reported; the enforced cap is the per-symbol notional limit, which is a dollar figure and not a share. The pushed alert on concentration defaults to 0.0, meaning off [config.py], and the reason is recorded at the setting: a one-instrument desk is 100 per cent concentrated and correct, so the right threshold is a house decision rather than something a config default can guess. Zero disables a rule outright rather than setting an unreachable threshold, because "off" and "never fires in practice" are different states and only one of them is honest.

One honest exception to the absence convention lives here. `effective_positions` returns 0.0 when $H = 0$, which happens only on an empty book, and a zero rather than a dash is a coercion of exactly the kind this system refuses elsewhere. It survives because `positions: 0` is carried immediately beside it, so the zero is readable in context rather than standing alone. It is named here rather than left for a reader to find.

Risk-budget allocation and the binding constraint

Every limit is expressed as the same triple, (used, limit, remaining) with utilisation = used/limit, and the book's headline status is derived from the constraint that actually binds:

$$(\text{name}^*, u^*) = \text{argmax}_c\, u_c \text{ over } \{\text{gross exposure}, \text{daily drawdown}\} \cup \{\text{symbol} : i\} \tag{34}$$

Limit	Default	What it is
`MAX_ORDER_NOTIONAL_USD`	50,000 [config.py]	hard ceiling on a single order, a fat-finger guard
`MAX_SYMBOL_NOTIONAL_USD`	150,000 [config.py]	per-symbol net notional ceiling
`MAX_GROSS_EXPOSURE_USD`	500,000 [config.py]	aggregate gross notional across open positions
`MAX_DAILY_DRAWDOWN_PCT`	0.05 [config.py]	circuit breaker as a fraction of start-of-day equity

`REDUCE_ONLY_THRESHOLD`	0.80 [config.py]	fraction of the drawdown budget at which risk-increasing orders are refused and risk-reducing ones still pass
`VAR_BUDGET_PCT`	0.02 [config.py]	advisory ceiling on one-day 95 per cent VaR, reported and never enforced
`STARTING_EQUITY_USD`	1,000,000 [config.py]	notional starting equity for the paper book

The reduce-only threshold is the only graduated response in the set, and it is the FIA practice of letting a desk close out of trouble but not deeper into it: between 0.80 and 1.0 of the drawdown budget the gate is directional rather than binary. The VaR budget is reported and never enforced, with the reason stated at the setting: VaR needs history, and denying orders on a missing covariance would halt a healthy book on its first day.

Utilisation is banded at 0.7 and 0.9 [web/lib/portfolio.ts] into headroom, warning and breach, in one table quoted by the status line, every headroom gauge and the limits table, because three copies of a threshold are three chances to disagree about whether 0.9 is a breach. A non-finite utilisation is caught explicitly rather than falling through both comparisons into the safe band, which is the direction an unchecked NaN fails in.

The status line carries a scar that a screenshot would not have caught. The utilisation was once re-derived locally as the maximum of gross and drawdown while the constraint's *name* came from the gateway. On a book whose largest position sat at 90 per cent while gross sat at 72 per cent [web/lib/portfolio.ts], the chip read "elevated, symbol exposure at 72 per cent": the right name beside the wrong number, one severity band too low, on the one indicator a PM glances at instead of reading the page. The gateway already computes which constraint binds and how hard; trusting that and taking the maximum against the headrooms visible locally means the label and the number cannot disagree.

The drawdown limit is the one whose headroom is deliberately reported in different units from its limit. The limit is a percentage of start-of-day equity; the headroom that matters is the equity cushion in dollars, which is the number a PM can compare against a position size. The row carries both a `unit` and a `headroomUnit` rather than being pre-formatted, because a single formatter would have to guess.

Leverage: what is measured, and what is enforced

Leverage is computed as gross exposure over current equity and reported to three decimal places on the portfolio view and the exposure command. Nothing is keyed on it. The enforced gates are dollar-denominated, per order, per symbol and in aggregate, plus the drawdown breaker and the rate limiter. A leverage limit is **not built**, and stating that is more useful than implying one exists.

A notional cap and a leverage cap diverge as equity moves, so the relationship is worth deriving. At the shipped defaults the implied leverage ceiling at start-of-day equity is

$$\frac{\text{MAX\_GROSS\_EXPOSURE\_USD}}{\text{STARTING\_EQUITY\_USD}} = 500\{,\}\frac{000}{1}\{,\}000\{,\}000 = 0.5\times \tag{35}$$

so the gross cap binds far below $1\times$ and a leverage gate would never be the binding constraint at these defaults. As equity falls the implied ceiling *rises*, since the cap is a fixed dollar figure: at the 5 per cent drawdown halt the same cap implies $500\{,\}000/950\{,\}000 \approx 0.526\times$. The effect is small at the shipped numbers and structural at any others, which is the reason to write the relationship down rather than the number.

The pushed alert on gross exposure over equity also defaults to 0.0, meaning off [config.py], with the reason recorded: a desk running deliberate leverage would be paged constantly, and the right number is a house decision.

What this chapter's subject matter does not have

Collected so that no reader has to infer it from silence:

- **No expected-return model**, therefore no mean-variance frontier and no tangency portfolio. The allocator answers how the risk should be spread, never what the book should own.
- **No combinatorially purged cross-validation.** The overfitting probability is the sequential reading over walk-forward folds, with the five-valued coarseness derived above, and it is not a gate.
- **No Newey-West standard errors.** The alpha t-statistic is plain OLS, so the reported significance is generous and the interface frames a significant alpha as "not explained by these three factors" rather than as real alpha.

- **No cross-sectional factors.** One instrument's OHLCV cannot produce SMB, HML or a momentum decile, and constructing something that merely resembles them from a single series would be the dishonesty the rest of the system exists to prevent.
- **No across-sweep multiplicity correction**, as discussed above.
- **No sleeve exposure decomposition.** Sleeve mix is a *flow* measure, traded notional per strategy over the audit log's lifetime, because the positions payload carries no strategy tag to build current exposure from. The panel says the word "flow"; calling it sleeve concentration without that word would describe a chart the data cannot draw.
- **No live emission of session summaries.** The producer is built, tested and called by the backfill tool, but nothing emits one in process, so on a running desk the summaries appear when the backfill is run and not before.
- **No real order routing.** Orders are paper, capped by the gateway's own gates, and the honesty ledger in the repository README carries the full list of what is mocked against what is implemented.

Each of these is an absence with a reason, which is a different object from a gap. A desk that knows which of its numbers are measurements can be argued with, and that is the only property that matters on the day the arithmetic disagrees with the person reading it.

3 Risk Manager and Quant Trader

The risk manager asks two questions: how much can this book lose, and who can stop the desk. The trader asks two more: what will this order cost, and where should it go. All four are answered against the same two objects — an in-memory position book marked once per second, and a cross-venue consolidated ladder rebuilt at feed rate — so the four answers cannot be made to disagree by being computed from different state. Section 3.1 through Section 3.7 are the risk manager's half; Section 3.8 through Section 3.14 are the trader's.

The organising discipline of both halves is that a loss estimate is a **model output**, and a model output that does not say what it assumed, how many observations it had, and what it refuses to answer without, is a number wearing a statistic's clothes. Every estimator below therefore has a refusal condition, and every refusal returns a typed absence with a named reason rather than a zero, an empty list or an exception.

3.1 One loss, three estimators

The book's one-bar loss distribution is estimated three ways. They are not redundant, and they are not averaged. Each makes a different assumption, and the **spread between them** is the information a single number destroys.

Parametric normal

Let $\boldsymbol{w} \in \mathbb{R}^n$ be the signed position weights, $w_i = \pm\,|N_i|\,/E$ for notional N_i and equity E, negative for a short. Let Σ be the sample covariance of per-bar simple returns, estimated with $\mathrm{ddof} = 1$ over the window every symbol shares:

$$\Sigma_{ij} = \frac{1}{m-1}\sum_{k=1}^{m}(r_{ik}-\bar r_i)(r_{jk}-\bar r_j) \tag{36}$$

Book volatility and the parametric quantiles follow:

$$\sigma_p = \sqrt{\boldsymbol{w}^\top \Sigma \boldsymbol{w}}, \quad \mathrm{VaR}_{95} = z_{95}\sigma_p E, \quad \mathrm{CVaR}_{95} = \kappa_{95}\sigma_p E \tag{37}$$

with $z_{95} = 1.6448536269514722$ and $\kappa_{95} = \varphi(z_{95})/0.05$, the normal expected-shortfall multiplier 2.0627128027825736 [`modules/quant_risk/_common.py`].

The window is truncated to the shortest series rather than padded, and the comment in `build_covariance` gives the reason: padding a short history with zero-return days understates that instrument's variance and, because the zeros line up across symbols, inflates every off-diagonal correlation toward one. A diversification claim manufactured by missing data is the failure this truncation exists to prevent.

Summation is sequential in both implementations — never `math.fsum`, never `numpy.dot`. Pairwise summation rounds differently from JavaScript's left-to-right accumulation, and the Python/TypeScript parity fixture exists to catch exactly that class of drift, so the slower loop is the one that keeps the two stacks provably identical.

A seam worth naming. The TypeScript constant is 2.0627128054846826 [`web/lib/portfolio-risk/risk.ts`], which differs from the Python constant in the ninth significant figure — a relative gap of about 1.3×10^{-9}, and neither equals $\varphi(z_{95})/0.05$ evaluated in double precision. No fixture pins the parametric CVaR across the two stacks (`risk-parity.json` carries the **historical** CVaR, which is a tail mean and uses no multiplier), and the Python-side assertion uses `pytest.approx` at its default relative tolerance of 10^{-6}, which cannot resolve the difference. The gap is far below any precision a desk reads, and it is recorded here because an unpinned seam between two implementations is the thing this architecture otherwise refuses to have.

Historical replay

No distribution is assumed. Today's weights are applied to each past bar's returns, producing a replayed per-bar book P&L series $L_t = \sum_i s_i r_{it}$ with $s_i = \pm\,|N_i|$; the series is sorted ascending, and with $k = \max(1, \lceil 0.05m \rceil)$:

$$\mathrm{VaR}^{\mathrm{hist}}_{95} = -L_{(k)}, \quad \mathrm{CVaR}^{\mathrm{hist}}_{95} = -\frac{1}{k}\sum_{j=1}^{k} L_{(j)} \tag{38}$$

The tail is selected **by rank**, never by value. Selecting every observation at or below the quantile is equal to the rank rule only when the quantile is not a repeated value, and in a strategy that is flat most of the time it very often is: a run holding nothing on most bars earns exactly zero, the fifth percentile lands on that atom of zeros, and the "tail" silently becomes every non-positive bar. The repository records the measurement that

caught this — a default RSI run understated CVaR_{95} by 19.8× [`web/lib/quant/tail-risk.ts`] and CVaR_{99} by 99× [`web/lib/quant/tail-risk.ts`], printing the two as the identical number, which is the visible tell that the selection was by value.

Bootstrap terminal distribution

A single-bar VaR answers "how bad is one bad bar". A desk unwinding over several bars wants the distribution of where the book **lands**. Each of P paths draws h per-bar P&L figures from the replayed series and accumulates them; the terminal values form the P&L distribution and the per-step columns form the percentile cone:

$$T^{(p)} = \sum_{t=1}^{h} X_{i_t}, \quad X_j \in \{L_1, \ldots, L_m\} \tag{39}$$

Loss quantiles are read off the sorted terminal vector by the same nearest-rank rule, at $k = \max(1, \lceil (100 - C)/100 \cdot P \rceil)$. The expression is written $(100 - C)/100$ rather than $1 - C/100$ deliberately: at $C = 95$ the first is the double 0.05 and the second is 0.05000000000000004, which is a different `ceil` for some path counts, and both stacks take the same route.

The conditions under which each is trusted

Estimator	Trusted when	Refuses below	What it cannot see
Parametric	returns are near-normal and the covariance window is representative	$m < 2$ aligned bars, or $\sigma_p^2 \le 0$	fat tails; it understates loss in exactly the conditions a risk manager cares about
Historical	the past window contains the kind of day being asked about	20 aligned observations	any loss larger than the worst day in the sample
Bootstrap	the sample is long enough to carry tail shape	60 observations; horizon clamped to 60 bars, paths to $[200, 20\,000]$	under i.i.d. draws, volatility clustering — a sustained drawdown arrives in runs

Every refusal returns `None` and is rendered as a dash with its reason, never as a zero. The floors are not arbitrary. Twenty observations is the point below which a fifth percentile is a single data point wearing a statistic's name; sixty is the point below which a bootstrap would be manufacturing tail shape the sample never showed, and a cone drawn from a dozen days gives false confidence to noise.

Validating the estimate: the Kupiec proportion-of-failures test

A VaR nobody has back-tested is an opinion. The forecast is re-estimated on a rolling window of $w = 60$ bars and scored against the **next** bar's realised book P&L, so it is never judged on data it was fitted to; a bar breaches when $-L_t > z_{95}\hat{\sigma}_{t-w:t}$. With x exceptions in n scored bars at level $\alpha = 0.05$ and $\hat{p} = x/n$:

$$\mathrm{LR}_{\mathrm{uc}} = -2\ln\left(\frac{(1-\alpha)^{n-x}\alpha^{x}}{(1-\hat{p})^{n-x}\hat{p}^{x}}\right) \quad \sim \chi^2_1 \tag{40}$$

The p-value is the exact survival function $\mathrm{erfc}\left(\sqrt{\mathrm{LR}/2}\right)$ in Python; the browser has no `erfc`, so it uses the Abramowitz and Stegun 7.1.26 rational approximation, accurate to about 1.5×10^{-7} — far tighter than the 0.01 and 0.05 thresholds it feeds. The parity test asserts the **zone**, not the p-value, because the zone is what a risk manager acts on and the sixth decimal is not.

The verdict is two-sided, and that is the point. Too many exceptions means the model understates risk. Too **few** means it overstates it, and the desk is holding capacity it never uses — a model with zero exceptions is not conservative, it is wrong in the expensive direction.

3.2 The second implementation, and why one exists at all

`POST /api/oracle/var` runs a Monte Carlo **inside Oracle 23ai** and returns a 99% terminal-value VaR. It is not there because the database is faster or the number is better. It is there because two independent implementations of one quantity are two chances to be wrong and one chance to notice, and when they disagree by more than sampling error, **the disagreement is the finding**.

The model is geometric Brownian motion under a drift adjustment, over a 365-day year with $T = \text{days}/365$:

$$S_T = S_0 \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}Z\right), \quad Z \sim \mathcal{N}(0,1) \tag{41}$$

$$\mathrm{VaR}_{99} = \max(S_0 - Q_{0.01}(S_T), 0) \tag{42}$$

The procedure generates paths in an inline view over `CONNECT BY LEVEL`, takes `PERCENTILE_CONT(0.01)` over that view, and persists nothing at all. Four properties of it are load-bearing, and each corrects a defect in the design it was built from:

1. `FORALL i IN 1..p_simulations PARALLEL` does not compile — `FORALL` iterates a bound collection and has no `PARALLEL` clause. The generator is set-based instead.
2. The percentile was taken over a persisted table with no run predicate, so two callers read each other's paths and the reported VaR drifted further from the truth on every invocation. That is a wrong number that looks plausible, which is the worst kind.
3. It wrote 100 000 [`oracle/02_monte_carlo.sql`] rows per call from a route reachable without authentication — a way for anonymous traffic to fill the tablespace of an Always Free instance.
4. The path cap is enforced **twice**: at 50 000 [`oracle/02_monte_carlo.sql`] in PL/SQL and again in the route. The route is the layer a future contributor can edit and an attacker cannot; the database is the layer neither can. Duplicating the limit means neither one alone is load-bearing.

Reading the disagreement correctly

The panel compares the simulated figure against the **closed form of the same model** — the lognormal quantile, not a normal approximation:

$$Q_{0.01}(S_T) = S_0 \exp\!\left(\left(\mu - \frac{1}{2}\sigma^2\right)T - z_{99}\sigma\sqrt{T}\right), \quad z_{99} = 2.3263478740408408 \tag{43}$$

This is worth the paragraph it costs, because the first version of the panel compared the simulation against $z_{99}\sigma\sqrt{T}E$, the **zero-drift** normal shortcut, while sending the procedure a modelled 8% annual drift. At a 30-day horizon the drift term alone accounted for the whole -22% [`web/lib/portfolio-risk/risk.ts`] "divergence" the panel then flagged as an input error. Same drift, same volatility, same lognormal quantile, same 365-day year: what remains between the two figures is sampling error, which is the only thing a divergence tile is entitled to measure. The comparison reads the **echoed** assumptions off the response rather than the component's own request, because the route clamps its inputs and a clamped input read against the unclamped original resurfaces as method disagreement.

The two VaRs are not interchangeable and the surface says so. This one is a terminal-value figure over a horizon; the covariance one is a one-bar figure on the current book. A terminal-value VaR says nothing about the path, so it is not comparable to a maximum-drawdown figure and must never be presented as one. Presenting them as one number with two sources would be the actual error.

3.3 The bootstrap's resamplers, and what clustering is worth

Two resamplers are offered, named identically on both stacks so a run cannot be called one thing on the chat card and another on the workspace card.

The i.i.d. draw takes each bar independently. Its limitation is stated in the code rather than in a footnote: it has no volatility clustering, so it understates a sustained drawdown where losses arrive in runs.

The stationary bootstrap (Politis and Romano, 1994) draws blocks of geometric length with expected size L: at each step, with probability $1/L$ a new block starts at a uniform position, otherwise the cursor advances by one and wraps modulo m. The wrap is what makes it stationary — without it, end-of-sample bars are systematically under-drawn. Left unspecified, L is the $\sqrt{N}$ heuristic clamped to $[5, 100]$ bars:

$$L = \min\!\left(100, \max\!\left(5, \left\lfloor \sqrt{N} + \frac{1}{2} \right\rfloor\right)\right) \tag{44}$$

written as $\lfloor x + 1/2 \rfloor$ rather than as a rounding call because Python rounds halves to even and ECMAScript rounds them up — a difference no integer square root can actually reach, spelled out so the two stacks stay provably one rule rather than two that happen to agree.

Three properties of this pair are design decisions rather than incidentals:

A contradiction raises rather than resolving itself. Asking for an i.i.d. draw with a mean block of ten bars, or a stationary bootstrap with a block of exactly one, refuses. Silently picking a winner would produce a run that used the other resampler with no card able to report it afterwards.

The i.i.d. path is a separate loop, deliberately. The block loop consumes two random values per step (a uniform to decide whether to start a block, then an index) where the i.i.d. loop consumes one. Routing $L =$

1 through the block loop would produce a different sequence for the same seed, so every existing figure would move, silently, with no code that looks like it changed a number.

The seed defaults to `zlib.crc32` of the input series, so a refresh against the same book redraws the same cone — reproducible without stored state.

What the clustering premium is worth on this book is not measured. The repository pins the two resamplers against each other for naming, block derivation, contradictions and loss-quantile rules (5, 6, 2 and 3 cases respectively [`web/tests/fixtures/mc-resampler-parity.json`]), but nothing in the tree records a measured ratio between a stationary-bootstrap tail and an i.i.d. tail on real book returns. A plausible figure could be asserted here in one line and it would be exactly the defect this document is built to avoid. Directionally the block draw produces the **fatter** terminal tail because runs of losses survive resampling; a figure such as 15% wider CVaR_{95} at $L = 8$ [illustrative] would be an invention, and is marked as one.

3.4 Tail-risk stress testing and scenario construction

A VaR is an estimate from history. A stress test is a hypothesis about a future that history has not shown, and the two answer different questions on purpose.

Four scenarios are named in code — a leveraged liquidation cascade (majors gap together, correlation goes to one), a broad risk-off, a melt-up (worth running precisely because a short book fails there), and a flat baseline whose only purpose is that any non-zero P&L on it would be a bug in the propagation. Each is a shock table mapping symbols, plus an optional `*` wildcard, to fractional moves.

The propagation rule is where the honesty lives. For each held position, the applied move δ_s is decided in four mutually exclusive ways, and the leg records **which**:

Basis	Move applied	What it means
`explicit`	δ_s as given	the scenario named this instrument directly
`beta`	$\beta_s \cdot \delta_{\text{ref}}$	co-movement measured against the reference over ≥ 20 aligned bars
`wildcard`	the `*` move	an assumption about an instrument whose co-movement could not be measured
`unsupported`	0	no measurable beta and no blanket move: left flat, and the total is understated rather than invented

with $\beta_s = \mathrm{cov}(r_s, r_{\text{ref}})/\mathrm{var}(r_{\text{ref}})$, returning a typed absence — not 1.0 — when it cannot be estimated. Defaulting an unmeasurable beta to one is the quiet way a stress test starts inventing exposure and reporting it as a measurement: every unmeasurable instrument would move exactly with the shocked one, and the resulting number would look like evidence.

The `wildcard` and `beta` bases are kept apart for the same reason. A blanket "everything falls 25%" is a legitimate thing for a scenario to mean, and it is a **stronger** assumption than $\beta = 1$ would have been, so the leg says so rather than being folded into the beta count.

On the operator's side, a hand-set shock of zero survives conversion rather than being dropped. "This instrument does not move" is a hypothesis an operator can state, and it is a different hypothesis from "propagate this one by its beta" — the first is a pinned zero, the second is an omission, and collapsing them would quietly delete a position's exposure from the total.

Time to liquidate

A stress test prices the damage; the liquidity view prices the exit. For each leg, against average daily volume ADV derived from the same OHLCV every other risk number is derived from, and a participation cap ρ defaulting to 0.10 [`web/lib/liquidity.ts`]:

$$D_i = \frac{|N_i|}{\rho \cdot \mathrm{ADV}_i} \tag{45}$$

Bands are `liquid` at $D \leq 1$ session, `moderate` to 3, `illiquid` beyond, and `unmeasurable` when ADV is absent or rests on fewer than 20 [`web/lib/liquidity.ts`] observations. The book's figure is the **maximum** over legs, not the mean: a book exits at the speed of its slowest position. Concentration of that exit risk is reported as a Herfindahl index over each leg's share of total liquidation days, which distinguishes "three positions each needing a day" from "one position needing three".

3.5 Factor exposures

Two decompositions answer two different questions, and neither substitutes for the other.

Risk contribution answers "which position carries the volatility". Share of notional is not share of risk, and the gap is the reason the decomposition exists. The standard Euler decomposition on the marginal vector $\Sigma \boldsymbol{w}$:

$$\mathrm{MCR}_i = (\Sigma \boldsymbol{w})_i, \quad \mathrm{CTR}_i = w_i (\Sigma \boldsymbol{w})_i, \quad \sum_i \mathrm{CTR}_i = \sigma_p^2 \tag{46}$$

A 13% sleeve in a volatile name can carry more risk than a 42% one in a quiet name, and a short that hedges the book contributes a **negative** amount — a number a notional-weighted view cannot produce at all. Beside it, the diversification ratio

$$\frac{\sum_i |w_i| \, \sqrt{\Sigma_{ii}}}{\sigma_p} \tag{47}$$

says how much of the book's calm is real and how much is one position.

Factor loading answers "is this edge generic". Three factors are constructible from a single instrument's bars: market (buy and hold), time-series momentum (long after a positive trailing 30-bar return, short after a negative one), and a volatility-regime factor (long while trailing volatility is below its expanding average, short above). Two disciplines make the loadings falsifiable:

The factors are executed with the same one-bar lag as the strategy. Regressing a strategy that trades at $t+1$ against a factor that trades at t credits the factor with information the strategy never had, and the resulting beta is an artefact of the timing mismatch rather than an exposure.

The volatility threshold is an expanding mean, not a full-sample median. A full-sample threshold would let the factor know at bar 100 what volatility looks like at bar 1900. That is look-ahead in its **insidious** direction: a benchmark built with hindsight is stronger than one anybody could have traded, so the strategy's alpha against it comes out too low and a real edge can be argued away by a factor that could not have existed. Bar i's own volatility is excluded from its own threshold, because a one-observation leak with a tidy explanation is still a leak.

3.6 The pre-trade limit ladder

Every order passes one choke point that can say no in microseconds. Seventeen gates are defined; a crypto order can reach fifteen of them. The declaration lives in one tuple that three things read and must agree on — the parity harness, the Postgres enum mirror, and a test that harvests the names the compiled `submit` method actually emits and asserts they are exactly that tuple.

#	Gate	Predicate	Applies
1	`kill_switch`	halt not engaged — one boolean, always first	always
2	`symbol_halt`	this instrument not individually suspended	always
3	`symbol_whitelist`	symbol in the live L2 universe, or backed by a trusted quote	always
4	`paper_execution_model`	`MARKET` only: a quote is a price, not a book	paper equity
5	`reference_freshness`	quote age within bound, and not dated into the future	paper equity
6	`duplicate_order`	`client_order_id` unseen	always
7	`rate_limit`	token bucket admits one order	always
8	`price_available`	a live mark exists	always
9	`order_sized`	quantity and notional both derivable	always
10	`max_order_notional`	$N \leq 50\,000$ USD	sized
11	`symbol_concentration`	projected per-symbol $\leq 150\,000$ USD	sized
12	`gross_exposure`	projected book-wide $\leq 500\,000$ USD	sized
13	`price_band`	limit within 500 bps of mark	`LIMIT`
14	`working_book`	resting book below 200 orders	`LIMIT`

15	`daily_drawdown`	session loss $< 5\%$ of start-of-day equity	always
16	`reduce_only`	order reduces the position	defensive regime, sized
17	`est_slippage`	routed slippage ≤ 75 bps	sized, routable

All limits are defaults [`Part2_Infrastructure/config.py`], overridable by environment variable at deploy time only: `Settings` is a frozen dataclass, so changing a hard limit requires a deploy and therefore a code review. A limit a compromised service can mutate at runtime is not a limit.

The order of evaluation, and why it is that order

Cheapest first, and cheap here means "reads the least state". Rows 1 to 3 are single boolean or set-membership reads on state already resident. Rows 6 and 7 touch small bounded structures. Rows 8 to 17 need a consolidated mark, which is the first thing in the battery that has to fold across venues. Row 17 needs a full merged-ladder walk and is last for that reason.

Two ordering decisions are worth defending. `kill_switch` is first because it is the gate that must never be reached late — an order that is halted should not have spent a token or a mark on the way to being told so. And `est_slippage` is last because it is the only gate whose cost scales with book depth; putting it earlier would make every rejected fat-finger order pay for a ladder walk it was never going to use.

Halting semantics

The battery does **not** short-circuit. This is the property most often assumed wrongly about a gate vector, so it is stated precisely:

```
for each gate g in GATE_ORDER:
    if g applies to this order:
        checks.append(CheckResult(name, passed, detail, observed, limit))
        # the return value is not branched on

rejected_by = [c.name for c in checks if not c.passed]
accepted    = (rejected_by == [])
```

Three consequences follow, and each is deliberate.

A rejected order still reports every violation, not the first. A trader who fixes the notional and resubmits should not discover the price band on the second attempt and the drawdown budget on the third. The vector is evidence, and evidence that stops at the first problem is a worse post-mortem.

A gate that did not apply and a gate that passed are different facts. The returned vector is the gates that **ran**, not a fixed-length row. Nine of the seventeen are conditional on the order in front of them: the two paper-equity rows need a quote; rows 10 to 12 each price a size and are skipped entirely when the order could not be sized — which is exactly the feed-outage case, where a `MARKET` order carrying a quantity but no live mark runs eight gates and never reaches a notional limit because there is no notional to compare. Collapsing "did not apply" into "passed" would be the same mistake as reporting a missing measurement as zero.

One gate mutates, and its position in the vector is therefore semantic. The rate-limit check **consumes** a token, so it runs exactly once and it runs even when an earlier gate has already failed: a rejected order still spends its token, because the bucket is defending the venue against request volume, not against accepted orders. The idempotency key behaves oppositely — a `client_order_id` is recorded only on acceptance, so a rejected order does not burn its identifier and a corrected resubmission under the same key is admitted.

The whole battery runs under one lock with one consolidated mark memo per symbol, cleared in a `finally` block so a raising gate cannot leave a monitor loop reading a mark frozen at a failed decision. Post-decision side effects — the audit write, the alert, and the automatic breaker check — run **outside** the lock, so an audit backend under pressure cannot extend the time the book is held.

The graduated regime

Between the soft threshold and the hard breaker the desk may still close positions but not open or add to them. With drawdown d, limit $D = 5\%$ and threshold $\theta = 0.80$, the regime is active when $d/D \geq \theta$, and an order is **reducing** when it is opposite in sign to the holding and no larger than it:

$$\text{reducing} = (\text{held} \neq 0) \wedge \big(\text{sign}(\text{held}) \neq \text{sign}\big(q_{\text{signed}}\big)\big) \wedge |q_{\text{signed}}| \leq |\text{held}| + \varepsilon \tag{48}$$

An over-sized “close” that flips the book is an opening trade in disguise, which is why the magnitude condition is there. An unsized order is refused rather than assumed either way: deriving the sign from a defaulted zero treated a missing quantity as reducing on a long book and not on a short one — the same order, two answers. Reduce-only also reaches the **resting** book. A limit order placed before the threshold was crossed does not know about it, and one that fills afterwards makes the book bigger, which would make the regime a claim rather than a control. The sweep therefore cancels resting orders that would add risk when the regime engages. The same argument applies to the hard halt: a halt that does not reach the resting book is not a halt.

A known hole, left open deliberately and documented where it lives. The drawdown fraction is floored at zero and divides by start-of-day equity, so a non-positive baseline disables both guards — an account that opened the session already wiped out reports “no drawdown” however much further it falls. It reads as good news, which is the class of wrong number nobody reports. It is not patched unilaterally because the browser holds a bit-for-bit mirror of the expression for the sandbox desk, and a Python-only clamp would make the sandbox and the gateway disagree about whether an order is blocked. The honest repair is a shared decision about what a fraction of a non-positive denominator **means**, taken across both implementations at once.

Two engines, one battery, twenty scenarios

The arithmetic exists three times: the Python reference, a TypeScript mirror for the browser, and a C++ core (pybind11) that owns the book ladders and every gate that is arithmetic. Between Python and C++ the standard is not a tolerance but **bit-exactness**. Twenty named scenarios are recorded from the reference into `web/tests/fixtures/gate-parity.json` [20 scenarios, version 1], and both engines must reproduce the same accept/reject verdict, the same gate order, and the same `observed` and `limit` doubles:

`happy_market`	**`happy_limit_resting`**
`kill_switch_on`	`symbol_halted`
`not_whitelisted`	`duplicate_client_id`
`rate_limited`	`no_price`
`oversize_notional`	`concentration_breach`
`gross_breach`	`price_band`
`working_book_full`	`slippage_breach`
`slippage_partial`	`paper_equity_happy`
`paper_equity_limit_rejected`	`paper_equity_stale_quote`
`drawdown_reduce_only_allows_close`	`drawdown_reduce_only_blocks_opening`

Getting to bit-exactness surfaced three silent-wrongness traps, each now pinned by that fixture or by a differential test:

1. CPython’s `sum()` uses Neumaier compensated summation, so a plain C++ `+=` fold lands one ULP off. The core reproduces each fold with the **matching** algorithm — compensated where Python used `sum()`, plain where it used an explicit loop — and the routed walk needs both in the same function.
2. FMA contraction fused a multiply-add one ULP off even under `-ffp-contract=off`, until pinned with `#pragma STDC FP_CONTRACT OFF`.
3. Python’s `list.sort` is stable **and stays stable under** `reverse=True`, so two venues quoting the same price fill in feed-iteration order. Getting that tie-break backwards moves the blended VWAP by a ULP; a randomised differential test over 400 cases [`docs/architecture/LATENCY_BUDGET.md` §2.1] on a shared price grid, of which 106 of the 125 multi-venue cases [`docs/architecture/LATENCY_BUDGET.md` §2.1] diverge if the tie-break is reversed, is what holds it.

3.7 Three latency planes, never blended

The single most common way a latency claim becomes untrue is by blending units. This system publishes three figures about three different things, in three different units, and every surface that shows one names which.

Plane	Unit	What it measures	Measured
Decision	µs	the whole of `submit()` under the lock, including the check vector and the response object	13.2 µs p50 native, 25.3 µs Python [`latency-bench.generated.json`, 2026-08-20]

Core	ns	the compiled arithmetic battery, timed by `steady_clock` inside the engine	83 ns p50, 84 ns p99 [same run]
Network	ms	order entry to the venue's matching engine	72.7 ms Binance origin, 6.2 ms Bybit origin [`tools/colocation_probe.py`, OCI Singapore]

The ratio is the argument. The compute is 0.02% [`docs/architecture/LATENCY_BUDGET.md`] of the path, and no further optimisation of it changes the system's latency in any way a trader could observe. Optimising the gate battery from 54.5 µs to 50.3 µs improved end-to-end by 0.006% [`docs/architecture/LATENCY_BUDGET.md` §2.3] moving the whole battery into C++ afterwards improved it by another 0.01% [same]. Both were done for what they prove about the compute, and neither is claimed as a win a trader would notice. The only lever that moves the number by orders of magnitude is geography.

```
72 700 us   order entry to Binance, origin   <- 5 860x the decision
69 100 us   market data from Binance         <- 5 570x the decision
 6 160 us   order entry to Bybit, origin     <-   496x the decision
  12.4 us   the risk decision (native p50)
  0.08 us   the arithmetic core inside it
```

Three properties keep the planes from contaminating each other:

The core figure is quantised, and the honest statistic is the fraction, not the percentile. `steady_clock` on the development machine advances in 41.677 ns [`latency-bench.generated.json`] steps and `duration_cast` truncates, so 83 ns **is** two ticks and 125 ns **is** three. Quoting "84 ns p99" alone would publish a rounding artefact as a speed-up. The figure with the resolution is the fraction of calls finishing inside two ticks: 0.9952 of 5 000 [`latency-bench.generated.json`], across nine runs ranging 0.9932 to 0.9976.

The µs histogram is log-linear over the whole process life, never a sliding window. Eight linear sub-buckets per power of two gives about 12% resolution; quantiles are nearest-rank over bucket upper edges, **clamped to the observed maximum** — an unclamped p99.99 once reported 1152 µs against a real maximum of 1125 µs, and a quantile above the slowest decision ever recorded is not a rounding artefact to explain in a footnote, it is a number that cannot be true. A sliding window is refused because "we had a 4 ms decision last night" is exactly the fact a window is designed to forget.

Synthetic samples never enter the µs plane. The gateway times the compiled battery once at startup on a synthetic two-venue book — 50 warm-up calls unrecorded, then 300 [`docs/architecture/LATENCY_BUDGET.md` §2.1] recorded — so the nanosecond figure exists before the first order. Those samples land only in the core histogram, are counted separately as `core_self_test_samples`, and touch neither the decision histogram, the order counters, the audit log, the token bucket nor the TCA engine. The desk chip reads "no orders yet" for the µs plane while showing a real ns figure beside it, and names the self-measure as its provenance.

What the published figures exclude, and cannot include on this hardware. There is no NIC hardware timestamping and no PTP source on a cloud VM, so every figure is in-process: it excludes the kernel network stack, the driver and the wire. A published in-process number is a floor on the real latency, never the real latency.

3.8 The consolidated book and the liquidity surface

Two venues are streamed over WebSocket: Binance as `@depth20@100ms` and Bybit as `orderbook.50`. The transport choice per venue is not stylistic. Binance's diff stream requires a REST snapshot plus buffered-delta reconciliation and silently corrupts the book if one message is dropped; the partial stream is self-healing because every message is a complete top-20 snapshot. Bybit's feed **is** sequence-tagged — `u` increments by exactly one per delta — so it is consumed as snapshot plus delta, and any other step is a gap that forces a resubscribe rather than trusting a book that may have holes.

The reference price is a depth-weighted consolidated mid, not a simple average:

$$M = \frac{\sum_v m_v w_v}{\sum_v w_v}, \quad w_v = \max\left(1, \mathrm{depth}_v^{\mathrm{bid}(5)} + \mathrm{depth}_v^{\mathrm{ask}(5)}\right) \tag{49}$$

A single venue's mid is unstable when that venue is thin; weighting by top-five depth gives a reference that does not jump when one book momentarily crosses. The **touch** is a separate query and deliberately not the

same object: a resting limit order crosses when somebody is actually showing a price through it, which is the best bid or offer anyone displays, not a stable weighted reference.

Three surface properties are reported rather than smoothed away. A venue whose book has not updated within its staleness clock is excluded from pricing before it can poison a fill estimate. Crossed consolidated books are shown, not clamped: across venues, best bid can genuinely exceed best ask for tens of milliseconds, and that is exactly the signal a cross-venue arbitrage desk watches. And when every feed is unreachable, a synthetic random-walk book keeps the system demonstrable while every payload derived from it carries `synthetic: true` and the surface marks it.

3.9 Smart order routing

The router merges every online venue's levels for the taking side into one ladder sorted by price, and walks it greedily against the target notional N, capping each level's contribution at its own notional $p_l q_l$:

$$\text{take}_l = \min\left(p_l q_l,\, N - \sum_{j<l} \text{take}_j\right), \quad Q = \sum_l \frac{\text{take}_l}{p_l}, \quad \text{VWAP} = \frac{\sum_l \text{take}_l}{Q} \tag{50}$$

Proposition. For a target notional fully absorbable by the merged ladder, the greedy price-ordered walk maximises the filled quantity Q, and therefore minimises the blended VWAP $= N/Q$.

Argument. Any admissible allocation spends N across levels subject to $0 \le n_l \le p_l q_l$, yielding $Q = \sum_l n_l/p_l$. The objective is linear in n_l with coefficient $1/p_l$, which is largest at the smallest price, so the optimum fills levels in ascending price order until N is exhausted — exactly what the greedy walk does. Because VWAP $= N/Q$ with N fixed, maximising Q minimises VWAP. The per-venue split of that one walk **is** the routing instruction.

The assumption in that proposition is named rather than hidden. It holds because level notionals are independent and no venue-specific fee enters the objective. This router models no per-venue fee differential and no size-tiered schedule; fees are applied at fill time as one flat rate. On a live desk with asymmetric maker/taker schedules the price-only optimum is not the cost optimum, and that is an absence in this implementation, not a property of the method.

The exit condition and the fillable verdict both run against a relative tolerance rather than an exact comparison:

$$\text{absorbs}(f, N) \equiv f \ge N(1 - 10^{-9}) \tag{51}$$

A ladder walk reaches the requested notional by subtracting one level at a time, so the total lands a few ULPs either side of the request, and a request not on a cent boundary never matches a figure quantised to cents anywhere along the way. Deciding fillability with a bare $\ge$ reported "this book cannot absorb the order" for orders the book demonstrably absorbed — the repository records a `SELL` of 99.95002498750625 units rejected with "only $10,095 of $10,095 routable", the two figures identical to the dollar, while the same order at exactly 99.95 went through. The tolerance is **relative** because this engine prices instruments from cents to tens of thousands: a fixed dollar epsilon is wrong at one end or the other. And the direction of the error matters more than its size — a false accept releases an order into a book that cannot fill it, whereas a false reject is cosmetic, so the tolerance is sized to absorb arithmetic noise and nothing more. No caller may substitute the requested notional for the measured one to make the comparison pass; that would make the gate a tautology.

The gate prices what the fill will do. The liquidity check runs against the routed execution, not the best single venue. Gating on the best venue alone would reject orders the router could actually fill and understate cost on the ones it accepted, and a dedicated test pins the slippage check and the fill price to agree.

3.10 Transaction cost analysis

The per-venue and routed estimates are reported side by side with the saving the route achieved against the **worst** fillable venue, in both bps and dollars — against the worst rather than the best, because the saving a router is responsible for is the one it avoided, and quoting it against the best venue would report a number near zero on every well-behaved book.

Slippage on any walk is measured against the consolidated mid at decision time:

$$\text{slip}_{\text{bps}} = \frac{\text{VWAP} - M}{M} \times 10^4 \quad \text{(BUY)}, \quad \frac{M - \text{VWAP}}{M} \times 10^4 \quad \text{(SELL)} \tag{52}$$

For research rather than execution, the cost model is explicit and its concavity is a **modelling choice a researcher makes**, not a fact a backtest discovered:

$$c_{\text{turnover}} = \frac{\text{fee}_{\text{bps}} + \text{slip}_{\text{bps}}}{10^4} + k\sqrt{\min(1, Q/\text{ADV})} \tag{53}$$

The square-root law is the standard concave form — doubling order size costs about 1.41 ×, not 2 ×, because a larger order is worked over more of the book. The impact coefficient defaults to zero, so an unconfigured run produces exactly the numbers the parity fixture pins, and a configured one is labelled as a model on the surface that shows it. Holding costs are separate: perpetual funding is charged on absolute exposure pro-rata to bar length against an 8-hour period, and borrow only on short exposure pro-rata against a year.

3.11 The order ticket and the paper-execution path

The ticket collects symbol, side, type, size (as quantity or notional), an optional limit price, time in force and an optional `client_order_id`, and returns the decision object with its full check vector, the failing gate named, the latency and the fill. Three presets exist because a demonstration that requires typing a plausible-looking bad order is a demonstration nobody runs: a valid $25k order, a $500k fat finger blocked by the per-order cap, and a twelve-order $1k burst that the token bucket stops partway through.

Time in force has no default of its own; the sensible default differs by order type, and is `GTC` for a `LIMIT` and `IOC` for a `MARKET`. Every client written before resting orders existed therefore behaves exactly as it did.

Three fill models, and why they are three

Taker. A market order, or a limit that crosses the spread, fills at the smart route's actual VWAP and pays `PAPER_FEE_BPS` (4 bps [`Part2_Infrastructure/config.py`]). Filling at mid is the single most common way a paper system flatters itself; walking the ladder means paper P&L carries live cost structure.

Maker. A resting order fills when the consolidated touch crosses it, at **its own limit price**, and pays `PAPER_MAKER_FEE_BPS` (1 bp [`Part2_Infrastructure/config.py`]). It is on the other side of that trade — somebody crossed the spread to reach it. Charging it a taker fee, or filling it at a route VWAP that walked **through** its own limit, would report a cost the desk did not pay. Its measured slippage against the mark is therefore often negative, which is price improvement, and which makes maker-versus-taker economics visible in the blotter rather than a footnote.

Paper equity. An order priced from a trusted vendor quote rather than an L2 ladder fills at $q(1 + \delta \cdot s/10^4)$ with $s = 8$ bps [`Part2_Infrastructure/config.py`] fixed, and the gate vector says so: no exchange depth is asserted, and the path accepts `MARKET` only because a quote is a price and not a book.

Resting orders and the state that is deliberately absent

Five states cover a resting order's whole life: `WORKING`, `FILLED`, `CANCELLED`, `EXPIRED` and `REJECTED`. A `DAY` order dies at the UTC session boundary; an `IOC` with nothing to be immediate against is accepted — it passed every gate — and immediately `EXPIRED`, which costs no machinery at all to say.

`PARTIALLY_FILLED` is absent, and its absence is load-bearing. The L2 feeds carry ladder snapshots, not trade prints, so how much of a resting order a crossing trade consumed **cannot be measured from this data**. A state that can never be reached honestly is a state that advertises a model this system does not have.

Committed capital is exposure whether or not it has landed. The concentration and gross gates therefore price the resting book into their projections; without that, two $140k orders each pass a $150k cap, both fill, and the book sits at 187% of a hard limit with no gate having fired.

3.12 Fill quality

Realised cost is measured against the same reference the gateway priced the decision at, which makes the effective spread exact rather than a stand-in:

$$\text{eff}_{\text{bps}} = 2|\text{slip}_{\text{bps}}|, \quad \text{fee}_{\text{bps}} = \frac{\text{fee}_{\text{USD}}}{|N|} \times 10^4 \tag{54}$$

Both are null-in, null-out. A fill nobody priced has no effective spread, and zero would claim it traded exactly at the mid. Per-venue means are taken over the count that **has** the measure, never the fill count, because averaging over rows that carry no figure drags every mean toward zero. Fills carrying no venue tag are reported as `unattributed` rather than absorbed, so the venue mix and the headline fill count reconcile on screen.

Realised spread is withheld, visibly. Separating impact from reversion needs the consolidated mid a few minutes **after** each fill. The gateway records mids in its snapshot table but publishes no endpoint that serves them by timestamp, so no post-trade reference exists on this surface. The column is drawn empty rather than at zero, and that empty column is the point of the chart rather than an apology for it: a two-bar chart of

impact and fee would look complete and quietly imply that cost has been fully decomposed. A spread measured at zero and a spread nobody measured are opposite claims.

Price improvement is counted directly — the share of priced fills whose signed slippage is negative, with the mean improvement over that subset — and the rate is null when there are no priced fills at all, because "no fills" and "no improvement" are different facts.

3.13 The decision-latency distribution on the desk

A p50 and a p99 describe two points; the shape between them is where a bimodal gate battery or a long tail shows up, so the desk draws the histogram as bars rather than a line — these are counts per bin, and a line between bin centres would imply a continuum the histogram is deliberately discretising. Below the sample floor it renders the count instead of a chart, because a distribution over a handful of decisions is a picture of nothing.

The same discipline governs the header chip. Its vocabulary distinguishes `checking`, `no gateway`, `not published`, `no orders yet` and `measured`, each carrying its own reason, and the p99.9 is withheld below 1 000 samples [`web/lib/decision-plane.ts`] because $\lceil 0.999 \times 1000 \rceil = 999$ — beneath that, the p99.9 is the maximum wearing a decimal point. The chip also names which engine answered, so a container that fell back to the Python reference is visible on the desk rather than only in a warning nobody reads.

The tail beyond p99.9 is not the language. Disabling the cyclic garbage collector around the same workload changed p50 not at all and p99 by about 0.1 µs [`docs/architecture/LATENCY_BUDGET.md` §2.2]. The core's own far tail says the same thing from the other end of the scale: about six samples per 5 000 land at 10 to 23 ticks, and they arrive in **bursts** — one slow call immediately following another — which is the signature of preemption on a shared hypervisor, not of a branch in the decision.

3.14 What is not built, and why it waits

Absent	Reason it waits
Live venue order entry	Execution is paper-only. Fills are priced off the live ladder, but nothing is sent to an exchange; the surface says so on every fill.
`PARTIALLY_FILLED`	Unmeasurable from snapshot feeds. See Section 3.11.
Realised spread	No endpoint serves a historical mid by timestamp. The column is drawn empty. See Section 3.12.
Per-venue fee schedules in the router	The route optimises price only. Naming the omission is cheaper than a router that looks fee-aware and is not.
A measured clustering premium	The two resamplers are pinned against each other for naming and rules, but no ratio between their tails on real book returns has been run. See Section 3.3.
The parametric CVaR parity pin	The two stacks carry expected-shortfall constants that differ in the ninth significant figure and nothing cross-checks them. See Section 3.1.
Co-location in the venue's region	Not done, and the 68 ms to 0.1–0.5 ms figure is an expectation, not a measurement. The plan is probe-first: stand up an instance, run the same probe, migrate only if it confirms.
Hardware timestamping	Neither cloud tier exposes NIC timestamping or PTP, so no true tick-to-trade measurement is possible here and none is claimed.

Every row above is a capability a competitor's document would omit. They are listed because the alternative — inferring absence from silence — is how a reader ends up trusting a number that was never measured, which is the single failure this system, and this chapter, are built to prevent.

4 Data, reliability and the developer plane

Three roles share one property the preceding chapters did not need: they are answerable for the system when it is **wrong**. A data engineer owns the claim that the number on the screen came from somewhere; a reliability engineer owns the claim that the system said so at the time; a quant developer owns the claim that two independently deployed units, written in two languages, agree about what the number is. This chapter is the machinery behind those three claims. It is deliberately specific about thresholds — a staleness rule with no number in it is a sentiment, not a control — and equally specific about where each is set, because a constant nobody can find is a constant nobody can change.

4.1 Part A. The data engineer

What "stale" means, and where it is set

The system holds two independent staleness thresholds because it has two independent ingest paths for the same L2 data, and merging them would be a lie about topology: the gateway subscribes to venue WebSockets in a long-lived Python process, and the browser subscribes to the **same venues directly**, one hop, so the order book on screen is not a relay of a relay.

Plane	Constant	Default	Where it is set
Gateway feed	`VENUE_STALE_AFTER_S`	10.0 s	`config.py:110`
Browser ladder	`STALE_AFTER_MS`	8 000 ms	`web/lib/livebook-socket.ts`
Ops snapshot	`SNAPSHOT_STALE_AFTER_SECONDS`	65.0 s	`modules/operations.py`
Provider health	`PROVIDER_HEALTH_STALE_AFTER_MS`	60 000 ms	`web/lib/reliability.ts`
Shared ops overlay	`SHARED_STALE_MS`	90 000 ms	`web/lib/observability/ops-ledger.ts`
Vendor quote contract	`FRESHNESS_LIMIT_MS`	24 h	`web/lib/providers/contracts/shared.ts`

The gateway's rule is one line of `BookState`: a book is stale when `age_s > settings.venue_stale_after_s`, where `age_s` is $t_{\text{now}} - t_{\text{last update}}$ in wall-clock seconds and is $+\infty$ for a book that has never updated. Formally, for venue v and symbol s,

$$\text{stale}(v, s, t) = \big[\, t - \tau_{v,s} > \Theta \,\big], \quad \Theta = 10 \text{ s}, \quad \tau_{v,s} = -\infty \text{ until the first update} \tag{55}$$

That threshold is not decoration. It is read by the `reference_freshness` pre-trade gate on the paper-equity path, and `tests/test_tca_patch_points.py` exists to prove the gate reads `settings.venue_stale_after_s` rather than a copy of it; it is also carried on every parity scenario (`LIMIT_FIELDS` in `tools/gate_fixture.py`), so changing the default cannot silently change what the fixture pins. The 65-second snapshot budget is derived rather than chosen: the web tier polls every 30 s, so two missed polls plus a scheduling margin is the point past which a last-good observation stops being evidence — and the source states that derivation, which is the difference between a tunable and a magic number.

Hysteresis, and why the rule is asymmetric

A single threshold applied to a feed updating at roughly that threshold is an oscillator. The browser's venue-status strip demonstrated it: the predicate was recomputed on every 5 Hz publish tick and any arriving frame set the status straight back to live, so a thin instrument in a quiet hour flipped between `live` and `stale` several times a minute. That is not a badge problem — `useLiveBook` merges only the venues it currently calls live, so the consolidated book that prices an order gained and lost a side of liquidity on the same flip. A twitching badge and a twitching price are the same defect seen twice.

`VenueLiveness` (`web/lib/venue-liveness.ts`) makes the rule asymmetric. Let $u(t)$ be the count of parsed books received, τ the arrival time of the last, and m the value of u at the moment the venue was last marked stale ($m = \text{nil}$ when it is not). With $\Theta = 8000$ ms and promotion streak $k = 2$:

$$\text{status}(t) = \begin{cases} \text{stale} & \text{if } t - \tau > \Theta \\ \text{live} & \text{if } t - \tau \le \Theta \text{ and } m = \text{nil} \\ \text{live} & \text{if } t - \tau \le \Theta \text{ and } u - m \ge k \\ \text{stale} & \text{otherwise} \end{cases} \tag{56}$$

Demotion is immediate because silence past the threshold means the book on screen is not the book at the venue, and a stale ladder must not price an order. Promotion requires k further updates because one straggling frame is precisely what caused the flip. A feed genuinely updating at the threshold therefore settles on `stale` — which is the honest reading of a venue you hear from every eight seconds — rather than alternating. The value $k = 2$ [`web/lib/venue-liveness.ts`, `PROMOTION_UPDATES`] is the smallest number that cannot be satisfied by the single late frame, matching `PROMOTION_STREAK` in `desk-source.ts` for the same reason.

Two subtleties survive review if got wrong. The stale mark is armed inside `update()` — when a book **arrives** and closes a gap — not only inside the status read: arming it on observation alone would make the hysteresis depend on somebody having looked during the quiet period, and the only caller is a `setInterval`, which browsers throttle to roughly once a minute in a background tab. Reading the inter-arrival time at the arrival site makes the decision a property of the data rather than of the observation schedule. The mark is also re-armed on every silent tick rather than only when unset: a venue that goes silent, sends one frame and goes silent again would otherwise keep the mark from the **first** silence, and the second frame would satisfy a streak that was never about it.

Venue liveness is then lifted to feed health in the gateway (`modules/tca_engine/supervision.py`), where the classification is over the set of symbols a venue is carrying:

Status	Condition
`down`	not connected, or connected and publishing no book at all
`stale`	every book with data is stale
`degraded`	some but not all books are stale
`up`	no book is stale

and lifted once more to the platform level in `modules/operations.py`, where `market_data.status` becomes `critical` when no real feed is usable, `degraded` when a synthetic feed is active or any real feed is not `up`, and `disabled` when market data is switched off entirely. Four states, each with a named reason, and none of them is an exception or an empty list.

The provider registry, ranking and the failover chain

The research and reference-data path is a different problem from the venue feeds: eight vendor adapters [`web/lib/providers/adapters.ts`, `ADAPTERS`], six of which need an environment value and two of which are keyless public APIs, with different licences, quotas and asset coverage. `web/lib/providers/` is the façade over them. Routes call `getQuote("AAPL")`; they do not name a vendor, do not learn which one answered, and do not change when a key is added or a provider fails.

Selection is by capability and asset class: `candidatesFor` filters the roster to adapters declaring that capability for that asset class and sorts by `meta.rank[capability]`, absent ranks last at 99. The result is a chain, and `dispatch` walks it, adding the **reliability policy** whose order is load-bearing:

1. **Simulated outage** — an operator knocked this provider out on purpose. Checked first so the reason shown is the one the operator caused; a deliberately disabled provider reporting "quota spent" sends someone hunting a problem that does not exist.
2. **Not configured** — no credential. An empty `keyEnv` declares a keyless public API, which is why a fresh clone with no secrets is still a working system.
3. **Circuit open** — recent consecutive failures; see **Circuit breakers** below.
4. **Unlicensed** — a capability-scoped refusal remembered from a previous 4xx, so the next dispatch skips without a call.
5. **Quota** — exhausted, or reserved against background traffic; see **The quota ledger** below.

Only then is a call made. Every refusal is pushed onto an `attempts` list that travels with the answer rather than being logged and dropped, because a failover the reader cannot see is a failover they will trust wrongly.

The error taxonomy on the way back out is equally deliberate. A thrown `ProviderError` carries a kind, and each kind has a different cost to the vendor's record:

Kind	Latency sample	Breaker	Meaning
`failed`	recorded, not ok	counts	the vendor broke
`no_data`	recorded, ok	no	the vendor correctly answered that there is nothing here
`unlicensed`	none	no	a refusal, not an answer
`quota`	none	no	a decline

Before this distinction existed, four “no profile for this symbol” answers made four healthy vendors read as degraded. The same distinction decides the status code when the whole chain is exhausted: if every provider that was actually asked answered `no_data`, the request is a 404 — the pool is healthy and the symbol is the problem — and anything else keeps the 503, because then a retry or a different key could change the answer.

A contract failure is treated exactly as a thrown error: the provider is failed, the breaker counts it, the chain moves on. The ordering inside the `try` block is the kind of bug that leaves no trace, so it is written down: `recordSuccess` **deletes** the breaker’s failure count, so evaluating the contract after it would clear the counter on every broken response and the breaker could never trip. A vendor emitting duplicated bar timestamps would be retried forever, burning quota, while the health matrix showed a zero per cent error rate.

Caching sits under all of it, a quota defence first and a latency optimisation second, with a TTL per capability because a fundamentals record is good for a day and a quote for seconds:

quote	bars	news	fundamentals	search	scrape
15 s	5 min	3 min	24 h	15 min	1 h

read from `TTL_MS` in `web/lib/providers/dispatch.ts`.

The quota ledger

Alpha Vantage’s free plan is twenty-five calls per **day**; Firecrawl’s is a thousand credits per **month**. Nothing about a naive integration warns you before a dashboard that auto-refreshes spends a day’s allowance before lunch. The ledger counts calls **before** they are made, and fences background traffic out of a reserve.

For an adapter with limit L, reserve fraction ρ and observed spend U in the current window, define $R = L - U$ and the reserve

$$R_0 = \lceil \rho L \rceil \tag{57}$$

. A request of priority p is admissible exactly when

$$\mathrm{admit}(p) = (R > 0) \wedge (p = \text{interactive} \vee R > R_0) \tag{58}$$

so background polling stops early and a human lookup at four in the afternoon still has budget. Windows are **calendar-aligned**, not rolling: vendors reset on calendar boundaries, and a rolling window would let the ledger believe it had budget on the first of the month that the vendor had already reset — and vice versa. The month key uses UTC, whereas most vendors reset on the account’s signup anniversary; that makes the count conservative near a boundary, which is the direction that fails safely.

Three properties were decisions against an obvious alternative. **Spend is counted before the call, not after**, because a request that times out still hit the vendor’s meter, and counting only successes under-counts exactly when the system is failing most. **The merged total replaces the local count rather than taking the maximum**: each serverless instance holds its own counter and the gateway merges them through the ops-sync round trip, so replacement is what lets an operator’s reset propagate instead of every instance re-asserting a stale high-water mark — spends between push and response are under-counted until the next sync, which is convergence rather than a race worth a lock. And **threshold warnings fire on the crossing, not on the condition**: comparing the count before and after each spend fires each of the three lines (0.5, 0.8, 0.95 [`web/lib/providers/quota.ts`, `QUOTA_THRESHOLDS`]) exactly once per window, because a log repeating “above 80%” twenty times is a log nobody reads.

The operator reset is honest about its limits: it zeroes **this ledger**, not the vendor’s meter. It exists because the ledger is a floor derived from one instance’s memory and can be badly pessimistic after a deploy — so the person pressing it needs to know they may be about to spend a real allowance.

Lineage, from vendor bytes to rendered number

`GET /api/system/inspect` answers a different question from `GET /api/quote`. The latter says what the price is; the former says how that number got here. It returns a seven-stage lineage, and it returns it **even**

when the lookup failed — HTTP 200 with `ok: false` and the full attempt list, because the trace is the product and a 503 would throw away the answer the caller came for.

```
Request          quote for BTCUSDT, classified as crypto, priority interactive
   |
Registry         4 candidates ranked for a crypto quote      [bybit, binance, fmp, ...]
   |
Cache            miss on quote:BTCUSDT                       (hit => no provider contacted)
   |
Reliability      alphavantage: quota_reserved; fmp: circuit_open
   |
Upstream         1 HTTP call, 0 failed                       (raw=1 retains the body)
   |
Adapter          Bybit answered in 11 ms
   |
Normalised       coerced to the shared quote shape;
                 provenance attached, NOT folded into the data
```

Listing 1: The seven lineage stages assembled by `web/app/api/system/inspect/route.ts`. The stage names and their order are read from that file; the values shown against them are an example trace [illustrative], not a captured run.

Two flags change behaviour rather than presentation. `refresh=1` evicts the cache entry first, because a debugger a cached value can satisfy is not debugging anything. `raw=1` retains the vendor's own JSON before normalisation — the difference between "the change field is null" and "the vendor renamed the change field and our parser silently coerced it".

Holding raw bodies safely is the interesting engineering. Two `AsyncLocalStorage` scopes exist: `trace.ts` for the opt-in inspector capture, and `raw-sink.ts` for the **always-on** raw-contract sink, because a contract check nobody switched on is not a check. Both refuse a module-level "current capture" variable — a route handler serves concurrent requests, and a bare global would attribute one provider's malformed payload to another provider's quarantine sample, a bug that appears only under concurrency, which is exactly when nobody is looking.

Redaction is by environment-variable **name** pattern (`/(_API_KEY|_APIKEY|_KEY|_TOKEN|_SECRET|_PASSWORD|_PWD)$/i`, with an allow-list for public identifiers) rather than a hand-maintained list of secrets, whose failure mode is a credential in a screenshot: two vendors carry the key in the query string and answer an authentication failure with an HTML page echoing the request URL, which the error path would otherwise quote into a public response.

L2 reconstruction and sequence-gap detection

The two venues are consumed under two different disciplines, and the choice is made per venue by what the venue's protocol can prove.

Binance is consumed as `<symbol>@depth20@100ms` (`modules/tca_engine/binance.py`, `web/lib/livebook-socket.ts`) — a self-contained top-20 snapshot every 100 ms. The diff stream would need a REST snapshot plus buffered-delta reconciliation, and it silently corrupts the book if a single message is dropped. The partial stream self-heals; the next frame is a complete book.

Bybit is consumed as `orderbook.50` (`modules/tca_engine/bybit.py`) snapshot plus deltas, because it is sequence-tagged and a dropped frame is **detectable**. Detecting it is the whole reason the incremental path is safe here.

The ladder is a price → size map per side, and the delta semantics are the reason deltas cannot be skipped: a level is removed by a delta carrying size zero. Writing B_t for the bid map at time t and δ for an arriving delta,

$$B_{t+1} = \{(p,q) : (p,q) \in B_t, p \notin \mathrm{dom}(\delta)\} \cup \{(p,q) \in \delta : q > 0\} \tag{59}$$

Now the gap test. Bybit increments `u` by exactly one per delta, so the correct predicate is

$$\mathrm{gap}(u_{\mathrm{prev}}, u_{\mathrm{new}}) = (u_{\mathrm{prev}} \neq 0) \wedge (u_{\mathrm{new}} \neq 0) \wedge (u_{\mathrm{new}} \neq u_{\mathrm{prev}} + 1) \tag{60}$$

The obvious-looking test $u_{\mathrm{new}} < u_{\mathrm{prev}}$ catches only a **backward** jump, which ordered TCP delivery makes impossible — the repository's note records that it never fired once in roughly ten thousand live messages, while a genuine forward gap sailed straight through into `apply()`. The consequence of applying a book with a hole is specific: deltas are the only source of level **removals**, so one dropped frame leaves a filled bid in the ladder

forever, sitting above the true ask. That is a permanently crossed book, and the UI reports it as a cross-venue arbitrage that does not exist.

Both zero-guards matter: a fresh subscription with no baseline, and a venue that omitted the field, must not be treated as gaps. On a real gap the gateway raises out of the read loop (`RuntimeError: bybit sequence gap on SYM: prev -> new`), which the supervisor turns into a reconnect and a fresh snapshot; the browser closes the socket for the same effect. Neither trusts a holed book.

Reconnection is exponential with jitter. With base d_0 and ceiling D:

$$d_{n+1} = \min(2d_n, D), \quad \text{wait}_n \sim d_n + \mathrm{U}(0, 0.3d_n) \tag{61}$$

with $d_0 = 1$ s, $D = 30$ s [`config.py`, `WS_RECONNECT_BASE_S` / `WS_RECONNECT_MAX_S`] in the gateway and $D = 20$ s [`web/lib/livebook-socket.ts`, `MAX_BACKOFF_MS`] in the browser. The browser adds one rule the gateway does not need: the backoff resets only once a socket has **proven stable** for ten seconds, not on handshake. Resetting on handshake defeats the ceiling on an accept-then-drop path — a proxy or a flapping venue completes the upgrade, drops, and every retry starts from one second again. The source records 54 reconnects in 60 s [`web/lib/livebook-socket.ts`] before that change.

An operator-forced restart is a distinct path from a failure reconnect, and every line of it repairs something the naive `ws.close(); open()` gets wrong: the pending retry timer is cleared (or two live sockets write into one ladder), handlers are detached before `close()` (or the dying socket's `onclose` schedules a third), the backoff resets (a reconnect somebody asked for is not evidence of instability), `seq` resets to zero (carrying the old session's sequence into a new one fails the gap test on the first delta and looks exactly like a venue outage in a reconnect loop), and the ladder is emptied — republishing the pre-restart book would be a stale price wearing a fresh timestamp. The venue drops out of the merged book until its next snapshot, and that gap is honest.

Inside the gateway the same ladders are mirrored into the C++ core. The mirror is refreshed in the two mutation funnels rather than maintained as a delta, so "the mirror is never stale" is a property of `BookState` rather than a rule every caller must remember. That funnel was measured and optimised: rebuilding the ladder mirror by sort-and-dedupe over a reused buffer instead of a hash map took `apply_snapshot` from 8.06 to 6.65 µs p50 [`docs/architecture/LATENCY_BUDGET.md` §2.1] and `apply_delta` from 5.06 to 3.40 µs p50 [`docs/architecture/LATENCY_BUDGET.md` §2.1], two rounds each, alternating builds at identical flags. The feed updates roughly sixty times a second per book while decisions are per order, so this is the right side of the boundary to spend on.

Schema validation and the quality ledger

Adapters already throw when a **primary** field is missing: a quote with no price fails loudly and the chain fails over. That covers the loud case. It does not cover the quiet one, which is the one that costs money — a bar series with a duplicated timestamp, a high below its low, a "live" quote stamped four days ago, a change field that silently became null when the vendor renamed it. Every one of those parses, validates, renders, and is wrong.

The contract suite (`web/lib/providers/contracts/`) evaluates expectations after normalisation and before anything is cached or shown, under three rules:

1. **Violations are attached, not thrown.** A stale timestamp does not justify discarding a price a trader can see is stale; it justifies saying so. Only `fatal` rejects, and `fatal` is reserved for data that is internally impossible.
2. **A check that cannot run is not a check that passed.** A provider that publishes no timestamp cannot fail a freshness check, and pretending it passed would make the least transparent vendor look like the most reliable. Such checks are listed in `notEvaluated`, which is carried on the wire and rendered, never folded into the pass count.
3. **Drift is reported separately from failure.** When a secondary field coerces to null while the rest of the payload is intact, the likely cause is a renamed vendor field, not a bad market.

Check	Severity	Why
`bars.prices_finite`	fatal	a null or non-positive price is not a bar
`bars.high_ge_low`	fatal	not a market condition, a broken record
`bars.unique_timestamps`	fatal	a duplicated timestamp double-counts a return and understates volatility

`bars.monotonic`	fatal	out-of-order bars backtest history backwards
`bars.no_gaps`	warn	irregular spacing points at a hole a strategy trades through
`quote.price_positive`	fatal	a quote with no positive price is not a quote
`quote.freshness`	warn	older than 24 h; `not_from_the_future` warns above 60 s ahead
`quote.change_derivable`	drift	null beside a present previous close

The gap check shows a threshold that had to be **measured** rather than reasoned. It compares the largest inter-bar interval to the median, not a count of gaps, because a count cannot separate a weekend from a hole. The original 3x fired on every US equity daily series the application can load — AAPL's largest gap is exactly 4.0x the median [`web/lib/providers/contracts/bars.ts`, recorded in the check's own comment], because a holiday Monday makes a four-day weekend and there are roughly nine a year. It is now 4.5x [`web/lib/providers/contracts/bars.ts`, `MAX_GAP_MULTIPLE`], which keeps every routine exchange holiday and still catches a genuine hole — a full missing week. A check that warns on every complete series is one a reader learns to scroll past, which costs more than the hole it was written to catch.

Flagged payloads go to a bounded quarantine ring — 50 records [`web/lib/providers/quarantine.ts`, `CAPACITY`] of 400 characters [`web/lib/providers/quarantine.ts`, `SAMPLE_CHARS`] each, redacted. It is a diagnostic buffer and not a data lake — an unbounded list in a long-lived process is a memory leak wearing an audit trail's clothes — and **a quarantined answer is never cached**, so the failover chain gets a chance at a cleaner source. The sample held is the **raw** body, not the normalised object, which is what the `raw-sink` scope exists to make possible.

Per-instance evidence is not enough: each serverless instance kept its findings in a per-lambda ring, so two polls landing on two instances described two different worlds and a restart forgot everything. Findings are now pushed through the ops-sync round trip the instance already makes and persisted in the gateway's durable quality ledger — four tables, `data_quality_findings`, `data_quality_escalations`, `data_schedule_runs` and `data_work_items`. Ingest de-duplicates on `(instance, seq)` and refuses both the stale and the future: a finding outside $[t - \text{retention}, t + \text{slack}]$ is a replay or a broken clock. Two rules then run on the window, per provider:

$$\text{fatal burst}: \quad F_{\text{fatal}} \geq 3 \quad \text{over a 15 min window} \tag{62}$$

$$\text{fail rate}: \quad N \geq 8 \quad \text{and} \quad \frac{F}{N} > 0.25 \tag{63}$$

with all five constants read from `config.py` (`DATA_QUALITY_ESCALATE_*`) and default retention of seven days. One escalation per `(rule, provider)` per 60 min cooldown [`config.py`, `DATA_QUALITY_ESCALATE_COOLDOWN_MINUTES`]. An escalation **auto-resolves when the condition clears** — it is not acknowledged away. Acknowledging is recorded and resolves nothing, which is the correct relationship between a human's attention and a machine's evidence.

Resolution carries a subtlety with an operational cost. `_resolve_cleared` originally ran only inside `ingest`, so an escalation cleared when the **same provider** sent more findings — and a provider that stops reporting entirely, which is what a badly broken one does, left its escalation open forever while the cooldown stopped a new one opening. The desk showed a permanent red against a condition that had ended. `resolve_loop` is the repair: an independent sweep, by default every sixty seconds, over a table holding at most a handful of open rows, doing nothing when there are none.

The backend is selected by `DATA_OPS_BACKEND`. Choosing `postgres` without credentials **raises at startup** rather than falling back, and an unknown value is refused for the same reason: a fall-back would leave a deployment reporting one backend and using another while the wire said `sqlite`. The aggregate read path is pinned across both — `_AGGREGATE` in Python and the `data_quality_rollup` view in SQL are asserted to produce the same six figures, because if one moves without the other both answer, neither errors, and they disagree.

Absence as a typed state

The rule that null is never coerced to zero appears three times in the data plane, each a different shape of one idea. **A check that could not run** is `notEvaluated`, counted and rendered apart from `passed`; with zero evaluated payloads the trust verdict is "Not yet proven — zero evidence is not a clean bill of health", not green. **A document with no embedding** is `embedding_status = "pending"`, counted as `pending_embeddings`

beside `indexed`, rather than written as an indexed document with a zero vector — a zero vector is a **point in the space** and would answer similarity queries.

A document that could not be delivered is counted **and kept**. A counter alone said that some documents had been lost and never which ones, so there was nothing to replay and no way to tell a rejected schema from an unreachable corpus. The dead-letter store publishes its depth, its discard count and a recent sample, because an operator needs to know there is something to replay before they can decide to.

4.2 Part B. The reliability engineer

Telemetry and the p99 tail

Two latency windows exist, deliberately not one, because they measure different things on different machines.

Window	Capacity	Time bound	Key space
Gateway HTTP	200 per route	900 s	60 routes, then `unmatched`
Web upstream	120 per key	15 min	per provider

read from `modules/metrics/request_latency.py` and `web/lib/observability/latency.ts` respectively. Both bounds are load-bearing on the gateway side. Without the time bound a single slow request during a cold start pins p99 for days on a quiet desk, and the alert that fires on it sends an operator hunting a problem that ended long ago. Without the route bound, an internet scanner hitting a few thousand distinct 404 paths adds a permanent series each — and `/metrics` is unauthenticated, so that budget is attacker-controlled. Unmatched paths are **aggregated** rather than dropped, because losing the fact that requests happened is worse than losing which path they were for. The `/metrics` route itself is excluded, so a scrape cannot perturb what it measures.

Quantiles are nearest-rank, in all three implementations:

$$q_p = x_{(i)}, \quad i = \min(n, \max(1, \lceil pn \rceil)) \tag{64}$$

over the sorted sample. Not linear interpolation: with n in the tens, interpolation invents a value no call actually took, whereas nearest rank always returns a latency some request genuinely experienced — a number an operator can go and find in the log. The Python implementation carries a scar worth repeating: an earlier `round(q*n + 0.5) - 1` looked equivalent to $\lceil qn \rceil - 1$ and was not, because Python rounds halves to even, so an exact qn returned the next index and p99 collapsed onto the maximum for any window under a hundred samples.

The tail is also governed by sample floors, which is the discipline that keeps a percentile from being theatre. A p99 is withheld below 20 samples [`web/lib/overview-latency.ts`, `LATENCY_MIN_SAMPLES`] and a p99.9 below 1 000 samples [`web/lib/decision-plane.ts`, `DECISION_P999_MIN_SAMPLES`] — $\lceil 0.999 \times 1000 \rceil = 999$, so below a thousand the p99.9 is the maximum wearing a decimal point. In both cases the surface renders a dash with its reason ("collecting, n=7 of 20"), never a number and never a zero.

The measured figures the desk quotes against these floors are in `docs/architecture/LATENCY_BUDGET.md`, and the discipline is that three planes are never blended:

Plane	Figure	Source
Whole decision, µs	12.4 µs p50 native, 23.1 µs Python [`LATENCY_BUDGET.md` §2.1]	`tools/bench_decision.py` dev Mac, `submit()` under the lock
Arithmetic core, ns	83 ns p50, 84 ns p99 [`latency-bench.generated.json`]	`steady_clock` inside the C++

		engine, same run
Same core, production	320 ns p50, 352 ns p99 [`LATENCY_BUDGET.md` §2.1]	live `/metrics` OCI `VM.Standard3.` 2026-08-17
Order entry, ms	72.7 ms origin RTT to Binance, 6.2 ms to Bybit [`LATENCY_BUDGET.md` §2.3]	`tools/colocation_probe.`

The nanosecond row carries a caveat the desk is required to repeat: on that machine `steady_clock` advances in 41.677 ns steps [`latency-bench.generated.json` , `core_ticks.tick_ns`], so 83 and 84 ns are both **two ticks** and nothing between them is representable. The figure with the resolution is not the p99 but the **fraction of calls finishing inside two ticks**, 0.9952 over 5 000 samples [`latency-bench.generated.json`], with nine repeats spanning 0.9932 to 0.9976. Quoting the 84 ns alone would be publishing a rounding artefact as a speed-up.

The far tail is attributed rather than assumed. Disabling the cyclic garbage collector changed p50 not at all and p99 by about 0.1 µs [`LATENCY_BUDGET.md` §2.2] the six samples per 5 000 at ten to twenty-three ticks [`LATENCY_BUDGET.md` §2.1] arrive **in bursts**, with seven of thirty slow samples immediately following another slow one, which is the signature of preemption rather than of code. Nothing in `decide()` can take 8.6 µs, so nothing in `decide()` caused the 8.6 µs outlier. The gateway runs on a virtualised two-OCPU shape; the hypervisor is the constraint, and the document says so instead of blaming the language.

The health and operations snapshot

`/api/ops/snapshot` is a typed, secret-free read model assembled in-process at read time from the same accessors that back `/health` and `/metrics` . It performs no network call and no storage query, so polling it cannot contend with the order path or turn a downstream outage into a gateway outage.

Because it crosses a deployment boundary, the web tier validates it at runtime rather than trusting the generated types: `isGatewayOpsSnapshot` checks `schema_version === 1` , a parseable `observed_at` , a positive `stale_after_seconds` , and every nested field of market data, risk, queue, audit, Telegram and route latency. Types are a compile-time agreement between two repositories' **sources**; a runtime gate is an agreement between two **deployed artefacts**, which can differ.

Freshness is computed from the snapshot's own budget and never borrowed from the local route:

$$\text{age} = \max(0, t_{\text{received}} - t_{\text{observed}}), \quad \text{state} = \begin{cases} \text{invalid} & \text{if } t_{\text{observed}} - t_{\text{received}} > 5\text{ s} \quad \text{(MAX\_FUTURE\_CLOCK\_SKEW\_MS)} \\ \text{stale} & \text{if age} > \text{stale after} \\ \text{fresh} & \text{otherwise} \end{cases} \tag{65}$$

A snapshot stamped more than five seconds **ahead** of the reader's clock is `invalid` , not `fresh` — a clock skew is a reason to disbelieve a timestamp, not to accept it.

Posture is then derived over three planes, ranked, in `deriveReliabilityPosture` :

Plane	Rule
Trading	`halted` if the kill switch is on or the gateway says halted; `critical` if a critical component or the audit log is unavailable; `degraded` for reduce-only, synthetic market data, or a degraded venue set; `unknown` when telemetry is stale
Research	`critical` only when every configured provider is unavailable; `degraded` on a reduced set
Notifications	never worse than `degraded` , and never allowed to impersonate either money plane

Three properties fall out and each is a decision. A research outage never paints the money path critical. A notification outage never paints either — the Telegram companion used to report through `platform.status` , so one chat transport blip announced a degraded **trading** path to a desk whose orders were routing normally; taking it off the money path entirely would have made an outage invisible instead of wrong, so it reports on its own axis. And **stale monitoring is `unknown` , while a configured gateway that cannot be reached is `critical`** — the difference between not knowing and knowing something bad.

The snapshot fetch has its own short timeout, 1 500 ms [`web/lib/reliability.ts`, `OPS_SNAPSHOT_TIMEOUT_MS`], with the reason written in the source: health must stay responsive even when the trading plane is the incident.

Circuit breakers and their state machine

The breaker exists to stop one dead provider adding its full timeout to every request on a route that has three working alternatives. Three states, with $T = 3$ consecutive failures and cooldown $C = 60$ s [`web/lib/providers/breaker.ts`, `BREAKER_THRESHOLD` / `BREAKER_COOLDOWN_MS`], and the breaker record held for $4C$ so a probe failure counts from a known state:

State	Entered when	Behaviour
`closed`	no record, or a success	calls pass; failures accumulate
`open`	failures reach T	calls skipped for C
`half_open`	C elapsed	the record is zeroed and flagged `probing`; the next call is the probe

Two details are structural rather than cosmetic. First, when the cooldown elapses the record is **zeroed, not deleted**. Deleting it reset the failure count correctly and also erased the only evidence that a circuit had been open, so the success that followed emitted nothing and the remediation ledger showed every self-healed circuit as still open, forever. Second, the state literals `"open"`, `"closed"` and `"half_open"` are a **contract**: `lib/remediation.ts` pairs events into incidents by matching them exactly, `half_open` is deliberately neither open nor closed (counting it as a closure would end an incident that is still open), and the strings are pinned by test rather than derived, because changing one renders the remediation ring empty while the breaker keeps working — a silent break.

`breakerSnapshot` is a separate, **non-mutating** read for the operator console. `breakerOpen` answers a boolean and, as a side effect, retires an expired breaker; a status panel that silently resets breakers by rendering is not a status panel. And an operator's manual close emits its own transition with `by: "operator"`, only when a circuit was actually holding, which is what makes the split between automatic and manual recovery a measurement rather than a guess.

Escalation rules, hysteresis and the resolve sweep

The same asymmetry that governs feed liveness governs alerting, for the same reason: an alert that repeats is an alert that trains people not to read the next one.

The risk monitor's drawdown warning is edge-triggered with a rearm below the fire threshold. With limit Λ and drawdown d: warn on the first tick where $d \geq 0.8\Lambda$, and rearm only once $d < 0.7\Lambda$ (`modules/risk_proxy/monitor.py`). Before that, the alert fired on every tick spent above the threshold — roughly seven hundred and twenty messages an hour at the old five-second cadence, and the monitor tick is now one second. The gap between 0.8 and 0.7 is what stops a drawdown hovering on the threshold from flapping once a second.

The feed watchdog transitions on **change** only, and treats the first observation as a baseline rather than an incident: a venue that is already down when the process starts does not page anyone at startup for a condition nobody caused. Every transition writes a risk event to the audit log with severity and reason, and then — separately — attempts to alert. The two are ordered so that an alert transport failing cannot be the reason the audit write is missed, and the health check is wrapped so that observability failing cannot be the reason the synthetic-book failover stops running.

Escalation delivery is addressed by **role**: a provider failing contract checks is a data engineer's problem first and a developer's second, and a portfolio manager receiving it learns nothing they can act on. A chat with no role receives it regardless. The channel actually reached is recorded on the escalation row, so "escalated to log" is never mistaken for a page.

The on-call rota is a mechanism shipped without a roster, and says so. Entries parse as `who@days=start-end`, first match wins so order is precedence, and a window whose end precedes its start wraps past midnight — which is what a night shift is. The wrapped case is handled by matching the after-midnight tail against the day the shift **began**, because testing the calendar day of the moment covers Monday 03:00 and leaves Saturday 03:00 uncovered, which is the one hour a weekday rota most needs to reach. An unparseable entry is **kept**, with its error published, and skipped only for resolution: a rota that silently ignores the line with the typo in it pages nobody and says nothing. An empty `DATA_ONCALL` is a supported state that reports itself, not a misconfiguration.

Failover topology

```
MARKET DATA      Binance WS --+
                 Bybit WS   --+--> TCAEngine books --> consolidated ladder
                 SIM        --+          ^
                                         |  only when EVERY real feed is dark
                                         |  AND ALLOW_SYNTHETIC_BOOK=1, and
                                         |  every payload is tagged synthetic

REFERENCE DATA   ranked chain per (capability, asset)
                 sorted by meta.rank[capability]; absent ranks last at 99
                 each hop may be skipped: outage / unconfigured / breaker
                                          / unlicensed / quota

DESK DATA        live --(any failure)--> cached --(2 successes)--> live
                  |                        |
                  +-- never ---------------+--> sandbox   (see The sandbox
                                                           doctrine, below)

PERSISTENCE          quality ledger : sqlite | postgres   (no silent fallback)
                     job queue      : in-process pool | Celery
                     audit log      : DuckDB | SQLite     (fallback only for absence,
                                                           NEVER for a lock conflict)
```

Listing 2: Four failover surfaces, each with a different rule about what may substitute for what.

The audit log's asymmetry is the one worth dwelling on. `duckdb.connect` reports two very different conditions as the same exception: **DuckDB is unavailable here**, for which the SQLite fallback is exactly right and nothing is lost but analytical SQL; and **another live process already holds this database**, for which falling back meant the second gateway did not fail — it opened a private ledger beside the first and began writing a divergent history, while `/health` reported `backend: sqlite` as though somebody had chosen it. An append-only ledger silently forking in two is the worst thing that subsystem can do, and a bare `except Exception` was the reason it was silent. The lock conflict is now a typed error and it is raised, as defence in depth behind the `flock(2)` claim `modules/single_writer.py` takes in `RiskGateway.start()`.

The same boundary explains why the gateway refuses to run multiple workers. `tests/test_container_contract.py` fails the build on `--workers` or `gunicorn`, with the reason inline: a second worker forks the in-memory book and localises the kill switch. Moving the four data-operations tables to Postgres removes the **storage** half of that constraint — a redeploy or a second container reads the same rows — and does not remove the other half, and the document says which is which rather than implying that a database made the process stateless.

The sandbox doctrine

The desk can show three tiers of number, and the tier decides what the reader is allowed to **do**, not merely what the caption says:

Tier	What it is made of	Writes
`live`	a payload the backend returned just now	enabled
`cached`	a payload the backend returned earlier, carried with its age	disabled
`sandbox`	a payload this browser generated, seeded and self-consistent	disabled

Two rules govern movement between them, and both are enforced by a state machine (`DeskSourceMachine`) rather than by a component.

Measured data is never replaced by generated data. Once a probe has succeeded even once, a failure demotes `live` to `cached` and stops there. The sandbox is reachable only from a desk that has never had a reading, or by a human pressing Sandbox.

Demotion is immediate; promotion requires a streak of two. Falling to `cached` on the first failure is the conservative direction, because writes are disabled there. A gateway alternating success and failure settles at `cached` and stays there, which is also the honest description of a gateway you can reach half the time.

During an incident, generated data may not stand in. The argument in full.

The temptation is strong and the reasoning is superficially good: a panel that says "gateway unreachable" is useless, a generated panel is at least **demonstrative**, and a banner can say which it is. That reasoning fails on four counts, and the failures compound.

First, it destroys the evidence at the moment it is worth most. An incident is precisely the interval during which someone is trying to establish what the system last knew. `cached` preserves that: real numbers from forty seconds ago, carried with their age, from which an operator can reason about what changed. Generated numbers preserve nothing. Substituting them converts an incident into an absence of an incident, and the reader loses the only artefact that could have told them when the desk stopped being right.

Second, it is a provenance failure, not a rendering failure, and no label repairs it. This repository has the counterexample in its own history: one hook discarded the last good book on a failed poll, so the whole execution cockpit — blotter, alerts, P&L strip, fill quality — swapped to a generated desk and swapped back on the next successful poll. At a four-second cadence against a gateway dropping one poll in three, that is a desk visibly alternating between real fills and invented ones every few seconds. Every frame of it was correctly labelled. The label did not help, because a reader integrating a number over seconds does not re-read the banner between frames.

Third, it breaks the one safety property that is not advisory. Only `live` enables a write. If an incident could route the desk to `sandbox`, then the tier that a human reaches for during an outage would be the tier whose numbers were invented — and every gate above it (the order ticket, the kill-switch arming, the operator actions) would be defending against a book that no venue ever quoted. Keeping the incident path at `cached` keeps writes disabled **and** keeps the numbers real, which are two different guarantees that happen to point the same way.

Fourth, it makes the outage unmeasurable. A surface that always renders something cannot report how often it had nothing. The reliability posture depends on the difference between "no authoritative source is configured" (a deployment fact) and "a configured source did not answer" (an incident), and a sandbox that fills both cases erases the distinction that the whole posture model is built on.

Where generated data is therefore permitted, and only there: on a desk that has never held a reading; when a human explicitly presses Sandbox; and on a deployment where no gateway is configured at all, which is the normal, designed state of the public workspace rather than a fault. In each of those the cause is named on the wire (`not-configured`, `chosen`) and is not `incident`.

The gateway's own synthetic feed obeys the same doctrine one layer down. It is enabled only when **every** real venue is dark and `ALLOW_SYNTHETIC_BOOK` is set; every downstream payload is tagged `synthetic: true`; the platform reports `market_data.status = degraded`; and the trading posture reads "the gateway is using synthetic market data and is not live-trading ready". It exists so that an offline demonstration has a ladder to draw, not so that an incident has one.

4.3 Part C. The quant developer

The compiled decision core and its binding

`native/decision_core/decision_core.cpp` is one of two implementations of the same seventeen-gate battery. The Python reference in `modules/risk_proxy/` is authoritative; the C++ core is a **delegate** for the book arithmetic and the numeric gates. The boundary between them is drawn for bit-for-bit parity, not for size, and it is written down in the translation unit's own header comment.

In the core, and timed	In Python, before the clock starts
`BookLadder` with dict-snapshot semantics; best bid/ask, mid, depth folds	kill switch, symbol halt, whitelist, paper-execution model, reference freshness, duplicate order
the consolidated depth-weighted mark for the order symbol	the rate-limit token consume — it **mutates**, so it must run exactly once
qty/notional derivation, `price_available`, `order_sized`	`working_book`, a bare length comparison

`max_order_notional`, `symbol_concentration`, `gross_exposure`, `price_band`, `daily_drawdown`, `reduce_only`	per-position marks (each an independent multi-venue consolidation)
`est_slippage` — the cross-venue merged-ladder walk	every `round()`, every f-string, every `CheckResult`

Three constraints make the parity claim survivable.

Fold algorithms are matched, not approximated. CPython 3.12′s `sum()` uses Neumaier compensated summation, so a plain `+=` fold in C++ lands one ULP away. The Python reference is split down that exact line — `BookState.depth_usd` and the P&L functions go through `sum()`, while `consolidated_mid` and `gross_exposure` are hand-rolled `+=` loops — and the core reproduces **each** fold with the matching algorithm. The routed walk needs both in one function.

Fused multiply-add is forbidden. `-ffp-contract=off` in the build plus `#pragma STDC FP_CONTRACT OFF` in the translation unit, because an FMA rounds once where CPython rounds twice, and that was measured as a one-ULP parity break. `-march=native` is deliberately absent: the Docker builder stage and a developer's Mac must emit the same doubles.

Tie-breaks are part of the specification. Python's `list.sort` is stable and **stays stable under** `reverse=True`, so two venues quoting the same price fill in feed-iteration order. Getting that backwards moves the blended VWAP by a ULP. A randomised differential test against the reference router covers 400 cases on a shared price grid [`LATENCY_BUDGET.md` §2.1], of which 106 of the 125 multi-venue cases [`LATENCY_BUDGET.md` §2.1] diverge if the tie-break is reversed — which is what makes that test the control rather than a comment.

The binding is pybind11, and two of its properties cost real engineering. Argument passing is **positional**, twenty-six arguments in the order of the C++ signature, because pybind11 resolves `py::arg` names through a dictionary on every call; the interleaved A/B measured the decision p50 faster in all five rounds by 1.37, 1.58, 1.21, 0.83 and 0.91 µs [`LATENCY_BUDGET.md` §2.1]. And object lifetime is not automatic: the mirror stores `BookLadder` pointers past the single synchronous call that `BookState.native_ladder()` documents them as valid for, which segfaulted the suite until each entry kept a `py::object` alongside the pointer.

The mirror also costs vigilance in exchange for its speed, and the source says so: every mutation of the position book must re-sync it — both fill sites, the paper execution, the audit replay and the session rollover, the last of which was found by a test rather than by inspection. When the mirror cannot be trusted — a venue joined or dropped, or any book has no native ladder — the **whole** decision falls back to the Python path rather than half of it deciding natively.

Finally, the core measures itself at startup. `run_core_self_measure` runs the same compiled `decide()` the order path runs, against two synthetic ladders built directly from the extension: fifty warm-up calls unrecorded, then three hundred recorded. The count of synthetic samples is published beside the total (`core_self_test_samples`) so a reader can tell provenance rather than infer it. What it deliberately never touches is the microsecond histogram, the order counters, the audit log, the token bucket or the TCA engine — a self-measure is evidence about the core, and nothing synthetic may enter the plane that measures whole decisions under the lock.

Engine selection and the bit-exact parity fixture

`DECISION_CORE` takes three values and each encodes a different deployment intent:

`auto`	**native if importable, else the Python reference with a warning. The default.**
`native`	refuse to start without the extension. For a deploy that must not degrade quietly.
`python`	the reference, always available.

The selected engine is published on `/health`, `/metrics` (`alphaengine_decision_engine{engine="native"}`) and the ops snapshot, and the desk marks a gateway that fell back — a deployment-integrity signal, not a correctness one, since the Python reference is exact.

One trap here is recorded in the source and is easy to reintroduce. The book mirror keys on whether the extension **imports**, not on which engine `DECISION_CORE` selected — different questions. A caller that forces the native engine (the parity suite does, so a quietly degraded build turns CI red) would otherwise find every book unmirrored under `DECISION_CORE=python` and fall back without saying so: the silent fallback that suite exists to catch, caused by the mechanism meant to make it fast.

Parity is pinned by `web/tests/fixtures/gate-parity.json`: 20 scenarios [`web/tests/fixtures/gate-parity.json`] against the seventeen gates in evaluation order. Each scenario carries a fixed clock, a seeded position book, resting orders, seen client ids, and **every one of the fifteen settings fields a gate reads** — so a change to a default cannot silently change what the fixture pins.

Scenario	Scenario
`happy_market`, `happy_limit_resting`	`kill_switch_on`, `symbol_halted`
`not_whitelisted`, `duplicate_client_id`	`rate_limited`, `working_book_full`
`no_price`, `oversize_notional`	`concentration_breach`, `gross_breach`
`price_band`, `slippage_breach`	`slippage_partial`
`drawdown_reduce_only_blocks_opening`	`drawdown_reduce_only_allows_close`
`paper_equity_happy`, `paper_equity_limit_rejected`	`paper_equity_stale_quote`

"Parity" here is stronger than "same verdict". The assertion is the same accept or reject, the same gate names in the same order, **and the same observed and limit doubles** — which is why the fixture caught the Neumaier and FMA defects, both of which produced identical verdicts and different bits.

The fixture is shared rather than duplicated: `tools/make_gate_fixture.py` records what the Python reference decides and `tests/test_gate_parity.py` asserts that the running engine still decides the same, through one loader (`tools/gate_fixture.py`). A recorder and a checker that build their gateway differently are two things to keep in step; one loader is one.

The API and WebSocket protocols

The REST surface is 73 paths and 144 schemas [`tools/openapi.json`, counted 2026-08-24], exported from the running FastAPI application and committed. WebSockets are **deliberately outside** that contract — OpenAPI does not describe them — and their shapes are pinned by tests instead of by the schema, which the module docstring states rather than leaving a reader to infer that the socket was forgotten.

Three transports carry live data, and they are chosen by what each hop can actually do:

1. **Browser to venue, direct.** `wss://stream.binance.com` and `wss://stream.bybit.com`, 100 ms depth frames, published to React at 5 Hz (`web/lib/livebook.ts`). One hop, no backend; routing it through the gateway would make it slower.
2. **Gateway `/ws/book/{symbol}`.** The consolidated ladder plus a live TCA report at 4 Hz (`modules/api/tca.py`, a 0.25 s send loop), for the depth-of-market visualiser. The payload is a tagged object (`type: "book"`) carrying the consolidated mid, the venues online, each venue's book and, when a report exists, the TCA block.
3. **Server-sent events, `/api/stream/desk`.** Risk state, proxied. A browser cannot open an `EventSource` to the gateway directly — the page is HTTPS and the gateway is plain HTTP, which is blocked as mixed content with no override — so this is proxied through a route handler.

The SSE proxy carries a design constraint worth recording because a previous attempt was removed over it. `EventSource` exposes neither the status code nor the body, so a deliberate 503 on a gateway-less deployment was invisible to the client and the panel read "Connecting…" forever — on precisely the deployment where that is the normal condition. The proxy now answers **200 in every case** and puts the state in the first frame:

```
event: desk-state
data: {"state":"unavailable","reason":"gateway_not_configured"}
```

On `unavailable` the hook closes the source rather than letting `EventSource` retry every three seconds, which would be a poll wearing a push's clothes. The stream is a **signal**, not a second source of the numbers: rebuilding the panels on its shape would give the same figures two sources that can disagree. Its sequence number moves only when the risk state actually changed, so an idle desk costs nothing and a moving one is refetched because something moved rather than because a timer expired.

The job queue and its two backends

A parameter sweep can take tens of seconds. Running it inside the request handler would block the event loop that also carries the kill switch — the one path that must never queue behind a backtest. Two interchangeable backends sit behind one interface: a bounded `ThreadPoolExecutor` by default (NumPy and Numba release

the GIL in the numeric inner loops, so thread parallelism is genuinely parallel here), and Celery when a broker URL is configured. The API surface, the console and the notification companion never learn which one is running.

Retries are opt-in **by job kind**, and each entry in the table is a claim that running the work twice is indistinguishable from running it once:

Kind	Attempts	Idempotence argument
`data.backfill`	3	merges bars keyed by (symbol, interval, ts); a repeated merge overwrites identical rows
`data.replay`	2	a second finding is a second observation, which is what the ledger is for — but it costs a provider call, hence 2 not 3
`ml.fit`	2	a retry writes a second run row, which is honest: two fits are two runs, and the seed and data hash say so
anything else	1	`backtest` is deliberately absent — it pushes a corpus card and a chat message on completion, and repeating those is visible to a reader

with backoff from 2 s to a 30 s ceiling [`modules/jobs.py`, `RETRY_BASE_S` / `RETRY_CEILING_S`]. Persistence happens in the completion hook, on the gateway's own event loop, for **both** backends: a worker fetches and validates and returns, and the gateway writes. That keeps a single writer to the ledger regardless of where the compute ran.

The queue's own telemetry is redacted more aggressively than a URL redactor would manage: the broker is reported as a transport **identity** — one of a known set of schemes, or `other` — rather than as a redacted URL, because even a redacted URL exposes host, port, database index and virtual host, none of which an operator needs in a queue summary.

The append-only audit log

One DuckDB handle (SQLite when DuckDB will not load) behind one lock, on a Docker volume, holding orders, order events, risk events, TCA snapshots, equity snapshots and job status. Every statement in the DDL is `CREATE TABLE IF NOT EXISTS`; columns added after the first databases existed are widened by an explicit migration step rather than by editing the DDL, because a column added to that list would be created on a fresh database and missing on every existing one.

The ledger is the system's memory across restarts, which is why the position book is **rehydrated by replay** rather than checkpointed, and why the lock conflict discussed under **Failover topology** must be raised rather than absorbed. It is also why the mirror to Supabase is a bounded, best-effort queue that is never on the order path: the authoritative record is local and append-only, and a managed database being slow must not be able to slow an order down or to lose one.

The generated-gate discipline

Seven files in this repository are generated, and three of them are gated by a checker that **recomputes the artefact itself**. Those three are the pattern worth reading, because it is the same each time: a generator writes a file, a checker recomputes it and fails the build on drift, and the file itself carries a header saying it must not be hand-edited.

Artefact	Generator	Checker	What drift would mean
`gateway-openapi-digest.generated.ts`	`tools/export_openapi.py`	`check-gateway-openapi-digest.mjs`	the web tier's generated client describes

			a gateway that is no longer deployed
`repository-manifest.generated.json`	`generate-codebase-manifest.mjs`	same script, `--check`	the repository catalogue on the Developer tab lists files that do not exist
`test-counts.generated.ts`	`refresh-test-counts.mjs`	`check-test-counts.mjs`, on the WEB line only	the desk quotes a suite size nobody measured

The OpenAPI gate hashes **canonical** JSON — keys sorted recursively — so that a re-export which reorders a dictionary does not read as a contract change. The committed digest today is `a0263f96…0205` [`web/lib/gateway-openapi-digest.generated.ts`, verified against `tools/openapi.json` on 2026-08-24].

The manifest gate compares **only the file list**, not the commit or the generation date, because those change on every commit by design and gating on them would fail every push. It also skips itself, with a message, when git is unavailable — a tarball build has nothing to compare against, and the gate holds where drift can actually happen. The current manifest carries 1 770 files [`web/lib/repository-manifest.generated.json`] at commit `ba58a40`.

The counts gate is the most interesting of the three, because it cannot live inside the thing it measures: **a test that checks the test count changes the test count**. So the check runs outside the suite, against the runner's own summary line teed to a log file. The committed figures, measured on 2026-08-24 [`web/lib/test-counts.generated.ts`], are 2 998 gateway tests (2 996 passed, 2 skipped), in the CI shape [`web/lib/test-counts.generated.ts`, refreshed with `RERANK_TEST_MODEL_PATH` blanked], 4 487 web tests across 984 suites [`web/lib/test-counts.generated.ts`] and 24 service tests

[web/lib/test-counts.generated.ts]. Only the web figure is gated: CI runs `check-test-counts.mjs` with the argument `web` and it reads that line alone, so the gateway and service lines beside it are dated records rather than contracts, and may legitimately differ from what a differently configured run prints — which is why the gateway figure here is quoted with the shape it was taken in. Nothing regenerates them automatically, because running three suites inside a production build would make every deploy pay for them — so the header names the date each figure was printed, and the desk renders that date beside the number.

This is what "keeping two deployed units honest with each other" means in practice. The web tier and the gateway are separate deployments on separate platforms with separate release cadences. Types shared through a generated client are an agreement between two **sources**; the digest, the runtime payload runtime payload gate on the ops snapshot and the parity fixture are agreements between two **artefacts**. Only the second kind survives one side being redeployed and the other not.

4.4 What is not built, and why it waits

Stating this is a strength of the document rather than an omission, and the product requirements already do it.

A second gateway process. The position book, the resting-order book, the token bucket and the kill switch are process-local mutable state. Moving the four data-operations tables to Postgres removed the storage half of that boundary and not the other half, and a test fails the build on any attempt to add workers. Horizontal scale needs those four structures externalised with a real concurrency story, which is a design, not a flag.

Tokyo co-location. The expected improvement from moving the gateway into the region where the matching engine already runs is an **expectation**, not a measurement, and the plan is probe-first: stand up an instance, run the same `time_connect` probe, and migrate only if it confirms. The free half of the same finding — that Bybit's origin answers in 6.2 ms [LATENCY_BUDGET.md §2.3] from the existing Singapore VM against 72.7 ms [LATENCY_BUDGET.md §2.3] for Binance — has been taken for research bar loading and deliberately **not** for order routing, because nearer is not deeper and conflating a latency decision with a routing decision is the mistake the whole latency document exists to prevent.

A green CI run has not reached Postgres. The live data-operations test skips with its reason printed unless credentials are present, because CI is network-free by design. What CI does prove is the **request** — the `Prefer` headers, the filter grammar, the conflict target — asserted against a mock transport, because that is where the translation lives and a test checking only the parsed response would pass with `Prefer` missing entirely.

The venue failover chain has no live exercise. Of Binance's 489 USDT pairs, 247 are absent from Bybit [LATENCY_BUDGET.md §2.5], but every symbol the workspace currently offers is on both — so no real request can exercise the fallback. The chain is therefore injectable and the fallback is tested with a stubbed venue, rather than left as an untested promise.

Hardware paths are out of scope with reasons. No crypto venue rents an FPGA feed-handler path; there is no microwave route into a cloud region, because the relevant distance is a VPC hop; and kernel bypass needs hardware this instance does not expose. With the decision at 12.4 µs [LATENCY_BUDGET.md §5] against a 69 ms [LATENCY_BUDGET.md §5] market-data round trip, none of the three has anything to save.

The honest summary of this chapter's three roles is the same sentence in three registers. The data engineer's numbers are traceable to a vendor byte or they are marked as not traceable; the reliability engineer's status is derived from an observation with an age on it or it is `unknown`; the developer's two implementations agree to the bit or the build is red. In every case the alternative — a plausible number with no provenance — is available, cheap, and refused.

5 Mathematical Foundations and Model Specifications

This chapter is the formal core of the document. Every model the desk runs is stated in the notation it is derived in, every symbol is defined, and every equation is numbered. For each model three things are given: the derivation, the assumption that makes it true, and the failure mode when that assumption breaks. Where the implementation departs from the textbook form the departure is named and argued, because the departures are the interesting part — a formula reproduced faithfully teaches nothing about the system that runs it.

Two facts about the codebase shape everything below. Almost every quantity here exists twice — once in Python for the gateway and the Telegram companion, once in TypeScript for the browser, because neither runtime can reach the other — and the pairs are pinned against recorded fixtures, because two implementations of one calculation are two chances to be wrong. And the arithmetic refuses more often than it approximates: a percentile over twelve observations, a payoff ratio with no losing trades, a covariance with a singular design each return a typed absence carrying its reason. That is a mathematical commitment rather than a presentational one, because it changes what the estimators are allowed to return.

The worked example running through this chapter is one recorded sweep committed as `web/lib/seed-run.json`: a moving-average crossover on BTCUSDT 4h bars, 1200 [seed-run.json] bars from 2026-01-22 to 2026-08-10 [seed-run.json], over 74 [seed-run.json] parameter combinations. Where a figure below is an evaluation of an equation on this page rather than a reading from the tree, it says so.

5.1 Notation and conventions

Symbol	Meaning
r_t	Per-bar arithmetic return of the strategy or book at bar t
μ, σ	Population per-bar mean and standard deviation of r_t
$\hat{\mu}, \hat{\sigma}$	Their sample estimates, $\hat{\sigma}$ always with ddof $= 1$
n	Number of return observations
a	Bars per year for the interval in question — the annualisation constant
S	Per-bar Sharpe ratio, μ/σ; $\hat{S}$ its estimate
γ_3, γ_4	Sample skewness and raw (Pearson) kurtosis; normal $\gamma_4 = 3$
Φ, φ	Standard normal CDF and density; Φ^{-1} the quantile function
z_α	$\Phi^{-1}(\alpha)$, the one-tailed normal quantile
N	Number of trials in a parameter search
$\boldsymbol{w}$	Vector of position weights as fractions of equity, signed
$\boldsymbol{\Sigma}$	Sample covariance matrix of per-bar returns, $p \times p$
σ_p	Per-bar portfolio volatility, $\sqrt{\boldsymbol{w}^T \boldsymbol{\Sigma} \boldsymbol{w}}$
E	Account equity in quote currency
Q	Order notional in quote currency; ADV average daily volume in the same

Three conventions hold throughout and are worth stating before they are used, because each one is a decision that could have gone the other way.

Per-bar is the unit of inference; annualised is a unit of display. Every inferential quantity in this chapter — the probabilistic Sharpe, the deflated Sharpe, the minimum track record length — is computed on per-bar Sharpe ratios. The annualised figure exists only to be printed. The engine de-annualises before it corrects for multiple testing and re-annualises only the reported hurdle (`web/lib/engine.ts`, lines 137 and 324). Section 5.3 shows what feeding an annualised Sharpe into a per-bar formula does; it does not produce a slightly wrong answer, it produces certainty.

The annualisation constant is a calendar fact, not a market fact. The desk trades instruments that do not close, so

interval	15m	1h	4h	1d
a	35 040	8 760	2 190	365

read from `modules/quant_risk/_common.py` and mirrored in the TypeScript's `BARS_PER_YEAR`. Note 365, not the 252 an equity desk would use: a continuously-traded instrument has no non-trading days to exclude. An unknown interval falls back to 8 760 in the TypeScript (`web/lib/quant/common.ts`) rather than raising, which is a defensible default for hourly data and a silent error for anything else — it is the one annualisation path in the tree that does not refuse.

A missing measurement is a typed state, never a zero. Every estimator below that can fail returns `null` with a reason attached rather than a number that happens to be small. The sample floors are stated with each model.

5.2 The Sharpe ratio and its scaling in time

Definition and estimator

For a return series with no risk-free deduction,

$$\hat{S} = \frac{\hat{\mu}}{\hat{\sigma}}, \quad \hat{\mu} = \frac{1}{n}\sum_{t=1}^{n} r_t, \quad \hat{\sigma} = \sqrt{\frac{1}{n-1}\sum_{t=1}^{n}(r_t - \hat{\mu})^2} \tag{66}$$

The risk-free rate is zero in the numerator throughout, and that is a modelling choice rather than an oversight. Financing is charged as an explicit cost instead: `holdingCost` in `web/lib/quant/cost-model.ts` debits perpetual funding at a rate in basis points per eight hours against absolute exposure, and an annual borrow rate against short exposure. Subtracting a financing rate in the numerator as well would charge the same cost twice. The consequence is that $\hat{S}$ here is a ratio of **net** return to volatility, not an excess return over cash, and it is not directly comparable to a published Sharpe that deducts a bill rate.

The Bessel correction is not optional anywhere in this tree. Both stacks use $\mathrm{ddof} = 1$ (`_stdev` in `modules/quant_risk/_common.py`, `stdev` in `web/lib/stats.ts`), and both return exactly zero when $n - \mathrm{ddof} \le 0$ rather than dividing.

Scaling from per-bar to annual

Let $r_t(q) = \sum_{j=0}^{q-1} r_{t-j}$ be the return over q consecutive bars. Under stationarity with autocorrelations $\rho_k = \mathrm{Corr}(r_t, r_{t-k})$,

$$\mathbb{E}[r_t(q)] = q\mu, \quad \mathrm{Var}[r_t(q)] = q\sigma^2 + 2\sigma^2 \sum_{k=1}^{q-1}(q-k)\rho_k \tag{67}$$

so the q-bar Sharpe is

$$S(q) = \frac{q\mu}{\sqrt{q\sigma^2 + 2\sigma^2\sum_{k=1}^{q-1}(q-k)\rho_k}} = S \cdot \eta(q), \quad \eta(q) = \frac{q}{\sqrt{q + 2\sum_{k=1}^{q-1}(q-k)\rho_k}} \tag{68}$$

The universal practice — and the practice in this tree — is $S(q) = S\sqrt{q}$. Comparing that with Equation 68 gives the exact condition under which it is correct:

$$\eta(q) = \sqrt{q} \iff \sum_{k=1}^{q-1}(q-k)\rho_k = 0 \tag{69}$$

Not $\rho_k = 0$ for all k, but a weighted sum of them equal to zero: a series whose positive and negative autocorrelations cancel under the triangular weight $q-k$ scales by $\sqrt{q}$ exactly, and a series with $\rho_1 > 0$ does not. For an AR(1) process with $\rho_k = \rho^k$ the sum closes:

$$\sum_{k=1}^{q-1}(q-k)\rho^k = \frac{\rho[(q-1) - q\rho + \rho^q]}{(1-\rho)^2} \tag{70}$$

The failure mode. Positive autocorrelation makes $\eta(q) > \sqrt{q}$, so naive scaling **understates** the annual Sharpe of a trending series and **overstates** that of a mean-reverting one. The direction most often complained about in the literature is the opposite of this: a monthly-reported illiquid book with smoothed marks has ρ_1 strongly positive, and its naive annualised Sharpe is too high — because the smoothing suppresses $\hat{\sigma}$ at the reporting frequency rather than because Equation 68 was applied wrongly. Both errors live in the same equation and they are not the same error.

What this series actually does. Autocorrelations of the winning run's per-bar returns, computed from the 1 200 values in `seed-run.json`:

lag k	1	2	3	4	5
ρ_k of r_t	-0.0048 [seed-run.json]	-0.0267 [seed-run.json]	-0.0068 [seed-run.json]	0.0139 [seed-run.json]	0.0254 [seed-run.json]
ρ_k of $\lvert r_t - \hat{\mu}\rvert$	0.4201 [seed-run.json]	0.3365 [seed-run.json]	0.3137 [seed-run.json]	0.2750 [seed-run.json]	0.3286 [seed-run.json]

Evaluating Equation 68 on the sample autocorrelations gives $\eta(5) = 2.2817$ against $\sqrt{5} = 2.2361$, and $\eta(10) = 3.2445$ against $\sqrt{10} = 3.1623$: naive scaling understates by 2.0% and 2.6% at those horizons. Both are evaluations of Equation 68 on the series in `seed-run.json`, not separate measurements.

And here is the limit the tree respects. The annualisation actually applied is $\eta(2190)$. Estimating that from Equation 68 requires ρ_k out to lag 2 189 from 1 200 observations, which is not an estimation problem, it is an absence of data. There is no honest correction factor to quote at the annual horizon, so none is quoted and none is applied. What the tree does instead is confine every **inference** to the per-bar quantities, where the sample supports it, and treat the annualised Sharpe as a display figure. That is why the deflated Sharpe machinery in the next two sections never touches an annualised number.

> **The correction that is not implemented**
>
> Lo (2002) gives $\eta(q)$ from Equation 68 with a consistent estimator of the autocovariance sum. It is not implemented in this tree, for the reason above: at $q = a$ the estimator has no data. Applying it at a short horizon and extrapolating would be worse than not applying it, because the result would carry a precision the sample cannot support.

5.3 The Probabilistic Sharpe Ratio

Derivation

The Sharpe estimator Equation 66 is a ratio of two sample moments and is therefore itself random. Under i.i.d. returns with finite fourth moment, the delta method gives its asymptotic variance in terms of the higher moments of the return distribution:

$$\operatorname{Var}\left[\hat{S}\right] \approx \frac{1}{n-1}\left(1 - \gamma_3 S + \frac{\gamma_4 - 1}{4} S^2\right) \tag{71}$$

Two terms, two effects, and they pull in opposite directions. Positive skewness **reduces** the variance of the Sharpe estimator; excess kurtosis **increases** it. Treating Equation 71 as the variance of an asymptotically normal estimator and asking for the probability that the true Sharpe exceeds a benchmark S^* gives the Probabilistic Sharpe Ratio:

$$\mathrm{PSR}(S^*) = \Phi\left(\frac{\left(\hat{S} - S^*\right)\sqrt{n-1}}{\sqrt{1 - \gamma_3 \hat{S} + \frac{\gamma_4 - 1}{4}\hat{S}^2}}\right) \tag{72}$$

implemented as `probabilisticSharpe` in `web/lib/stats.ts` and `probabilistic_sharpe_ratio` in `modules/backtester/statistics.py`, with the radicand clamped below at 10^{-12} in both so that an extreme skew cannot produce a negative variance and a complex denominator.

The measured effect of non-normality, and the scale trap

The winning run's per-bar returns have $\gamma_3 = 1.1618$ [seed-run.json] and $\gamma_4 = 14.917$ [seed-run.json] — right-skewed and very fat tailed — at $\hat{S} = 0.032510$ per bar. The two correction terms are then

$$-\gamma_3 \hat{S} = -0.037769, \quad \frac{\gamma_4 - 1}{4}\hat{S}^2 = +0.003677 \tag{73}$$

The skew term is an order of magnitude larger than the kurtosis term, and the reason is structural rather than particular to this run: the skew term is $O\left(\hat{S}\right)$ and the kurtosis term is $O\left(\hat{S}^2\right)$, so at per-bar Sharpes of order 10^{-2} the kurtosis correction is negligible however fat the tails are. The denominator comes to 0.98281 against 1.00026 if normality had been assumed, and

$$\mathrm{PSR}(0) = 0.87398 \text{ [seed-run.json]} \quad \text{against} \quad 0.86980 \text{ under } \gamma_3 = 0, \gamma_4 = 3 \tag{74}$$

The scale trap. Both terms are functions of a **per-bar** Sharpe, and the kurtosis term is quadratic in it. Substituting the annualised $\hat{S} = 1.5214$ into Equation 72 gives a denominator of $\sqrt{1 - 1.7676 + 8.0530} = 2.699$ and a z-score of 19.5, so PSR $= 1.000$: the same data, the same formula, one unit error, and the answer is certainty. This is why both implementations state the per-bar convention in the docstring of every function that takes a Sharpe, and why the engine de-annualises the candidate vector explicitly before calling them.

Minimum track record length

Equation 72 inverts in closed form for n. The observation count at which $\mathrm{PSR}(S^*)$ first reaches confidence c is

$$n^* = 1 + \left(1 - \gamma_3 \hat{S} + \frac{\gamma_4 - 1}{4}\hat{S}^2\right)\left(\frac{z_c}{\hat{S} - S^*}\right)^2 \tag{75}$$

`minTrackRecordLength` in `web/lib/stats.ts` and `min_track_record_length` in `modules/backtester/statistics.py`, using the identical variance clamp so that the two remain exact inverses under extreme skew. For the worked example against $S^* = 0$ at $c = 0.95$, Equation 75 gives 2 473.55, reported as 2474 bars [seed-run.json] — 1.13 years [seed-run.json] of 4h bars, against the 1 200 bars the run actually has. The response records `sufficient: false`.

Against the multiple-testing hurdle of the next section, $\hat{S} < S^*$ and Equation 75 diverges. Both implementations return $+\infty$, and the engine converts that to `bars: null` rather than a large integer, because no finite record can establish an edge that is not there. The absence is the answer.

Failure mode. Equation 71 assumes i.i.d. returns — the $\sqrt{n-1}$ is the independent-sample scaling — so under autocorrelation the effective sample size is smaller than n and the PSR is overconfident by roughly the η factor of Equation 68. Uncorrected, and it compounds with the uncorrected annualisation: both errors point the same way for a positively autocorrelated series.

5.4 The Deflated Sharpe Ratio and the expected maximum of N trials

The null

A parameter sweep does not estimate an edge; it reports a **maximum**. Running 74 combinations over a pure random walk reliably produces an impressive winner, and the whole apparatus of this section exists to price that. Under the null that every trial has true Sharpe zero and the trial Sharpes $\{\hat{S}_i\}_{i=1}^{N}$ are i.i.d. $\mathcal{N}(0, V)$, what should the best of them look like?

Let $M_N = \max_i Z_i$ for Z_i i.i.d. standard normal. With $b_N = \Phi^{-1}\left(1 - \frac{1}{N}\right)$ and $a_N = 1/(N\varphi(b_N))$, the normalised maximum $\frac{M_N - b_N}{a_N}$ converges to a standard Gumbel, whose mean is the Euler–Mascheroni constant $\gamma = 0.5772156649015329$. Hence $\mathbb{E}[M_N] \approx b_N + \gamma a_N$. Using the standard approximation $a_N \approx \Phi^{-1}\left(1 - \frac{1}{Ne}\right) - \Phi^{-1}\left(1 - \frac{1}{N}\right)$ and collecting terms gives the two-quantile form the code implements:

$$S_0^* = \sqrt{V\left[\{\hat{S}_i\}\right]}\left[(1-\gamma)\Phi^{-1}\left(1 - \frac{1}{N}\right) + \gamma\Phi^{-1}\left(1 - \frac{1}{Ne}\right)\right] \tag{76}$$

The Deflated Sharpe Ratio is then simply the PSR measured against this hurdle instead of against zero:

$$\mathrm{DSR} = \mathrm{PSR}(S_0^*) \tag{77}$$

The hurdle grows like $\sqrt{2\ln N}$ — slowly, and without bound. Evaluating Equation 76:

N	**10**	**74**	**100**	**1 000**	**10 000**
$S_0^*/\sqrt{V}$	1.5746	2.4228	2.5306	3.2551	3.8607

Those are evaluations of Equation 76, not measurements. The shape is the point: a hundredfold increase in the size of the search raises the hurdle by about 50%, so the correction is real but never punitive enough to make a large grid safe by itself.

The worked example, end to end

Every figure here is read from `web/lib/seed-run.json` or reproduces from it.

Quantity	**Per bar**	**Annualised ($\times\sqrt{2190}$)**
Trials N	74 [seed-run.json]	—
$\sqrt{V\left[\{\hat{S}_i\}\right]}$	0.015667	0.73316
$\Phi^{-1}(1 - 1/N)$	2.21113	—
$\Phi^{-1}(1 - 1/(Ne))$	2.57782	—

Hurdle S_0^* (Equation 76)	0.037957	1.77628 [seed-run.json]
Selected $\hat{S}$	0.032510	1.52140 [seed-run.json]
PSR(0)	0.87398 [seed-run.json]	—
DSR = PSR(S_0^*)	0.42391 [seed-run.json]	—

The selected Sharpe sits **below** the hurdle the search alone would have produced. Stated in one sentence: an annualised Sharpe of 1.52, which is 87% likely to beat zero, is only 42% likely to beat the best of 74 coin flips over the same 1 200 bars. The promotion gate's threshold is DSR ≥ 0.95 [promotion.ts], and this candidate fails it.

The departure, and why it matters

Both implementations compute the dispersion V of the trial Sharpes and deliberately **discard their mean**. Adding the sample mean back — the common implementation slip, named as such in the docstrings of both `deflatedSharpe` and `deflated_sharpe_ratio` — would make the hurdle $\bar{S} + S_0^*$. On a grid that is uniformly unprofitable $\bar{S} < 0$, the hurdle goes negative, and a losing strategy clears it. The mean candidate Sharpe of this particular grid is 0.0177 annualised, barely positive; a grid one degree less lucky would have demonstrated the failure directly.

Failure mode: trials are not independent. The 74 combinations of a moving-average grid overlap heavily — $(30, 60)$ and $(30, 80)$ share most of their trades — so the effective number of independent trials is well below 74. Since $\mathbb{E}[\max]$ of N dependent draws is below that of N independent ones, Equation 76 overstates the hurdle and the DSR is **conservative**. That is the safe direction, and it is the reason no effective-trial estimator is implemented: a correction that raises a passing probability needs a much stronger argument than one that lowers it. What is not available anywhere in the tree is an estimate of how far the hurdle is overstated.

Failure mode: the search and the test share the data. Equation 77 conditions on the selected Sharpe as though it were one draw when it was chosen for being the largest, and the PSR sampling distribution is not adjusted for that selection beyond the shift in the benchmark. Walk-forward, in Section 5.5, is the independent evidence; DSR is a price, not a proof.

5.5 The promotion gate as a joint test

Six vetoes stand between a candidate and execution, read from `web/lib/quant/promotion.ts`, and each is shown whether it passes or fails, because a panel that appears only on success teaches readers that silence means safety.

Gate	Hurdle	What it prices
Deflated Sharpe	≥ 0.95	The search itself, Equation 77
Walk-forward OOS Sharpe	> 0	Performance on bars the parameters never saw
Walk-forward efficiency	≥ 0.5	Median $\mathrm{SR}_{\mathrm{oos}}/\mathrm{SR}_{\mathrm{is}}$ across folds, defined only where $\mathrm{SR}_{\mathrm{is}} > 0$
Parameter neighbourhood	plateau or slope	Neighbour Sharpe retention on the grid lattice
Alpha t-statistic	$\lvert t \rvert \geq 2$	Return not explained by the three factors of Equation 78
Trade count	≥ 30	Sample supporting every ratio above

The efficiency gate's domain restriction is the interesting one: dividing by a negative in-sample Sharpe yields a **positive** ratio for a fold that lost money in both windows, so such folds report `null` and are counted separately rather than folded into a median that would flatter the strategy.

The worked example passes 2 of 6 [seed-run.json] — parameter neighbourhood (`slope`) and alpha t-statistic (3.25 [seed-run.json]) — and fails DSR (0.424 [seed-run.json]), out-of-sample Sharpe (-1.06 [seed-run.json]), efficiency (-0.26 [seed-run.json]) and trade count (8 [seed-run.json]), at an overfitting probability across four folds of 0.75 [seed-run.json].

What the gate is not. Six thresholds applied conjunctively is not a test of size α. The joint distribution of the six statistics under the null is not known, is not estimated anywhere in the tree, and cannot be recovered from the marginals — DSR and the walk-forward statistics are computed on overlapping data and are strongly dependent. The gate is a set of vetoes chosen so that each failure is individually interpretable; its joint false-pass rate is unquantified, and saying so is cheaper than implying otherwise.

5.6 Multi-factor risk decomposition

The factor set

Three factors, all constructible from one instrument's bars, all executed with the same one-bar lag as the strategy (`web/lib/quant/factors.ts`):

$$f_t^{\text{mkt}} = r_t^{\text{px}}, \quad f_t^{\text{trend}} = \operatorname{sgn}\left(\prod_{j=t-L}^{t-1}\left(1+r_j^{\text{px}}\right)-1\right)\cdot r_t^{\text{px}}, \quad f_t^{\text{vol}} = \begin{cases} +r_t^{\text{px}} \text{ if } \hat{\sigma}_{t-1,L} \le \bar{\sigma}_{t-1} \\ -r_t^{\text{px}} \text{ otherwise} \end{cases} \tag{78}$$

with $L = 30$ bars. The threshold $\bar{\sigma}_{t-1}$ is an **expanding** mean of every trailing volatility observed strictly before t, not a full-sample median, and the reason is the direction of the leak: a full-sample threshold would let the factor at bar 100 know what volatility looks like at bar 1 900, producing a benchmark stronger than anything anyone could have traded and making the strategy's alpha against it too **low**. Look-ahead that flatters the benchmark argues away real edge, and it is the direction nobody checks for.

Ordinary least squares, and the standard errors that are not corrected

With design matrix $\boldsymbol{X} \in \mathbb{R}^{n\times k}$ (intercept in column zero),

$$\hat{\boldsymbol{\beta}} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}, \quad \hat{\sigma}_\varepsilon^2 = \frac{\boldsymbol{e}^T\boldsymbol{e}}{n-k}, \quad \operatorname{se}\left(\hat{\beta}_a\right) = \sqrt{\hat{\sigma}_\varepsilon^2\left[(\boldsymbol{X}^T\boldsymbol{X})^{-1}\right]_{aa}} \tag{79}$$

solved by Gauss–Jordan with partial pivoting on the augmented $[\boldsymbol{X}^T\boldsymbol{X} \mid \boldsymbol{I}]$, which yields the inverse as a by-product — required anyway, since its diagonal is what turns a coefficient into a t-statistic. A pivot below 10^{-12} returns `null` rather than emitting infinities: a perfectly collinear factor set is a modelling error, and `NaN` propagating into a screen of plausible numbers is worse than an honest gap.

Measured on the worked example, $n = 1200$:

Regressor	$\hat{\beta}$	t	p
Intercept (per bar)	0.00033033 [seed-run.json]	3.2457 [seed-run.json]	0.00117 [seed-run.json]
Market	0.38935 [seed-run.json]	33.97 [seed-run.json]	$< 10^{-15}$
Trend (TSMOM)	0.26116 [seed-run.json]	22.50 [seed-run.json]	$< 10^{-15}$
Volatility regime	0.09274 [seed-run.json]	8.05 [seed-run.json]	8.9e-16 [seed-run.json]

with $R^2 = 0.5928$ [seed-run.json], idiosyncratic share 0.4072 [seed-run.json], annualised intercept 0.7234 [seed-run.json] and information ratio 4.397 [seed-run.json] — the last defined against **residual** volatility rather than total, since the risk actually taken beyond the factors is what the ratio is meant to price.

The departure. These are plain OLS standard errors. Strategy returns are heteroskedastic and mildly autocorrelated, so a heteroskedasticity- and autocorrelation-consistent estimator

$$\hat{\boldsymbol{\Omega}}_{\text{HAC}} = \hat{\boldsymbol{\Gamma}}_0 + \sum_{k=1}^{m} w_k\left(\hat{\boldsymbol{\Gamma}}_k + \hat{\boldsymbol{\Gamma}}_k^T\right), \quad w_k = 1 - \frac{k}{m+1} \tag{80}$$

would **widen** them. The significance reported above is therefore, if anything, generous. This is stated in the source rather than assumed away, and it is why the interface frames a significant intercept as "not explained by these three factors" and never as "real alpha". Equation 80 is not implemented.

The covariance estimator, and its conditioning

Both stacks compute the plain sample covariance over the window every symbol shares:

$$\hat{\Sigma}_{ij} = \frac{1}{n-1}\sum_{t=1}^{n}(r_{i,t}-\hat{\mu}_i)(r_{j,t}-\hat{\mu}_j) \tag{81}$$

Series are **truncated to the shortest common length, never padded**, because the padding failure is silent: zeros read as zero-return days, which understates that symbol's variance and — because the padded zeros line up across symbols — inflates every correlation toward one. Both errors make the book look safer than it is. Summation is sequential in both implementations, never `math.fsum` and never `numpy.dot`: pairwise summation rounds differently from JavaScript's left-to-right accumulation, and the parity fixture exists to catch that drift.

Conditioning. Equation 81 has rank at most $\min(n-1, p)$. Under the Marchenko–Pastur law with aspect ratio $c = p/n$, the eigenvalues of a sample covariance from i.i.d. Gaussian data with true covariance $\sigma^2\boldsymbol{I}$ spread over $\left[\sigma^2(1-\sqrt{c})^2, \sigma^2(1+\sqrt{c})^2\right]$, giving a condition number

$$\kappa \approx \left(\frac{1+\sqrt{c}}{1-\sqrt{c}}\right)^2 \tag{82}$$

At twelve symbols and sixty observations $c = 0.2$ and $\kappa \approx 6.9$ [illustrative]; at twelve symbols and the twenty-observation floor $c = 0.6$ and $\kappa \approx 62$ [illustrative]. Both are evaluations of Equation 82 under an assumption — i.i.d. Gaussian returns — that the desk's data does not satisfy, which is why they are marked illustrative rather than measured. The qualitative statement they support is the one that matters: the estimator degrades sharply as p approaches n, and the minimum-variance solver of Section 5.6.4 inverts against it.

No shrinkage is applied. The Ledoit–Wolf estimator $\hat{\Sigma}_\delta = \delta F + (1-\delta)\hat{\Sigma}$, with F a structured target and δ chosen to minimise expected Frobenius loss, is not implemented. The honest reason is that the shrinkage intensity δ would become a number the desk would have to defend on every screen that quotes a VaR, and the alternative — a documented sample floor with a refusal below it — is auditable in a way a shrunk matrix is not.

A divergence between the two implementations, and the fixture that cannot see it. The TypeScript requires at least twenty observations per symbol **and** at least twenty in the common window (`web/lib/portfolio-risk/covariance.ts`), returning `null` otherwise. The Python requires two (`build_covariance` in `modules/quant_risk/covariance.py`). A book whose history is short therefore gets a refusal in the browser and a two-observation covariance from the gateway — one estimate, two floors. The parity fixture cannot detect this: `tools/make_risk_fixture.py` generates 220 observations [make_risk_fixture.py] with a 60-bar window [make_risk_fixture.py], so no scenario in it ever presents a series short enough for the floors to disagree. This is stated here because it is exactly the class of defect that two implementations plus a fixture are supposed to prevent, and in this instance do not.

Euler allocation of risk to positions

Portfolio volatility $\sigma_p(\boldsymbol{w}) = \sqrt{\boldsymbol{w}^T\boldsymbol{\Sigma}\boldsymbol{w}}$ is homogeneous of degree one in $\boldsymbol{w}$: $\sigma_p(\lambda\boldsymbol{w}) = \lambda\sigma_p(\boldsymbol{w})$ for $\lambda > 0$. Euler's theorem for homogeneous functions therefore gives an **exact** additive decomposition, not an approximation:

$$\sigma_p = \sum_{i=1}^{p} w_i \frac{\partial \sigma_p}{\partial w_i}, \quad \mathrm{MCR}_i = \frac{\partial \sigma_p}{\partial w_i} = \frac{(\boldsymbol{\Sigma}\boldsymbol{w})_i}{\sigma_p}, \quad \mathrm{CCR}_i = w_i \mathrm{MCR}_i \tag{83}$$

The additivity is the entire point. A "risk contribution" that does not sum to the total is a ranking, not an attribution, and cannot answer the only question a risk manager asks of it: what do I cut to lose the most risk per dollar. The guard suite pins the identity directly — the components must sum to σ_p to within 10^{-12}, and the shares to one (`web/tests/portfolio-risk-contributions.test.ts`).

Two implementation decisions follow from Equation 83 and are worth naming.

Weights are fractions of equity, not of gross. Risk is measured against the capital that absorbs the loss. Using gross notional would report identical volatility for a book at $1\times$ and one at $5\times$ leverage. The suite pins the consequence: five times the exposure is five times the VaR, to 10^{-9}.

Contributions are signed. A hedge enters the quadratic form with a negative weight, its covariance with the longs subtracts, and its component contribution is **negative**. A position can be 30% of the notional and reduce total risk. That number cannot be produced by a notional-weighted view at all, which is why it exists.

The two stacks then diverge in what they summarise. The Python reports a diversification ratio $\sum_i |w_i|\ \sigma_i/\sigma_p$; the TypeScript reports the largest absolute pairwise correlation in the book. Both are defensible; they are different statistics, computed from the same matrix, and no fixture requires them to agree because they are not the same quantity.

Marginal, component and incremental VaR

Under the parametric model of Section 5.7, $\mathrm{VaR}_\alpha = z_\alpha \sigma_p E$ inherits the homogeneity of σ_p, so the decomposition carries over directly:

$$\mathrm{MVaR}_i = z_\alpha E \frac{(\boldsymbol{\Sigma}\boldsymbol{w})_i}{\sigma_p}, \quad \mathrm{CVaR}_i^{\mathrm{comp}} = w_i \mathrm{MVaR}_i, \quad \sum_i \mathrm{CVaR}_i^{\mathrm{comp}} = \mathrm{VaR}_\alpha \tag{84}$$

Incremental VaR — the change in book VaR from removing position i entirely — is a different object and does not sum to the total:

$$\mathrm{IVaR}_i = z_\alpha E\left(\sigma_p - \sqrt{\sigma_p^2 - 2w_i(\boldsymbol{\Sigma}\boldsymbol{w})_i + w_i^2\Sigma_{ii}}\right) \tag{85}$$

Equation 85 is exact, needs no new estimation, and costs one square root per position. It is **not implemented anywhere in this tree**. For small w_i it converges to $\mathrm{CVaR}_i^{\mathrm{comp}}$; for a large position the two diverge by the

concavity of σ_p, and the gap is precisely the interaction term a risk manager wants when the question is "what if we exit this line entirely" rather than "what is this line's share". Naming the absence is cheaper than letting a reader assume the component figure answers the incremental question.

5.7 Value at Risk and its validation

The parametric figure

$$\mathrm{VaR}_\alpha = z_\alpha \sigma_p E, \quad \mathrm{ES}_\alpha = \frac{\varphi(z_\alpha)}{1-\alpha}\sigma_p E \tag{86}$$

with $z_{0.95} = 1.6448536269514722$ and $z_{0.99} = 2.3263478740408408$, identical literals in both stacks. The expected-shortfall multiplier $\varphi(z_{0.95})/0.05$ is where they part company:

Source	Constant	Deviation from $\varphi(z_{0.95})/0.05$
`modules/quant_risk/_common.py`	2.0627128027825736 [_common.py]	-4.7×10^{-9}
`web/lib/portfolio-risk/risk.ts`	2.0627128054846826 [risk.ts]	-2.0×10^{-9}
Exact, double precision	2.0627128075074275	—

The two implementations differ by 1.3×10^{-9} relative and neither equals the exact value. The consequence is bounded: on a book with $\sigma_p E$ of ten thousand dollars the two expected shortfalls differ by about 3×10^{-5} dollars, far below the cent at which any surface rounds. It is recorded because the fixture tolerance, not the constants, is what keeps them agreeing — `web/tests/parity.test.ts` compares Sharpe and its relatives at 10^{-6} relative, three orders of magnitude looser than the gap, so the divergence is invisible to the mechanism built to catch divergences.

The empirical figure, and why the tail is selected by rank

The historical VaR replays today's book weights over past returns and reads the loss distribution directly, assuming nothing about its shape. With the replayed P&L sorted ascending and $k = \lceil (1-\alpha)n \rceil$,

$$\mathrm{VaR}^{\mathrm{hist}}_\alpha = -x_{(k)}, \quad \mathrm{CVaR}^{\mathrm{hist}}_\alpha = -\frac{1}{k}\sum_{j=1}^{k} x_{(j)} \tag{87}$$

The tail is selected **by rank**, never by value. Averaging everything at or below the VaR threshold is equal to Equation 87 only when the quantile is not a repeated value, and in a backtest it very often is: a strategy that is flat most of the time earns exactly zero on every bar it holds nothing, so at low exposure the fifth percentile lands on that atom of zeros, the "tail" becomes every non-positive bar, and the mean is taken over most of the sample. The comment in `web/lib/quant/tail-risk.ts` records what that cost when it was live — CVaR95 understated by 19.8 times [tail-risk.ts] and CVaR99 by 99 times [tail-risk.ts] on a default RSI run, with the two printed as the same number, which is the visible tell that selection was by value.

Both stacks refuse below twenty aligned observations. A percentile over a dozen days is a single data point wearing a statistic's name.

For the worked example the per-bar figures are $\mathrm{VaR}_{95} = -0.00809$ [seed-run.json], $\mathrm{CVaR}_{95} = -0.01305$ [seed-run.json], $\mathrm{VaR}_{99} = -0.01793$ [seed-run.json], $\mathrm{CVaR}_{99} = -0.02121$ [seed-run.json], with an ulcer index of 0.03853 [seed-run.json].

Backtesting the forecast: Kupiec

A VaR nobody has back-tested is an opinion. Count exceptions — days whose realised loss exceeded the forecast — and test the count against the model's own claim. With n scored observations, x exceptions, nominal rate $\alpha' = 1-\alpha$ and $\hat{p} = x/n$, the proportion-of-failures likelihood ratio is

$$\mathrm{LR}_{\mathrm{uc}} = -2\ln\left(\frac{(1-\alpha')^{n-x}\alpha'^{x}}{(1-\hat{p})^{n-x}\hat{p}^{x}}\right) \sim \chi^2_1 \tag{88}$$

and for one degree of freedom the survival function is exactly $P(\chi^2_1 > y) = \mathrm{erfc}\left(\sqrt{y/2}\right)$ — no series expansion, no dependency, correct to machine precision in the Python and to 1.5×10^{-7} in the TypeScript, which lacks `erfc` and uses the Abramowitz–Stegun 7.1.26 rational form. Both branch explicitly at $x = 0$ and $x = n$, where Equation 88 contains $\ln 0$.

The forecast is re-estimated on a rolling sixty-bar window and scored against the **next** bar's realised P&L, so it is never judged on data it was fitted to, with the book held at today's weights so that what is measured is the **model** rather than the trading. A p-value below 0.05 rejects in **either** direction: too many exceptions

understates risk, and too few means the desk is holding capacity it never uses. A model with zero exceptions is not conservative, it is wrong in the expensive direction.

Failure mode, and the test that is missing. Equation 88 examines **unconditional coverage only**. It cannot see clustering: five exceptions spread evenly across a hundred days and five arriving in one week produce the identical statistic, and only the second is a model failure a desk would care about. Christoffersen's independence test, built from the two-state transition counts n_{ij} of the exception indicator,

$$\mathrm{LR}_{\mathrm{ind}} \sim \chi^2_1, \quad \mathrm{LR}_{\mathrm{cc}} = \mathrm{LR}_{\mathrm{uc}} + \mathrm{LR}_{\mathrm{ind}} \sim \chi^2_2 \tag{89}$$

is **not implemented**. That absence is more pointed here than it would be elsewhere, because the measured absolute-return autocorrelation of 0.4201 [seed-run.json] at lag one says clustered exceptions are the **expected** failure of this data, not a hypothetical one. Power is also thin by construction: at sixty scored bars and $\alpha' = 0.05$ the expected exception count is three, and Equation 88 discriminates poorly between three and six.

5.8 Optimal execution

The Almgren–Chriss formulation

Liquidate X units over horizon T in N intervals of length $\tau = T/N$. Let x_j be the holding at $t_j = j\tau$, with $x_0 = X$ and $x_N = 0$, and $n_j = x_{j-1} - x_j$ the trade in interval j. Impact is split into a permanent component $g(v) = \gamma v$, which moves the reference price and never comes back, and a temporary component $h(v) = \varepsilon\,\mathrm{sgn}(v) + \eta v$, which is paid on the trade that causes it. The price evolves

$$S_j = S_{j-1} + \sigma\sqrt{\tau}\xi_j - \tau g(n_j/\tau), \quad \tilde{S}_j = S_{j-1} - h(n_j/\tau) \tag{90}$$

with ξ_j i.i.d. standard normal. Implementation shortfall $C = XS_0 - \sum_j n_j \tilde{S}_j$ then has

$$\mathbb{E}[C] = \frac{\gamma}{2}X^2 + \varepsilon\sum_{j=1}^{N}|n_j| + \frac{\tilde{\eta}}{\tau}\sum_{j=1}^{N} n_j^2, \quad \mathrm{Var}[C] = \sigma^2\tau\sum_{j=1}^{N-1} x_j^2 \tag{91}$$

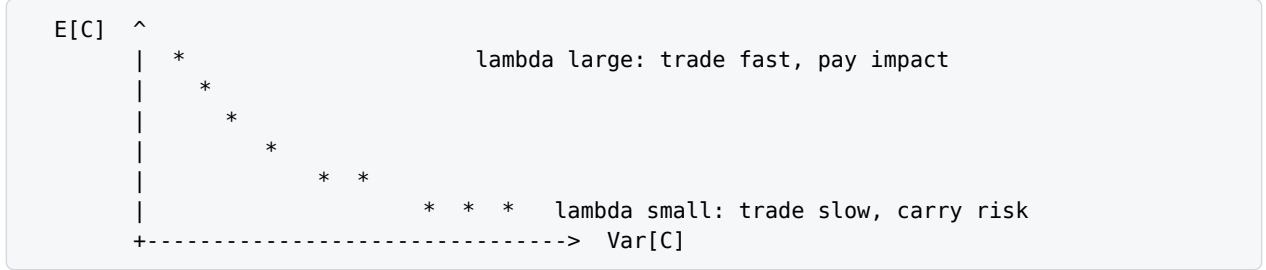
where $\tilde{\eta} = \eta - \gamma\tau/2$. The trader minimises $\mathbb{E}[C] + \lambda\,\mathrm{Var}[C]$ for risk aversion $\lambda > 0$. Differentiating with respect to an interior x_j — noting x_j appears in both n_j and n_{j+1} — gives

$$\frac{2\tilde{\eta}}{\tau}\left(2x_j - x_{j-1} - x_{j+1}\right) + 2\lambda\sigma^2\tau x_j = 0 \quad\Longrightarrow\quad \frac{x_{j-1} - 2x_j + x_{j+1}}{\tau^2} = \tilde{\kappa}^2 x_j \tag{92}$$

with $\tilde{\kappa}^2 = \lambda\sigma^2/\tilde{\eta}$. Equation 92 is a discrete second-order boundary-value problem; with $x_0 = X$, $x_N = 0$ its solution is the hyperbolic trajectory

$$x_j = X\frac{\sinh\left(\kappa(T - t_j)\right)}{\sinh(\kappa T)}, \quad \frac{2}{\tau^2}(\cosh(\kappa\tau) - 1) = \tilde{\kappa}^2 \tag{93}$$

and $\kappa = \tilde{\kappa} + O(\tau^2)$ as $\tau \to 0$. The two corners are worth naming because the desk lives in one of them. As $\lambda \to 0$, $\kappa \to 0$ and Equation 93 degenerates to $x_j = X(1 - t_j/T)$, the constant-rate liquidation — TWAP. As $\lambda \to \infty$ the trajectory collapses to $x_1 = 0$: sell everything immediately and pay the impact. Sweeping λ traces the efficient frontier of execution, along which $d\mathbb{E}[C]/d\,\mathrm{Var}[C] = -\lambda$, with $1/\kappa$ the half-life of the position.



Each point on that curve is one optimal trajectory Equation 93, and the desk occupies its lower-right end by default: constant participation, Equation 95.

What the desk implements, and what it does not

The Almgren–Chriss scheduler is NOT BUILT. The reason is structural rather than mathematical: the order path is single-shot. An order is priced, gated and either sent or refused; there is no child-order clock, no working-order slicer, and therefore nothing that would execute a trajectory $x(t)$. A solved trajectory with no executor is precisely the capability-with-no-caller defect this repository keeps a scar about, so it is absent rather than stubbed.

What does exist is three pieces of the same problem, each measurable, each cited.

A concave impact model. `turnoverCost` in `web/lib/quant/cost-model.ts` charges

$$c(Q) = \frac{\text{fee}_{\text{bps}} + \text{slip}_{\text{bps}}}{10^4} + k\sqrt{\min(1, Q/\ \text{ADV})} \tag{94}$$

the square-root law, in place of the linear $h(v) = \eta v$ of Equation 90. Doubling order size costs about 1.41 ×, not 2 ×. The consequence is immediate and is the reason the AC closed form cannot simply be adopted here: Equation 93 is a consequence of the cost being **quadratic** in the trade rate. A concave h gives a different Euler–Lagrange equation and no hyperbolic solution. The coefficient k defaults to zero, so an unconfigured run reproduces exactly the figures the parity fixture pins, and the interface labels the result a model rather than a measurement — a slippage figure a researcher chose is an assumption they are making.

A ladder walk that measures rather than models. `walkBook` and `smartRoute` in `web/lib/venues/fill-tolerance.ts` consume a real consolidated book level by level and return the achievable VWAP and its slippage against the depth-weighted mid. Under a blended-slippage bound $v_{\max}$ a boundary level is partially consumed in closed form — with running notional N and quantity Q at level price p, the admissible take for a buy is $t = p(v_{\max}Q - N)/(p - v_{\max})$ — and a cap with no resolvable mid routes nothing, because enforcing a cap without a reference price would be a lie.

A constant-participation horizon. `timeToLiquidate` in `web/lib/liquidity.ts` reports

$$T_{\text{exit}} = |Q\frac{|}{P \cdot \text{ADV}}, \quad P = 0.10 \text{ [liquidity.ts] by default} \tag{95}$$

Equation 95 **is** the $\lambda \to 0$ corner of Equation 93, expressed in days rather than as a trajectory: constant participation is constant rate is TWAP. The desk therefore already computes the risk-neutral schedule's **duration** without ever computing the schedule.

And the pre-trade gate. `MAX_EST_SLIPPAGE_BPS` defaults to 75 [config.py] and `MAX_PRICE_DEVIATION_BPS` to 500 [config.py] an order whose measured walk exceeds the first is refused.

What the missing scheduler would be worth

The pieces above admit the trade-off symbolically, which is the honest way to say what is being forgone. Under Equation 94 the total **dollar** cost of a single-shot order is

$$C_1(Q) = Q \cdot k\sqrt{Q/\ \text{ADV}} = kQ^{3/2}/\sqrt{\text{ADV}} \tag{96}$$

Splitting into m equal children of size Q/m, each paying impact on its own size,

$$C_m = m \cdot \left(\frac{Q}{m}\right) k\sqrt{\frac{Q}{m\ \text{ADV}}} = C_1/\sqrt{m} \tag{97}$$

Two consequences follow immediately. First, the single-shot figure the gate enforces is an **upper bound** on the cost of any schedule, so the gate errs in the safe direction — it refuses some orders a scheduler could have worked. Second, the saving has a price: spreading over m children at fixed spacing τ carries timing risk over a horizon $m\tau$, which under Equation 91 is proportional to $\sigma Q\sqrt{m\tau}$. Minimising $Am^{-1/2} + \lambda Bm^{1/2}$ gives

$$m^* = \frac{A}{\lambda B}, \quad C(m^*) = 2\sqrt{\lambda AB} \tag{98}$$

the same square-root frontier shape as Equation 92 produces, with a different exponent inside it. Equation 98 is a derivation, not a measurement: no schedule has been run, and no number is quoted for A, B or λ because the desk has never measured its own risk aversion.

5.9 Tracking error, drawdown and position sizing

Tracking error and the information ratio

Against an external benchmark with returns b_t, the active return is $a_t = r_t - b_t$ and

$$\text{TE} = \sqrt{a} \cdot \sqrt{\frac{1}{n-1}\sum_t (a_t - \bar{a})^2}, \quad \text{IR} = \frac{a \cdot \bar{a}}{\text{TE}} \tag{99}$$

`compareToBenchmark` in `web/lib/benchmark.ts`, which reuses Equation 79 with the benchmark as a single factor rather than writing a second two-variable regression that would drift from the shared one within a release. Correlation is recovered as $\text{sgn}\left(\hat{\beta}\right)\sqrt{R^2}$, identical to computing it directly for a single-factor fit and one fewer place for the two to disagree by a rounding step.

Two guards, both with measured reasons. The information ratio is `null` when $\text{TE} \le 10^{-9}$: a strategy that replicates its benchmark produces active returns that are zero to within a couple of ulps, not exactly zero,

leaving a tracking error near 10^{-17} and an information ratio in the billions — float residue presented as the best risk-adjusted result ever recorded. And the join is on a bucket key derived from the interval rather than on raw timestamps, because two vendors rarely stamp the same bar identically and an empty intersection through a regression is not an error but a `null`. Fewer than 30 [benchmark.ts] aligned bars and no comparison is reported at all.

Two different objects both called drawdown

$$\mathrm{DD}_t = \frac{E_t}{\max_{s \le t} E_s} - 1 \quad \text{against} \quad \mathrm{DD}_t^{\text{open}} = \max\left(0, -\frac{E_t - E_{\text{open}}}{E_{\text{open}}}\right) \tag{100}$$

The first is drawdown from the running high-water mark, which is what `drawdownSeries` in `web/lib/portfolio-analytics.ts` draws and what the ulcer index $U = \sqrt{\overline{\mathrm{DD}^2}}$ integrates. The second is what the circuit breaker actually reads (`daily_drawdown_pct`, `modules/risk_proxy/accounting.py`): loss from **start-of-day equity**, floored at zero. They are not the same number, and $\mathrm{DD}^{\text{open}} \le |\mathrm{DD}|$ always — a book that rallied 3% and then fell 4% is 1% down on the open and 4% off its peak. The breaker therefore trips strictly later than a peak-referenced rule would, and any calibration of it must be done in the units it measures.

The response is a graduated ladder rather than a switch, read from `config.py`:

Level	Default	Behaviour
`ALERT_DRAWDOWN_PCT`	3% [config.py]	Breach pushed to subscribers; trading unaffected
`REDUCE_ONLY_THRESHOLD`	0.80 [config.py] of budget, i.e. 4%	Risk-increasing orders refused, risk-reducing orders still pass
`MAX_DAILY_DRAWDOWN_PCT`	5% [config.py]	Halt

The alert sits well below the halt on purpose: an alert that fires at the same moment as the kill switch is a notification, not a warning. The middle rung is the FIA practice of letting a desk close out of trouble but not deeper into it.

From drawdown budget to position size

Model the book's log equity over one session as Brownian motion with zero drift and per-session volatility σ_d. The breaker monitors continuously, so the quantity it tests is a first-passage probability, and by the reflection principle

$$P\left(\min_{t \le 1} \ln(E_t/E_{\text{open}}) \le -D\right) = 2\Phi(-D/\sigma_d) \tag{101}$$

Inverting for a target trip frequency p gives the largest session volatility consistent with the budget:

$$\sigma_d^{\max} = \frac{D}{-\Phi^{-1}(p/2)} \tag{102}$$

At $D = 5\%$ and one expected trip per 365-session year, Equation 102 gives $\sigma_d^{\max} = 1.56\%$ [illustrative] per session. It is marked illustrative because the model behind it — zero drift, continuous monitoring, Gaussian log-returns — is one the desk's measured $\gamma_4 = 14.9$ flatly contradicts; fat tails make the true trip probability higher than Equation 101 at the same σ_d. The structural claim survives the marking: **the drawdown budget is a volatility ceiling**, and position sizing is the mechanism that enforces it.

Kelly, quartered and capped

For a strategy winning a fraction W of trades at payoff ratio $R = \overline{\text{win}}/\overline{\text{loss}}$, the growth-optimal fraction is

$$f^* = W - \frac{1 - W}{R} \tag{103}$$

implemented identically as `kellySizing` (`web/lib/quant/sizing.ts`) and `kelly_fraction` (`modules/quant_risk/sizing.py`). The desk trades 0.25 [sizing.ts] $\cdot \max(0, f^*)$, capped at 0.20 [sizing.ts] of the book, and the cap names itself in the result so a fraction that silently stopped growing cannot read as a recommendation.

The quarter is not timidity, and it is derivable. In the continuous Gaussian approximation with per-unit-time drift μ and volatility σ, log-wealth grows at $g(f) = f\mu - f^2\sigma^2/2$, maximised at $f^* = \mu/\sigma^2$ with $g(f^*) = \mu^2/(2\sigma^2)$. The function is a downward parabola through the origin with its second root at $2f^*$:

$$g(2f^*) = 0 \tag{104}$$

Betting **twice** the optimal fraction produces zero long-run growth at maximal volatility. Since W and R are estimates from a search that has already been optimised once, over-estimation is the expected error, and Equation 104 says the penalty for over-betting is unbounded while the penalty for under-betting is merely slower growth.

The drawdown link makes the choice quantitative. For a book run at fraction cf^* and rebalanced continuously, log-wealth is Brownian with drift $m = cf^*\mu - (cf^*)^2\sigma^2/2$ and volatility $s = cf^*\sigma$, so the probability of **ever** falling to a fraction $a < 1$ of starting wealth is $\exp(-2m\ln(1/a)/s^2) = a^{2/c-1}$:

$$P(\text{ever reach } \; aE_0) = a^{2/c-1} \tag{105}$$

At full Kelly ($c = 1$) the probability of ever halving the book is $1/2$. At $c = 1/4$ it is $(1/2)^7 \approx 0.008$. Equation 105 is the argument for `DEFAULT_KELLY_FRACTION = 0.25` stated in the units the drawdown budget of Equation 100 is written in, and it is a derivation from a stated model, not a measurement of this desk.

The worked example, and why the sizing module refuses. From `seed-run.json`: $W = 0.75$ [seed-run.json], $\overline{\text{win}} = 0.047141$ [seed-run.json], $\overline{\text{loss}} = 0.033603$ [seed-run.json], so $R = 1.4029$ and Equation 103 gives $f^* = 0.5718$ — full Kelly would put 57% of the book on this strategy. Quarter Kelly is 14.3%, inside the 20% cap. And the strategy has 8 trades [seed-run.json], against a `MIN_TRADES_FOR_SIZING` of 30 [sizing.ts], so the result carries `thinSample: true`. The formula is indifferent to whether R came from eight samples or eight hundred; Equation 104 says the consequence of being wrong is not. This is the same candidate whose DSR is 0.42 and whose out-of-sample Sharpe is -1.06.

Three refusals are written into the implementation and each is a modelling statement. A strategy with no losing trades has an **undefined** payoff ratio, not an infinite one — treating it as infinite drives Equation 103 to W itself and sizes a small lucky sample at maximum, so it returns zero. A negative f^* returns zero rather than an inverted position, because an edge that only exists when you flip the signal is a fitting artefact. And the cap is reported with the name of the constraint that bound it.

What is not implemented. Equation 103 is a single-strategy result. The correlated multi-sleeve solution is $\boldsymbol{f}^* = \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}$, which requires a vector of expected returns. The allocation engine deliberately refuses to forecast one — `propose_allocation` allocates by risk alone and its own note says so: "no expected return is forecast, so this answers how should the risk be spread, never what should we own". The per-strategy cap of 20% is what stands in for the joint solution, and it is a blunt instrument rather than an approximation of one.

5.10 The bootstrap

The stationary bootstrap

Politis and Romano (1994). Draw block lengths $L \sim \text{Geom}(p)$, $P(L = \ell) = (1-p)^{\ell-1}p$, so $\mathbb{E}[L] = 1/p = b$, and concatenate blocks starting at uniformly chosen positions, wrapping the index modulo n:

$$i_{t+1} = \begin{cases} \text{Uniform}\{0 \\ \dots \\ n-1\} & \text{with probability } 1/b \\ (i_t + 1) \bmod n & \text{otherwise} \end{cases} \tag{106}$$

The wrap is what earns the name. Künsch's moving-block bootstrap draws fixed-length blocks without wrapping, and the resulting resample is **not** stationary — observations near the ends of the series are systematically under-drawn. Equation 106 with the geometric length and the modular wrap produces a resample that is strictly stationary, which is the property the downstream percentile estimates rely on.

Both stacks implement Equation 106 with the identical two-draws-per-step order — a uniform to decide whether to start a block, then an index — (`modules/quant_risk/montecarlo.py`, `web/lib/montecarlo.ts`), pinned by `web/tests/fixtures/mc-resampler-parity.json`. The Python separates the $b = 1$ case into its own loop that consumes **one** draw per step rather than two, specifically so that routing $b = 1$ through the block loop cannot silently move every existing Monte Carlo figure for the same seed. That is a reproducibility constraint expressed as control flow.

The block-length heuristic, and its departure

$$b = \text{clamp}\left(\left\lfloor \sqrt{n} + \frac{1}{2} \right\rfloor, 5, 100\right) \tag{107}$$

For the worked example $n = 1200$ and Equation 107 gives 35, matching `meanBlockLength: 35` [seed-run.json] in the recorded run. The `floor(x + 0.5)` is written out rather than spelled `round` because Python rounds halves to even and ECMAScript rounds them up — a difference no integer square root can actually reach, written explicitly so the two stacks are provably one rule rather than two that happen to agree.

The departure. Politis and White (2004) show the optimal mean block length for the stationary bootstrap grows as $O(n^{1/3})$ for variance and distribution estimation. At $n = 1200$ that rate suggests roughly 10.6; Equation 107 gives 35, over-blocking by about 3.3 ×. The tree names it a heuristic rather than an optimum, and the clamp to $[5, 100]$ is what keeps it bounded. The cost of over-blocking is fewer effective blocks per resample and therefore higher variance in the resulting quantile estimates; the benefit is that more of the dependence structure survives. Section 5.10.4 measures which side of that trade this data lands on, and the answer is not the one the documentation assumes.

What blocks preserve that an i.i.d. draw destroys

The autocorrelation table of Section 5.2.2 answers this directly and the answer is sharper than the usual telling. The **linear** autocorrelation of these returns is negligible — $|\rho_1| = 0.0048$, and no lag out to five exceeds 0.027 in absolute value. There is essentially nothing there for a block draw to preserve. The autocorrelation of **absolute deviations** is 0.4201 [seed-run.json] at lag one and remains above 0.27 out to lag five. What an i.i.d. resample destroys in this series is not linear dependence, which is absent, but **volatility clustering**, which is large.

That distinction matters because the two are conflated constantly. A series can be serially uncorrelated — and so pass every test a linear model applies — while being strongly dependent in a way that governs exactly the tail behaviour a Monte Carlo is drawn to estimate.

The measured direction, which is not the one the docstring asserts

`web/lib/montecarlo.ts` states, without qualification, that i.i.d. resampling "destroys the clustering and reports a cone that is too narrow exactly where it matters, in the tails". That claim is testable against the tree's own resampler. Running `bootstrap_terminal_distribution` on the 1 200 recorded per-bar returns scaled to a \$1 000 000 book, 20 000 paths, comparing the i.i.d. draw against $b = 35$:

Horizon (bars)	i.i.d. VaR_{95}	block-35 VaR_{95}	ratio
5	\$18 611	\$18 935	1.017
10	\$24 991	\$25 249	1.010
20	\$35 088	\$30 784	0.877
60	\$57 358	\$39 480	0.688

measured
2026-08-22 with `bootstrap_terminal_distribution(pnl, h, paths=20000, seed=20260822, ...)` over `seed-run.json`'s `bestRunReturns`. The $h = 60$ ratio reproduces at 0.688, 0.695 and 0.694 across three independent seeds, so it is not sampling noise.

The direction **reverses at $h \approx b$**, and the mechanism is legible. While the horizon is no longer than the mean block, a simulated path is essentially one contiguous historical window: it inherits the real clustering, and the block draw is the wider one, as the docstring says. Once the horizon is several blocks long, the path is a sum of block-totals that have already been partially Gaussianised **inside** each block by the central limit theorem, so single extreme bars can no longer stack — while the i.i.d. sum of 60 draws from a distribution with $\gamma_4 = 14.9$ keeps its kurtosis-driven tail alive. At $h = 60$ against $b = 35$ the i.i.d. figure is the more conservative one by 45%.

Neither number is wrong. The unqualified claim is: the direction is a property of the series and the horizon relative to the block, not of the resampler. A desk reading the 60-bar terminal distribution and choosing the block draw because it is "the conservative one" would, on this data, be choosing the narrower tail.

Failure modes of the bootstrap

It cannot manufacture tail shape the sample never showed. This is not a limitation to be worked around; it is what resampling **is**. Hence the floor: `bootstrap_terminal_distribution` returns `None` below sixty observations, and either figure is reported **beside** the historical VaR, never instead of it.

The two stacks default to different resamplers, deliberately. The Python's unstated default is the i.i.d. draw; the TypeScript's is the derived block. Each is the draw that surface has always shown, and changing either would move a figure a desk has been reading with no code that looks like it changed a number. A caller

who cares names the resampler, and naming it gives the same resampler on both sides. A contradiction — i.i.d. with a block above one, or stationary with a block of exactly one — raises rather than picking a winner silently, because a run that quietly used the other resampler is the one thing no card could report afterwards. **Seeding is provenance, not decoration.** The seed defaults to the CRC32 of the input series, so refreshing the same book redraws the same cone without any stored state, and a different book draws a fresh one.

5.11 Summary of departures from the textbook form

Model	Textbook	Here, and why
Sharpe annualisation	$\eta(q)$ from Equation 68	$\sqrt{a}$, uncorrected. $\eta(a)$ needs autocorrelations to lag $a-1$ from $n < a$ observations. All inference is done per bar instead.
Expected max of N	$\bar{S} + S_0^*$ in some implementations	Mean discarded. Adding it back makes the hurdle negative on a uniformly losing grid.
Factor t-statistics	HAC / Newey–West, Equation 80	Plain OLS. Stated as generous rather than corrected; framing is "not explained by these factors".
Covariance	Shrinkage toward a structured target	Sample covariance, sequential summation for cross-language bit-parity, with a documented sample floor and a refusal below it.
Sample floors	One floor	Twenty observations in TypeScript, two in Python. A real divergence the 220-observation parity fixture cannot see.
Expected shortfall	$\varphi(z_\alpha)/(1-\alpha)$	Two literals differing at 1.3×10^{-9}, both below the exact value; fixture tolerance is 10^{-6}.
Tail selection	Mean of everything below the quantile	Mean of the worst $\lceil (1-\alpha)n \rceil$ by **rank**. Value-selection understated CVaR95 by $19.8\times$ on a live run.
VaR validation	Christoffersen conditional coverage, Equation 89	Kupiec unconditional coverage only. The independence half is absent, and clustering is the measured failure mode of this data.
Execution	Linear $h(v) = \eta v$, trajectory Equation 93	Square-root law Equation 94, from which the hyperbolic solution does not follow; and no scheduler at all. Single-shot order path, no child-order clock. Equation 95 computes the risk-neutral schedule's duration without the schedule.
Kelly	$\boldsymbol{f}^* = \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}$ across sleeves	Per-strategy Equation 103 at one quarter, capped at 20%. No expected-return vector is forecast anywhere, deliberately.
Block length	$O(n^{1/3})$	$\sqrt{n}$ clamped to $[5, 100]$, over-blocking by roughly $3.3\times$ at $n = 1200$.

5.12 What is not built, and the reason each waits

Stated plainly, in the manner of the product requirements document, because an absence named is auditable and an absence implied is not.

Autocorrelation-adjusted annualisation (Lo 2002) — the estimator has no data at $q = a$, and extrapolating from a short horizon would carry a precision the sample cannot support.
HAC standard errors, Equation 80 — would widen the factor t-statistics, so the current figures are generous in a known and stated direction.
Ledoit–Wolf shrinkage — the shrinkage intensity becomes a number the desk must defend on every screen quoting a VaR, where a documented floor with a refusal is auditable.
Incremental VaR, Equation 85 — exact, one square root per position, absent. The component figure answers a different question.
Christoffersen independence, Equation 89 — the half of VaR validation that detects clustered exceptions, which the measured $\rho_1(|r|) = 0.4201$ [seed-run.json] says is this data's expected failure.
Effective trial count for the DSR — the grid's trials are strongly dependent, so Equation 76 overstates the hurdle by an unknown margin. The safe direction is not a reason to stop saying so.
The Almgren–Chriss scheduler — no child-order clock exists to execute a trajectory. The single-shot cost is an upper bound by Equation 97, so the pre-trade gate errs safe, and Equation 95 already reports the risk-neutral horizon.
Joint Kelly across correlated sleeves — requires an expected-return vector the allocation engine deliberately refuses to forecast. The 20% per-strategy cap stands in for the solution rather than approximating it.

6 Infrastructure and telemetry specifications

This chapter is the operational half of the document. Chapters 2 to 5 argue what the desk decides and why the mathematics is sound; this one specifies where the resulting state lives, which store is believed when two disagree, what travels over which protocol, at what threshold an automatic control engages and releases, and which continuous-integration job proves which claim. Every schema below is transcribed from the file that creates it, every threshold from the file that reads it, and every figure carries the artefact it came from.

6.1 The persistence topology, and what "authoritative" means here

Five stores are in play and only one of them is authoritative for decisions. The distinction is not decorative. A system with two authoritative stores has no authoritative store; it has a reconciliation problem it has not noticed yet.

Store	Authoritative for	Durability	Rebuild
DuckDB audit log	orders, order events, risk events, TCA snapshots, backtest runs, equity snapshots	Docker named volume on the OCI VM	none, it is the source
SQLite data-operations ledger	quality findings, escalations, schedule runs, work items	same mounted volume	none, it is the source
Supabase Postgres	nothing; mirror of decisions plus the research	managed	replay from the audit log

	corpus		
Neo4j	nothing; a projection of `research_edges`	managed, optional	drop and re-project
Oracle Autonomous Database	nothing; an in-database simulation surface	managed, optional	re-apply the schema

The rule that follows: **the audit log is embedded on purpose**, because the desk must keep recording decisions when every network dependency is unreachable, and everything downstream is a view. If Supabase is empty, the desk trades. If the audit volume is empty, the desk has forgotten what it did, which is the only failure in this list that cannot be repaired from somewhere else.

One writer, enforced twice

The gateway runs as a single Uvicorn worker because the position book, the resting-order book, the token bucket and the kill switch are plain objects on one heap. Two mechanisms stop a second writer, and they close different holes.

`tests/test_container_contract.py` fails the build if `--workers` or `gunicorn` appears in the committed image definition. That reads the artefact, so it cannot see `docker compose up --scale gateway=2`, a second container pointed at the same named volume, or a command typed at a shell. `modules/single_writer.py` closes that with a POSIX advisory `flock(2)` claim on one file in `DATA_DIR`, taken in `RiskGateway.start()` and held for the life of the process. The reason given for `flock` over a lock row in a database is that the kernel releases it when the holder dies by any route, `SIGKILL` and out-of-memory included, so there is no stale lock, no lease to renew and no timeout to tune.

Behind both sits a third guard in the audit store. `AuditStore._connect` formerly caught every exception from `duckdb.connect` and fell back to SQLite at a different path — correct for one of the two meanings of that exception and catastrophic for the other, because a second live process holding the database is also reported as an IO error. The second gateway therefore did not fail: it opened a private ledger and began writing a divergent history while `/health` reported `backend: sqlite` as though somebody had chosen it. The lock case is now matched on the two phrases DuckDB builds that message from and raised as `AuditLedgerConflict`; every other IO error still takes the fallback it always took (`modules/audit/store.py`).

6.2 The DuckDB audit log

Ten tables, created with `CREATE ... IF NOT EXISTS` in `modules/audit/schema.py` and nowhere else. Columns added after the first databases existed are widened by `AuditStore._migrate` instead: a column appended to the DDL list is created on a fresh database and missing on every existing one, so the two diverge silently and only whichever query names that column ever finds out.

The two central tables:

```
CREATE TABLE IF NOT EXISTS orders (
    ts              TIMESTAMP,   order_id        VARCHAR,
    client_order_id VARCHAR,     strategy        VARCHAR,
    symbol          VARCHAR,     side            VARCHAR,
    order_type      VARCHAR,     quantity        DOUBLE,
    notional        DOUBLE,      limit_price     DOUBLE,
```

```
    accepted        BOOLEAN,     rejected_by     VARCHAR,
    reason          VARCHAR,     latency_ms      DOUBLE,
    fill_price      DOUBLE,      fill_qty        DOUBLE,
    fee_usd         DOUBLE,      slippage_bps    DOUBLE,
    venue           VARCHAR,     checks_json     VARCHAR,
    source          VARCHAR,     status          VARCHAR,
    time_in_force   VARCHAR,     decided_at      TIMESTAMP
);

CREATE TABLE IF NOT EXISTS risk_events (
    ts   TIMESTAMP, event VARCHAR, severity VARCHAR,
    actor VARCHAR,  symbol VARCHAR, detail VARCHAR, payload VARCHAR
);
```

`decided_at` sits beside `ts` because a resting order fills hours after it was decided and one timestamp cannot carry both facts. `checks_json` carries the full gate battery rather than a single verdict, because an order routinely fails several gates at once and a blotter that can name only one has lost the diagnosis. The other eight tables are `order_events` (place, amend, cancel, fill, with `replaces` linking an amendment to what it superseded), `tca_snapshots`, `backtest_runs` (carrying `dsr`, `pbo`, `oos_sharpe` and the `data_hash` tying a run to the exact bars it saw), `equity_snapshots`, `jobs`, `subscribers`, `telegram_link_tokens` and `ohlcv_cache`.

Append-only by convention, and two different write contracts

Nothing in `modules/audit/` issues `UPDATE` or `DELETE` against `orders` or `risk_events`. DuckDB cannot enforce that with a trigger, so the property is carried by the module boundary and by review; the Postgres mirror, which can enforce it, does (§6.4).

Within that module there are deliberately two write contracts, and mixing them up would be a real defect in either direction:

- **Fire and forget.** `_exec` swallows a failed write and `query` returns an empty list. This is right for evidence: losing a TCA snapshot must never take the order path down with it.
- **Strict.** `modules/audit/boundaries.py` holds the two durable replay boundaries, `book_reset` and `session_rollover`. Both writes raise on failure and both reads raise on a closed or unreadable store, because silently treating an audit failure as a flat book would understate exposure. Both reads also refuse an exact timestamp tie rather than guess which of two boundaries happened second.

Replay as an algebra

The current session's paper book is not persisted as a position snapshot; it is recomputed. Let F_d be the accepted, filled orders of UTC session d ordered by $(t, \text{order\_id})$, and let $r_d = \max\{t : \text{book\_reset} \in \text{risk\_events}, t \in d\}$ be the last reset boundary in that session. Then the rebuilt position in symbol s is

$$q_s = \sum_{f \in F_d, t_f > r_d} \varepsilon_f \cdot q_f, \quad \varepsilon_f = \begin{cases} +1 \text{ if } f \text{ is a buy} \\ -1 \text{ if } f \text{ is a sell} \end{cases} \tag{108}$$

with the realised cash leg accumulated over the same set. Two properties are load-bearing: `book_reset` is durable, so a reset survives a restart and is not undone by replay; and the filter is `status IS NULL OR status = 'FILLED'` rather than a test on `fill_qty`, because an accepted order that was cancelled or expired never filled by design, and letting it through would make the caller read missing fill evidence as corruption and refuse to start. What replay deliberately does **not** reconstruct is overnight **positions**: only the profit and loss earlier sessions banked and the equity this one opened on survive the boundary, from `latest_session_rollover`. Positions need a durable snapshot this schema does not carry, and the module says so rather than approximating it.

6.3 The SQLite data-operations ledger

Four tables hold the state the gateway must not forget across a restart and for which the audit log's fire-and-forget contract is wrong: a work item a person just edited, or a quality finding another instance is about to read, needs a write that raises when it fails and an `UPDATE` that reports whether it hit a row. They live in a stdlib `sqlite3` file on the same mounted volume (`modules/data_ops_store.py`).

```
CREATE TABLE IF NOT EXISTS data_quality_findings (
    id INTEGER PRIMARY KEY, instance TEXT NOT NULL, seq INTEGER NOT NULL,
    source TEXT NOT NULL, observed_at REAL NOT NULL, received_at REAL NOT NULL,
    capability TEXT NOT NULL, provider TEXT NOT NULL, symbol TEXT,
    key TEXT NOT NULL, passed INTEGER NOT NULL, fatal INTEGER NOT NULL,
    warn INTEGER NOT NULL, drift INTEGER NOT NULL, not_evaluated INTEGER NOT NULL,
    checks_json TEXT NOT NULL, UNIQUE(instance, seq));

CREATE TABLE IF NOT EXISTS data_work_items (
    id TEXT PRIMARY KEY,
    kind     TEXT NOT NULL CHECK(kind     IN ('request','ticket','bug')),
    priority TEXT NOT NULL CHECK(priority IN ('P0','P1','P2','P3')),
    status   TEXT NOT NULL CHECK(status   IN ('intake','ready','progress','resolved')),
    title TEXT NOT NULL, summary TEXT NOT NULL DEFAULT '',
    owner TEXT NOT NULL DEFAULT 'Unassigned', area TEXT NOT NULL DEFAULT 'Pipeline',
    opened_at REAL NOT NULL, sla_due_at REAL, resolved_at REAL,
    created_by TEXT NOT NULL, updated_at REAL NOT NULL, updated_by TEXT NOT NULL,
    version INTEGER NOT NULL DEFAULT 1);
```

`UNIQUE(instance, seq)` is what makes finding ingestion idempotent under retry from any number of web instances. `not_evaluated` is a first-class count beside `passed`, `fatal`, `warn` and `drift`, which is the schema-level expression of the house rule that a check which could not run is not a check that passed. The remaining two tables are `data_quality_escalations` (with `acknowledged_at` and `acknowledged_by` added later through a conditional widening, `NULL` meaning nobody has taken it, which is also true of every row that predates the column) and `data_schedule_runs`, keyed by `schedule_id` with `last_run_at`, `last_job_id` and `last_outcome`.

The connection pragmas, and the concurrency lesson behind them

```
conn = sqlite3.connect(path, timeout=BUSY_TIMEOUT_S,
                       check_same_thread=False, isolation_level=None)
conn.execute("PRAGMA journal_mode=WAL")
conn.execute("PRAGMA synchronous=NORMAL")
conn.execute("PRAGMA foreign_keys=ON")
```

`BUSY_TIMEOUT_S` is 30 s [modules/data_ops_store.py] against Python's default of five, and the reason is a diagnosis rather than a superstition. `PRAGMA journal_mode=WAL` is the first statement on every fresh connection and it has to read the file. A WAL reader never waits for a WAL writer, but it does wait for the `EXCLUSIVE` lock the **last** connection takes as it closes, to checkpoint the WAL and delete the `-wal` and `-shm` files. Under the previous test fixture that close happened whenever the garbage collector reached a leaked handle, on whatever thread it was running, while the next test was opening the same file, and on a loaded runner the checkpoint outlasted five seconds. Thirty seconds is a wait a lock-holder cannot plausibly exceed and is far shorter than a red build.

A busy timeout covers a lock another connection **holds**. It does not cover two connections in one process opening the same file at the same instant and both running the WAL pragma. Measured on a fresh file with six threads released by a barrier, 2 of 240 opens [modules/data_ops_store.py] failed immediately with `database is locked`, in 0.00 s, with the busy handler never consulted. That is how the gateway's own shutdown failed in the suite: the event loop and a worker thread each found the shared store unbuilt and each opened it. A process-wide `threading.Lock` around `open_data_ops_db` turns a race SQLite refuses to wait out into a queue; opens are rare and cost about a millisecond.

The connection is long-lived on purpose. A store that reconnected per statement would pay the WAL open every time and, as the last handle each time, the checkpoint-and-delete on close, which is precisely the contention the module exists to avoid.

Versioned edits, and the second backend

Concurrent edits use optimistic concurrency rather than a lock held across a human's thinking time: `BEGIN IMMEDIATE`, then `UPDATE ... WHERE id = ? AND version = ?`, and a row count of zero is reported to the caller

as a conflict rather than retried. `DATA_OPS_BACKEND=postgres` swaps the store for one speaking PostgREST, and the translation is exact rather than approximate.

Concern	`sqlite` (default)	`postgres`
Storage	one file on the mounted volume	Supabase, over PostgREST
Identifier allocation	`MAX(...) + 1` in a transaction	a sequence, via `next_work_item_id`
Versioned edit	`BEGIN IMMEDIATE` then `WHERE version=?`	`PATCH ?id=eq.X&version=eq.N`
Aggregates	`GROUP BY` in the query	`data_quality_rollup`, one round trip

Two boundaries are stated plainly in `docs/architecture/DATA_OPS_BACKEND.md`. Selecting `postgres` without credentials raises at startup rather than falling back, because a fall-back would leave a deployment reporting one backend and using another while the `backend` field on the wire said `sqlite`. And moving these four tables to Postgres does **not** make a second gateway process possible: the book, the bucket and the kill switch remain process-local, held by the two mechanisms of §6.1.

What the offline suite proves about the Postgres path, and what it does not

`tests/test_data_ops_postgrest.py` asserts the **request** the store builds, the `Prefer` headers, the filter grammar and the conflict target, against `httpx.MockTransport`. That is where the translation lives, and a test that only checked the parsed response would pass with `Prefer` missing entirely. The live pass skips with its reason printed unless the credentials are exported. `tests/test_data_quality_rollup.py` pins the Python `_AGGREGATE` and the SQL `data_quality_rollup` to the same six figures, because if one moves without the other both backends answer, neither errors, and they disagree.

6.4 Postgres and Supabase: the mirror and the research corpus

The order blotter mirror

```
create table public.order_blotter (
  id uuid primary key default uuid_generate_v4(),
  desk_id uuid not null default '00000000-0000-0000-0000-000000000001',
  user_id uuid references auth.users(id) on delete cascade,
  decided_by public.desk_decider not null,
  gateway_order_id text, client_order_id text,
  symbol text not null, side public.order_side not null, order_type text,
  quantity numeric(24,10), notional numeric(15,2), venue text,
  fill_price numeric(18,8), filled_notional numeric(15,2) default 0.00,
  slippage_bps numeric(10,4), fee_usd numeric(12,4), latency_ms numeric(10,3),
  verdict public.order_verdict not null,
  rejected_by public.order_verdict[] not null default '{}',
  checks jsonb, status text, strategy_tag text, source text,
  decided_at timestamptz, occurred_at timestamptz not null default now(),
  created_at timestamptz not null default now(),
  constraint unique_decider_order unique nulls not distinct (decided_by, gateway_order_id));

create trigger order_blotter_append_only
  before update or delete on public.order_blotter
  for each row execute function public.reject_blotter_mutation();
```

The trigger is the point: what the DuckDB log holds by convention, Postgres holds by constraint. `unique_decider_order` is what makes a retried mirror write idempotent rather than a duplicate fill, and `rejected_by` is an array because an order fails several gates at once. The blueprint this schema descends from hardcoded `latency_ms` at `0.19` — a fabricated measurement, removed on principle — and the column now carries the `RiskDecision.latency_ms` the gateway timed; the numeric defaults mirror `config.py`'s real limits, with `tests/test_supabase_schema.py` pinning the equality so the two cannot drift.

Delivery cannot slow an order. `modules/supabase_mirror.py` enqueues with `put_nowait` into a bounded queue, so it cannot block, cannot raise past its own frame, and on a full queue **counts the drop** rather than waiting. A mirror that can slow an order down has become load-bearing, and this one structurally cannot.

The pgvector research corpus

```
create table public.research_documents (
  id uuid primary key default gen_random_uuid(),
  desk_id uuid not null default '00000000-0000-0000-0000-000000000001',
  user_id uuid references auth.users(id) on delete cascade,
  kind public.research_doc_kind not null,
  source_ref text not null, symbol text, interval text, strategy text,
  occurred_at timestamptz not null, title text not null, body text not null,
  metrics jsonb not null default '{}', data_hash text,
  embedding extensions.vector(384), embedding_model text,
  embedding_status text not null default 'pending'
    check (embedding_status in ('pending','ready','failed')),
  created_at timestamptz not null default now(),
  constraint unique_desk_kind_ref unique (desk_id, kind, source_ref));

create index idx_research_docs_embedding
  on public.research_documents using hnsw (embedding extensions.vector_cosine_ops);
create index idx_research_docs_kind_time
  on public.research_documents (kind, occurred_at desc);
```

Three columns carry an argument each. `body` stores the exact text that was embedded, so a later change to the card renderer cannot silently invalidate every stored vector with no way to detect it. `data_hash` ties a document to the exact bars its backtest saw. And `embedding` is nullable with `embedding_status = 'pending'` on failure, **never** a zero vector: under cosine distance the zero vector is equidistant from every query and would surface as "similar" to anything asked, which is a wrong answer wearing the shape of a right one. The same rule is restated in the Oracle schema for the same reason (§6.6).

Two indexes, two access patterns: HNSW under cosine for the dense arm of retrieval, and a plain B-tree on `(kind, occurred_at desc)` for the ordinary "most recent documents of this kind" read that a vector index answers badly.

Row-level security, and realtime as a subscription filter

The corpus and the blotter ship deny-by-default: `enable row level security`, an explicit `revoke all ... from anon`, and policies only for `authenticated` keyed on `(select auth.uid()) = user_id`. A later migration adds exactly one `anon` policy, and it is narrow in three clauses at once:

```
create policy "Public demo desk blotter is readable anonymously"
  on public.order_blotter for select to anon
  using (desk_id = '00000000-0000-0000-0000-000000000001'::uuid
         and user_id is null
         and decided_by = 'gateway');
```

`user_id is null` is the clause that makes shipping a login later safe: rows mirrored by the gateway have no owner, an authenticated trader's rows do, so the policy cannot retroactively publish anyone's blotter. `decided_by = 'gateway'` keeps the labelled two-gate sandbox from ever being served as the desk's own decision. Risk limits stay closed entirely, on the grounds that publishing where the gates sit tells anyone how to size an order that passes.

The property worth naming for a protocol chapter is what happens next:

> **RLS is the subscription filter**
>
> Supabase Realtime's `postgres_changes` delivers a row only if the subscribing role could `SELECT` it. The policy above therefore **is** the subscription filter, and an anonymous client cannot subscribe its way past it. There is no second authorisation surface to keep in step with the first, which is the usual way a websocket fan-out leaks rows a REST endpoint would have refused. `replica identity full` is set even though the table is append-only and only `INSERT` is ever delivered, because a published table without it produces silently incomplete payloads the day that stops being true.

Two limits are recorded rather than hidden. RLS on the research corpus is still **bypassed** in practice, because the gateway reads with the service-role key and the writer sets no `user_id`; what shipped instead is an optional `filter_desk_id` predicate applied inside the candidate CTE **before** either ranking is taken, so a scoped rank is a rank among rows the caller was allowed rather than a rank among everybody. And the tenant scope is per desk, not per user, because one shared gateway token means there is no per-user identity to key on yet.

6.5 Neo4j as a rebuildable read model

Postgres owns `research_edges`. `modules/research_graph_projection.py` merges that derived state into Neo4j on a six-hourly sweep, and a daily sweep partitions the corpus and writes both label sets back off one read (`DEFAULT_RECONCILE_SCHEDULES =` `("reconcile:graph@every=6h", "reconcile:communities@every=1d")`). A dual write was the rejected alternative: two systems that must agree, with drift detectable only if somebody goes looking. Projection makes divergence a non-event, since a wrong graph is dropped and re-projected.

Four constraints govern the sweep, each with its reason in the file: `RECONCILE_BATCH = 200` documents per tick, because edge derivation is quadratic in what it is given and a backlog should drain across ticks rather than scan the corpus; a retained tick history of 32, since an unbounded history is a leak that grows for as long as the process stays healthy; a backoff doubling from 60 s to a 3 600 s ceiling, so an unreachable dependency is retried within the hour rather than on all of its ticks; and an idle heartbeat every 20 ticks, because silence is indistinguishable from a stopped scheduler.

Three properties keep the read model honest. A writer may not read its own output, so the sweep is forced onto the corpus path; labels written by two different sweeps refuse as "mid-rebuild", because community identifiers are comparable only within one sweep; and request-time **traversal** never touches Neo4j at all, since `/{document_id}` runs a Postgres recursive CTE. Absent Neo4j, both the sweep and the read model report a named reason and the whole suite passes. What is **not** claimed is that the algorithms run in the graph: Louvain and PageRank live in the GDS library, which the Aura Free tier does not have and CI cannot install, so the read model serves what the sweep computed rather than computing genuinely different things under one field name.

6.6 Oracle: in-database simulation

The Oracle Autonomous Database 23ai schema (`oracle/01_schema.sql`) exists for one capability the other stores do not offer: running the simulation where the data is, rather than moving rows to compute over them. Every statement is re-runnable, since Oracle has no `CREATE TABLE IF NOT EXISTS` and a schema script that cannot be run twice is one nobody dares run once.

```
CREATE TABLE strategy_research_rag (
  research_id      NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  kind             VARCHAR2(32)  NOT NULL,
  source_ref       VARCHAR2(200) NOT NULL,
  strategy_name    VARCHAR2(100), symbol VARCHAR2(32), data_hash VARCHAR2(64),
  title            VARCHAR2(300) NOT NULL,
  body             CLOB          NOT NULL,
  metrics          CLOB CHECK (metrics IS JSON),
  embedding        VECTOR(384, FLOAT32),
  embedding_status VARCHAR2(16) DEFAULT 'pending' NOT NULL,
  occurred_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
  CONSTRAINT rag_ready_has_vector_ck
    CHECK (embedding_status <> 'ready' OR embedding IS NOT NULL),
  CONSTRAINT rag_source_uk UNIQUE (kind, source_ref));
```

`VECTOR(384, FLOAT32)`, not the blueprint's 1536, because the only embedding source in the repository is the `embed-research` edge function running `gte-small` at 384 dimensions. At 1536 the index could never be populated from the existing corpus, and a query embedded by a different model returns confident, meaningless neighbours, which is a failure that looks exactly like success. `web/tests/oracle-contract.test.ts` asserts that the three numbers agree. `rag_ready_has_vector_ck` is the zero-vector rule expressed as a constraint: a row may claim `ready` only if it actually has a vector.

The HNSW index is created in its own block and is **allowed to fail**. An in-memory neighbour graph needs `VECTOR_MEMORY_SIZE`, which is not enabled by default and is not always settable on an Always Free instance. `VECTOR_DISTANCE` falls back to an exact scan, which over a corpus this size is milliseconds, so a missing index is a performance note rather than an outage. The rejected repair is explicit in the file: do not lower the target accuracy, because a wrong neighbour is worse than a slow one for evidence retrieval.

The Monte Carlo procedure, and four defects fixed

`run_monte_carlo_portfolio` simulates terminal equity under geometric Brownian motion,

$$S_T = S_0 \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}Z\right), \quad Z \sim \mathcal{N}(0,1) \tag{109}$$

and reports $\mathrm{VaR}_{99} = S_0 - Q_{0.01}(S_T)$, the mean terminal equity, and the path count the cap actually allowed. Four differences from the blueprint it descends from are load-bearing and each is recorded in the file's header: `FORALL ... PARALLEL` is not valid PL/SQL, so the generator is set-based; the `monte_carlo_runs` table it wrote to was never created anywhere; the percentile was taken over that whole table with no predicate, so two callers read each other's paths and the figure drifts further from the truth on every invocation; and it persisted 100 000 rows per call from a procedure reachable by a public serverless route. Nothing is written now, since the paths live in an inline view that disappears when the statement ends.

The path cap, `c_max_paths = 50000` with `c_min_paths = 100`, is enforced in the procedure and not only in the caller, because the route can be changed by anyone editing TypeScript and the database is the last line that decides how much CPU one anonymous request may spend. One honesty note travels with the output: this is a **terminal-value** VaR, it says nothing about the path, and it must not be presented as comparable to a maximum-drawdown figure.

6.7 API and websocket protocols

The committed REST surface

The gateway's OpenAPI schema is a contract between two separately deployed units, so it is committed rather than generated at runtime: 73 paths carrying 76 operations [tools/openapi.json, counted 2026-08-24] — 54 GET, 20 POST, 1 PATCH, 1 DELETE [tools/openapi.json] — clustered by the tag each router carries: research (15), risk and orders (14), data and data-quality (11), the Kalshi coherence lab (7), coherence (5), meta (5), audit (4), diffusion (4), coherence history (3), machine-learning research (3), telegram (3) and TCA (2). The paths are fewer than the operations because a few serve two verbs each — `/api/orders` submits and lists, `/api/data/work-items` creates and lists.

Two of those are deliberately unauthenticated, `/health` and `/metrics`; every other route resolves an actor first (`modules/api/meta.py`). The web workspace verifies the contract before it ships: `prebuild` runs `check-gateway-openapi-digest.mjs`, which canonicalises the JSON with sorted keys and compares its SHA-256 against `COMMITTED_GATEWAY_OPENAPI_SHA256` in `web/lib/gateway-openapi-digest.generated.ts`. Two independently deployed units assert their shared contract before either one of them deploys.

Server-sent events: `/api/stream/desk`

The desk's equity, drawdown and kill-switch status previously reached the browser by polling every 4 to 15 seconds against a book the gateway re-marks every second: most requests returned a number the client already had, and the ones that mattered arrived late. The stream replaces the transport, not the meaning.

```
id: 41
event: risk
data: {"equity":998214.5,"kill_switch_active":false, ...}

: ping
```

Four protocol properties, each with a stated reason in `modules/api/risk.py`:

- **Emission is on change, not on a timer.** A tick whose serialised body is identical to the last sends nothing, so an idle desk costs one heartbeat every 15 s rather than a full payload every second. The comparison is on the serialised body, which is what a client would have to diff anyway.
- **Every event carries a monotonic `seq`.** A reconnecting client can distinguish "nothing happened while I was gone" from "I missed something", and SSE's own `Last-Event-ID` carries it back automatically. A UI that cannot tell those apart will eventually show a stale number as if it were live.

- **Heartbeats are SSE comments** (`: ping`), which every `EventSource` implementation discards silently. They exist so an idle connection is not reaped by an intermediary that cannot tell a healthy quiet stream from a dead one.
- **The tick interval is clamped to `risk_monitor_interval_s`.** Polling the stream faster than the mark-to-market cadence cannot produce a newer number, only the same one more often.

A browser cannot open an `EventSource` to the gateway directly: the page is HTTPS and the gateway is plain HTTP, which no browser will mix, and the Caddy sidecar's pinned internal certificate authority does not help a browser that has no reason to trust it. The stream therefore reaches the client through a same-origin route handler, and the first attempt at that proxy was removed for a protocol reason worth recording. `EventSource` exposes neither the status code nor the body, so a deliberate 503 on a gateway-less deployment was invisible to the client and the panel read "connecting" for ever, on precisely the deployment where that is the normal condition. The proxy now answers **200 in every case** and puts the state in the first frame, where a client that cannot read status codes can still read it:

```
event: desk-state
data: {"state":"unavailable","reason":"gateway_not_configured"}
```

On `unavailable` the hook closes the source rather than letting `EventSource` retry every three seconds, which would be a poll wearing a push's clothes. The stream is also a **signal**, not a second source of the numbers: it carries `RiskState` while the book polls `/api/gateway/portfolio`, a larger and different shape, because rebuilding the panels on the stream's shape would give the same figures two sources that can disagree.

The book websocket

`/ws/book/{symbol}` pushes the consolidated ladder and a live TCA report at approximately 4 Hz for the depth visualiser:

```
{ "type": "book", "symbol": "BTCUSDT",
  "consolidated_mid": 64182.15, "venues_online": 2,
  "books": [ { "venue": "BINANCE", "bids": [[p,q]...], "asks": [[p,q]...] } ],
  "tca":   { "per_venue": [...], "consolidated": {...} } }
```

Websockets are never part of an OpenAPI document, so this shape is pinned by tests rather than by the committed schema, and `modules/api/tca.py` says so rather than leaving a reader to wonder why the contract omits it. The browser's own order book is a separate, faster path entirely: the page opens sockets directly to Binance and Bybit, receives 100 ms depth snapshots and publishes to React at 5 Hz. One hop, no backend; routing that through the gateway would make it slower rather than faster.

The server-side proxy boundary

`ALPHAENGINE_GATEWAY_URL` and `ALPHAENGINE_GATEWAY_TOKEN` are read in `web/lib/gateway.ts` and nowhere in the client bundle; 44 route handlers under `web/app/api/` are the only path from a browser to gateway credentials. The module does four things every route would otherwise repeat: resolve the base URL, attach the token, bound the wait (8 s default), and **classify** the failure.

The classification is the part that matters. A 401 from the gateway is not a 401 from this application, and relaying it straight through would make a browser prompt for the wrong credential entirely. So upstream statuses are translated into this application's own vocabulary — `gateway_not_configured`, `gateway_misconfigured`, `gateway_auth_failed`, `gateway_unreachable`, `gateway_timeout`, `gateway_rejected`, `gateway_invalid_payload` — each carrying the status this application should answer with alongside the upstream status it saw, so the caller always learns which side failed.

URL resolution is a four-state typed function rather than a nullable string, because the three ways a URL can be wrong are three different statements:

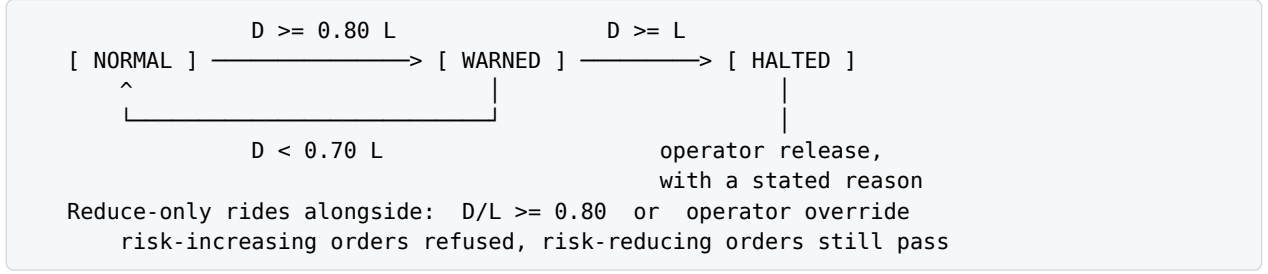
$$\text{gatewayState} : \text{Env} \to \{\text{url}(u), \text{absent}, \text{invalid}(r), \text{loopback}(r)\} \tag{110}$$

`absent` means this is a demo deployment and the honest degraded path may run; `invalid` and `loopback` mean an operator set a value that can never work and must be told so rather than have it dressed up as an outage. `loopback` is in the taxonomy because it shipped once: a serverless function fetching `127.0.0.1` fetches **itself**, and the mistake spent a day reading as a gateway outage rather than a configuration error.

6.8 Circuit breaker and failover state machines

The drawdown breaker

Let D be the daily drawdown as a fraction of start-of-day equity and $L = \text{MAX\_DAILY\_DRAWDOWN\_PCT} = 0.05$ its limit (config.py [Part2_Infrastructure/config.py]). The monitor loop runs every 1 s [config.py, risk_monitor_interval_s], reduced from 5 s because every step is arithmetic over an in-memory book, so the faster tick costs almost nothing and the breaker reacts four seconds sooner.



Three thresholds and one asymmetry. The kill switch trips at $D \geq L$ with actor `circuit-breaker` and no human in the loop. The warning is **edge-triggered with hysteresis**: it fires at $D \geq 0.8L$ and rearms only below $0.7L$, because alerting on every tick spent above 80 % of the limit was roughly 720 Telegram messages an hour at the old 5 s cadence and the tick is now 1 s. A warning that repeats every second is not a warning; it is why the next real one goes unread. With $w_t \in \{0, 1\}$ the warned state,

$$w_t = \begin{cases} 1 & \text{if } D_t \geq 0.8L \\ 0 & \text{if } D_t < 0.7L \\ w_{t-1} & \text{otherwise} \end{cases} \tag{111}$$

which is a Schmitt trigger with a 10 % of budget dead band, the same shape the venue watchdog uses.

Between warning and halt sits a graduated regime. At $D/L \geq \text{REDUCE\_ONLY\_THRESHOLD} = 0.80$, or under an operator override, the desk goes reduce-only: risk-increasing orders are refused while risk-reducing ones still pass. `config.py` records the reasoning as the practice of letting a desk close out of trouble but not deeper into it; setting the threshold to 1.0 restores the all-or-nothing behaviour.

Three details of the trip are handled explicitly. `trigger_kill` cancels the working book **before** it alerts, because the alert says "all new orders are now rejected" and with resting orders alive that sentence is false: a halt that does not reach the resting orders is not a halt. The release records **why**, since "resumed after the feed recovered" and "resumed because the desk wanted to keep trading" are the two answers a post-incident review must tell apart, and the halt's own reason is carried onto the resume as `tripped_by` so the pair reads as one incident. And the session rollover, which runs first on every tick, sits in its own exception guard: a propagating durable-write failure would skip the drawdown check, and because a failed roll deliberately leaves `session_date` on yesterday it would skip it again on every tick, for ever. Measuring against an un-rolled baseline is stale in the conservative direction, because that baseline still holds whatever the desk lost, so the breaker can only trip sooner than it should. Fail-closed on the boundary is right; fail-closed on the breaker is not.

The venue feed watchdog and synthetic failover

Classification is per venue, on a 5 s tick, and produces four states with a human reason attached (`modules/tca_engine/supervision.py`):

$$\text{state}(v) = \begin{cases} \text{down} & \text{if not connected, or connected with no book} \\ \text{stale} & \text{if every symbol's book is stale} \\ \text{degraded} & \text{if some but not all are stale} \\ \text{up} & \text{otherwise} \end{cases} \tag{112}$$

with staleness at `VENUE_STALE_AFTER_S = 10 s`. Only **transitions** are recorded, so a venue that stays down does not re-alert; and the first observation of a venue is treated as a baseline rather than an incident, which is what stops a restart from paging on startup order. Severity follows the destination state: `info` on recovery, `warning` on degraded, `critical` on stale or down, each written to `risk_events` with actor `feed-watchdog` and the previous state in the payload.

```
all venues dark for one 5 s tick
──────────────────────────────────> enable SIM (synthetic book)
[ LIVE ] <──────────────────────── [ SYNTHETIC ]
           any venue live again
```

Failover to the synthetic book engages only when **every** real venue is dark, is gated on `ALLOW_SYNTHETIC_BOOK`, and every payload it produces is tagged `synthetic: true` — including the `synthetic` column on `tca_snapshots`, so a snapshot taken during a blackout can never later be mistaken for market data. The health check is wrapped in its own guard above the failover, because observability must never be the reason the failover stops running.

The reconnect state machine underneath is exponential backoff with jitter. With base $b = 1s$ and ceiling $c = 30s$,

$$d_0 = b, \quad d_{n+1} = \min(2d_n, c), \quad \text{sleep}_n = d_n + U(0, 0.3d_n) \tag{113}$$

and a clean exit resets d to b. The jitter term is what stops every symbol's socket from reconnecting in the same instant after a shared outage.

The order rate limiter

`TokenBucket` is a classic bucket at 5 orders/s sustained with a burst of 10 [config.py] and the choice is argued rather than assumed: a fixed window lets twice the limit through across a boundary, which is precisely the pattern that triggers exchange bans. It also keeps a 256-entry deque of consume timestamps purely for observability, so `observed_rate` reports what actually happened rather than what the configuration permits. The consume **mutates**, so it must run exactly once per decision and therefore stays in Python outside the compiled core.

6.9 The telemetry planes, and what each may claim

The house rule that most shapes every surface is that a figure never appears under another plane's label. A tile that puts a nanosecond figure under a microsecond label is the defect, not a rounding choice.

Plane	Unit	What is timed	Measured
Whole risk decision	µs	`submit()` under the lock, gates to verdict	12.4 µs p50 native, 23.1 µs Python
Compiled core	ns	the C++ arithmetic battery inside `decide()`	83 ns p50 dev Mac, 320 ns production VM
Network	ms	data age in, order transit out	69.1 ms in, 72.7 ms out (Binance)

All three columns are read from `docs/architecture/LATENCY_BUDGET.md`, whose table regenerates from `tools/bench_decision.py` (5 000 orders after 500 warm-up, two venues, arm64, Python 3.12.14, median of nine runs). The production figures were read off the live `/metrics` on 2026-08-17 from a `VM.Standard3.Flex`, 2 OCPU of a shared Xeon 8358; the two machines' numbers are published side by side and never merged, because the Mac number is what the code can do and the VM number is what the deployment does.

Three instrument-design decisions follow from what these planes are for.

Never a mean. The decision histogram reports p50, p99, p99.9, p99.99 and max. The mean of a latency distribution is the one statistic that reliably hides the thing being measured.

Never a sliding window. `modules/metrics/decision_latency.py` is a log-linear histogram over the whole life of the process, at eight linear sub-buckets per power of two from 1 µs to about 1 s, or roughly 12 % bucket resolution. A 200-sample ring cannot express p99.9 at all: the 99.9th percentile of 200 samples is the maximum wearing a decimal point. Every sample is counted for ever, because "we had a 4 ms decision last night" is exactly the fact a window is designed to forget. Memory is bounded regardless of volume because counts are stored per bucket, and the record path allocates nothing: `bisect_left` over a preallocated edge list, and an increment that reuses CPython's small-integer cache. A recorder that allocates is measuring its own garbage.

Never a fabricated zero. In `modules/metrics/exposition.py`, `_num` returns `None` for an absent reading and the writer skips the line entirely rather than emitting a zero, since a fabricated zero is indistinguishable from a measured one and poisons every average downstream. The complementary rule is that a metric whose value is genuinely zero is still emitted: `alphaengine_jobs{status="failed"}` exists at 0 before the first failure, because "the kill switch metric disappeared" must not read as "the kill switch is fine". The two rules together are the whole exposition policy — **absent is absent, zero is zero**, and they are different series.

The bucket resolution is deliberately coarse for the same reason: reporting p99.9 to four significant figures out of a process sharing a core with two feed handlers would be inventing precision the measurement does not have. The core row makes this concrete. `steady_clock` on the development machine advances in 41.677 ns steps, so 83 ns **is** two ticks and 125 ns **is** three; the figure with the resolution is not the p99 but the fraction of calls inside two ticks, which nine runs at $n = 5000$ put at 0.9932 to 0.9976 [docs/architecture/LATENCY_BUDGET.md].

A second plane the same discipline governs: the research bulkhead

The re-rank stage runs in the same process as the pre-trade risk checks, and the number that bounds it was set by measurement rather than by intuition. `tools/bench_rerank.py` against real `BAAI/bge-reranker-base` weights on an 18-core arm64 machine, median of seven runs, twenty pairs:

Input	Wall clock	CPU
short rows, about 200 characters	197 ms [modules/research_stages.py]	1 776 ms
at `MAX_DOCUMENT_CHARS` = 2 000	1 523 ms [modules/research_stages.py]	12 573 ms

The CPU column divided by the wall column is the finding: `asyncio.to_thread` occupies exactly one thread, and onnxruntime's intra-op pool then spreads that single re-rank across about nine of the eighteen cores, so a bulkhead counting **workers** was bounding the wrong resource entirely. Measured directly, two simultaneous re-ranks took 307 ms against 199 ms for one on short rows and 2 122 ms against 1 456 ms at the ceiling — a second slot buys 1.30 to 1.37 times the throughput at 1.46 to 1.54 times the latency on every request, and double the CPU claim against the plane that may not wait. The semaphore is therefore **one**. A `wait_for` timeout stays rejected because `to_thread` cannot cancel the thread: a timeout at 1.5 s would release the waiting request while the abandoned one still owned nine cores.

The same discipline sets the generation budgets. Text generation is bounded at `TIMEOUT_MS = 20 000`; multimodal generation is bounded separately at `VISION_TIMEOUT_MS = 45 000`, from two live multimodal calls of 20.6 s and 29.9 s [modules/research_generate_vision.py], because a timeout that fires on healthy calls trains people to retry and doubles the spend for the same answer.

6.10 Generated-gate contracts and CI topology

The generated artefacts and their gates

Seven files in the tree are generated, and each one exists because the same fact was previously written by hand in two places and drifted. Each is paired with the gate that proves it is current.

Artefact	Generator	Gate, and what it proves
`web/lib/gateway-openapi-digest.generated.ts`	`tools/export_openapi.py`	`prebuild` re-hashes canonical `tools/openapi.json`; the web build refuses to ship against a contract it has not seen
`web/lib/test-counts.generated.ts`	`scripts/refresh-test-counts.mjs`	`check-test-counts.mjs`, run one step **outside** the suite against the runner's own summary line
`web/lib/repository-manifest.generated.json`	`generate-codebase-manifest.mjs`	`--check` in `prebuild`
`web/lib/gateway-contract.generated.ts`	`generate-gateway-client.ts`	typecheck: a route the gateway removed stops compiling
`web/lib/mc-parity-reference.generated.ts`	`generate-mc-parity.ts`	the Monte Carlo parity suite
`supabase/apply_all.generated.sql`	`tools/bundle_migrations.py`, at the repository root	`tests/test_migration_bundle.py` fails when the bundle is behind `supabase/migrations/`; the fix is to regenerate it, never to edit the SQL

`docs/architecture/latency-bench.generated.json`	`tools/bench_decision.py`	regenerated, never edited; the block stamps its own UTC date

The test count is the one figure in the repository that **cannot** be asserted from inside the thing it measures, since a test that checks the total changes the total. It is therefore generated, checked from outside, and is a measurement with a date rather than a contract. As of 2026-08-24 [web/lib/test-counts.generated.ts] the committed figures are gateway 2 998 collected, 2 996 passed, 2 skipped [web/lib/test-counts.generated.ts, refreshed in the CI shape with RERANK_TEST_MODEL_PATH blanked], web 4 487 tests across 984 suites [web/lib/test-counts.generated.ts] and service 24 [web/lib/test-counts.generated.ts]. The two gateway skips are the two opt-ins described below, and the shape belongs with the figure: seed the re-ranker weights and the same tree collects eight more tests and reports one skip instead of two, so a gateway count with no shape stated is not yet a measurement. Prose elsewhere in the tree still quotes earlier figures, which is exactly what this file exists to prevent: never quote a count from a document, including this one. Run the suite, or read the generated file.

A second contract of the same kind pins the two implementations of the pre-trade gates against each other. `web/tests/fixtures/gate-parity.json` holds 20 bit-exact scenarios [web/tests/fixtures/gate-parity.json] at `version: 1` — among them `concentration_breach`, `gross_breach`, `price_band`, `slippage_breach`, `slippage_partial`, `rate_limited`, `kill_switch_on`, `symbol_halted`, `working_book_full`, `duplicate_client_id`, the two paper-equity paths and the two reduce-only directions. Each carries the books, the token-bucket state (`tokens`, `rate`, `burst`), a frozen clock, and the **entire** expected check vector with every gate's `limit`, `observed` and `passed`. Freezing the clock and the bucket is what makes it reproducible: a rate-limit scenario whose bucket refilled between runs would be a flaky test dressed as a parity test. For the C++ core the tolerance is zero, and the fixture is what caught two silent-wrongness bugs — CPython's compensated `sum()` against a plain C++ fold, and FMA contraction fusing a multiply-add one unit in the last place off even under `-ffp-contract=off` until pinned with `#pragma STDC FP_CONTRACT OFF`.

Six workflows, and what each is allowed to prove

Workflow	Trigger	What it proves
`ci.yml`	push, PR, dispatch	the code is correct, offline
`deploy.yml`	push to main	the gateway image builds, ships, swaps and verifies, or rolls back
`e2e.yml`	dispatch, twice daily	everything deployed as it is right now still works together
`schema.yml`	dispatch only	committed DDL applied to Oracle and Supabase
`oracle-keepalive.yml`	schedule	an Always Free database stops itself after 7 idle days and is reclaimed after about 90; only a session resets that timer
`openbb-keepalive.yml`	schedule	a Vercel function stays warm 5 to 15 minutes, and a cold OpenBB import lands in the desk's upstream tail as a genuine multi-second sample

The separation between the first three is the argument. `ci.yml` is network-free by design so that **a red build means the code broke, never that a service was slow**. `e2e.yml` is the opposite job and contacts the live gateway, the live Vercel deployment and the live databases, answering the one question no offline suite can. It is manual and scheduled and never on push, because a venue outage or an idle database is not a reason to block a code change. The justification for its existence is empirical: every deployment failure this repository has actually had passed the full offline suite first — a directory excluded from the upload, an image that pushed but never swapped, a database that requires a wallet, an environment file truncated to five variables. `schema.yml` is manual for a different reason: DDL that rides a code deploy is how a table gets altered by someone who was shipping a stylesheet change.

`ci.yml` runs four always-on jobs. **Gateway**: build the native extension (a missing core is a red build, never a silent fall-back to Python), `ruff` over the whole tree, `pytest`, `export_openapi.py --check`, and `tools/synthetic_probe.py` as a smoke over the money path from book to cost to risk gate to audit. **OpenBB service**: its own `pytest`. **Web**: `npm ci`, the suite piped through `tee` under `set -o pipefail` — mandatory, because GitHub's default bash does not set it and `tee` alone would mask a red suite behind its own exit zero — then the committed-count check, `tsc --noEmit`, and a production build, because a type-safe component can still fail during route analysis, metadata generation or bundling and CI must exercise

the artefact Vercel will deploy. **Repository audit**: `check_repo_complete.sh`, which exports `HEAD` with `git archive` to catch a file that builds locally only because a `.gitignore` pattern silently matched it.

Network-free by construction

Offline is arranged, not hoped for. `tests/conftest.py` sets the environment **before** `config` is imported, precisely so it wins over a local `.env`, since python-dotenv does not override a variable that already exists. The interesting part is that two different mechanisms are used and they encode two different policies.

Mechanism	Variables	Policy
`os.environ.setdefault`	`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `NEO4J_URI`, `NEO4J_PASSWORD`, `DB_PATH`, `ENABLE_MARKET_DATA`	consent: an exported variable survives, so a documented opt-in stays possible
`os.environ[...] = ""`	`GEMINI_API_KEY`, `RERANK_MODEL_PATH`	refusal: an exported variable is overwritten, so nothing can leak in

The difference is the whole claim. `setdefault` only wins over a `.env`; an **exported** variable is already in `os.environ` and reaches `settings` untouched. `tests/test_research_answer.py` drives the real `/api/research/rag/ask` route and patches neither the settings nor the SDK — it relies on that one assignment — so under `setdefault` a shell exporting a real key would spend a live model call per test while the file said it could not happen. The conftest records that this was **measured**, not deduced. `RERANK_MODEL_PATH` is assigned for the second half of the rule, so a seeded fastembed cache cannot make the "no model downloaded" tests load about 110 million parameters off a directory nobody mentioned.

The two deliberate opt-ins, and what they name when they skip

The skip line is a report, not noise. Each of the two suites that can skip states in full what it did **not** exercise.

1. **Live Postgres.** `tests/test_data_ops_postgrest.py::TestAgainstTheRealProject` skips unless `SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` are exported. Its message says CI is network-free by design, so a green suite there has **not** reached Postgres. `live-smoke` in `ci.yml` is the dispatch-only counterpart: it sends a `HEAD` to PostgREST and treats 401 or 403 as the pass, because deny-by-default RLS means anon can read nothing and **a 200 there is the bug**.
2. **Seeded re-ranker weights.** `tests/test_research_rerank_real.py` skips unless `RERANK_TEST_MODEL_PATH` is set, naming that no cross-encoder weights were offered and the real ONNX path was not exercised. The `rerank-real` job is its counterpart, and is a separate job rather than a step in `gateway` for a measured reason: `BAAI/bge-reranker-base`'s fp32 ONNX blob is 1 112 459 588 bytes, about 22 s to fetch cold [.github/workflows/ci.yml], and hanging a gigabyte off the job that gates every push would slow the default red-green for everyone in exchange for an extra a normal deployment does not configure.

Two assertions inside that job are worth transcribing, because they are what stops an opt-in from proving nothing. The suite runs with `HF_HUB_OFFLINE=1`, which is the assertion rather than a precaution: "no network at request time once the model directory is seeded" is the entire argument for choosing local ONNX over a hosted re-rank API, and running with the hub switched off is what turns that sentence into a test. And the job greps its own output for `skipped` and fails if it finds one, because a suite that skipped would leave the job green having proved nothing. **Count the skips, not the passes.** A mirrored step then runs the same files with the opt-in unset on a runner where the weights are sitting on disk and asserts that exactly one skip occurs, which is how the default suite is held weight-free.

A missing native core, by contrast, is a **failure** and never a skip: `tests/test_decision_core_native.py` treats an unimportable `modules/_decision_core` as a red build unless `DECISION_CORE=python` was set deliberately, because a quiet fall-back to the Python reference is exactly what CI exists to catch. The deploy job applies the same rule to the running container, refusing to keep one that fell back when native was built for it.

What this chapter does not claim

Three absences are load-bearing and are named rather than glossed. There is no hardware timestamping and no PTP source on a cloud VM, so every latency figure here is in-process: it excludes the kernel network stack, the driver and the wire, and is a **floor** on the real latency rather than the real latency. The gateway remains single-process, and moving the data-operations tables to Postgres does not change that. And a

green CI run has, by construction, not reached Supabase, Neo4j, Oracle, a venue or a model provider — which is the property that makes it trustworthy as a statement about the code, and useless as a statement about the deployment. `e2e.yml` exists because those are different questions.